



Object-oriented Programming in C# 14

Understanding the foundations of OO programming in C#

Module introduction

Hi there, and welcome to this course on Object-oriented Programming in C# 14. My name is Gill Gleeren, and I will be your guide throughout this course. Now, if you already know some C# syntax, that is perfect because that's exactly where this course starts. We're not going to start from zero here, but at the same time we're also not going to stay at the level of syntax and keywords and class declarations only. What I really want to do in this course is move from knowing the OO building blocks in C# to actually using them to build better software. So yes, this is a course about object-oriented programming in C#, but even more than that, it is a course about OO design. Let me show you right away where we are going with this course. We're going to start from business problems, not from theory in isolation. Then we will build a first object model. And from there, we'll gradually move into building the right objects, improve weak designs, and reshape that model so that it stays maintainable as it evolves. So the journey through this course is really this one, from OO syntax to OO design. Now before we dive in, let me be clear about what I expect you already know. You should already be comfortable with the basic C# syntax, and you should already have seen classes and objects and the first OO principles before. If those things still feel completely new, then this course will move a bit too fast for you. And if you want to refresh that foundation first, then I have a few other courses in the library that can definitely help you with this. If you're starting with C#, first look in the library for the C# Foundations course. That is followed by the C# Syntax and Structure course, which covers, as the name gives away, the syntax in more detail. Then I have the introduction to Object-oriented Programming in C# course, which really shows you the syntax and core knowledge around OO. As you can tell, those courses are much more focused on the syntax and the first OO IDs. In this course, we're going to build on top of that and take things into a more design-oriented direction. Throughout the course, we are going to work in a small, but realistic domain, and that is the one from Bethany's Pie Shop again. It is the same idea as we also used in the previous courses, so not to overcomplicate things. So we will have things like pies, products, customers, orders, discounts, and gradually more and more design decisions around these concepts. And I like using one domain throughout a course like this because it allows us to not just see, say, isolated examples, but a model that will grow over time. So this first module is really a short alignment module. We are not starting from scratch and we're not going to reteach the entire OO introduction, either. What I want to do



here is make sure we are looking at OO in the same way before we start modeling more seriously. This module is not about learning OO syntax again. That is a set covered in the earlier courses. Instead, it is really about making sure we think about object-oriented programming in the same way before we move into the rest of the course. That way, when we start improving models later on, we are all starting from the same understanding. And I think that is an important distinction to make early. I will probably say this a couple of times again. This course is taking you further. There's a big difference between knowing OO concepts and using OO well. You can absolutely write code with classes and inheritance and methods and still end up with a weak design. So from the start, I want us to aim a bit higher than just code that compiles or code that uses OO features, since after hearing about classes and objects and inheritance, that might be the message you got. And honestly, I don't blame you for understanding it this way. Often, you start with understanding things around OO and then you move to modeling using OO, and that is what we will do here. Along the way, the question is going to become is this actually a good model for the problem since we are building code that holds up over time, or I should say, we should at least try to do this.

Understanding OOP in C#

So let's begin with a very short recap of object-oriented programming, or OOP for short, in C#. You have already seen the common OO ingredients in C# in the previous courses. There are, of course, classes and objects, methods, properties, inheritance, polymorphism, and interfaces. Those are the building blocks that the language gives us, and of course they matter a lot. We are going to use all of them throughout this course, but like I said, knowing the list is not the same as understanding what these tools are really helping us to do in the real world, that is. Because, as mentioned, for many developers, OO can start to feel like a checklist. It's often taught that way, and with this course, I'm going to change that. Often, you hear create classes, move some data into them, add a few methods, use inheritance where types seem related, and yes, that is all still valid, but it is still a fairly shallow way to look at OO. OO is not only a set of language features supported by C#. It is not just creating classes. It is not just moving coding files or adding inheritance because we can. That is the syntax side of it. Useful, definitely, but not yet the full value of object-oriented programming. OO becomes much more interesting when we use it for modeling. That means we start thinking about concepts, assigning behavior, structuring collaboration, and making responsibilities clearer. And that to me is where OO really starts becoming useful. You may be thinking, uh, Gill, I don't understand what these things mean, right? Well, don't worry, that it will all become clear, and it all starts with a very simple idea. And that is objects should represent meaningful concepts. A customer is not just some random thing holding a couple of values. A product is not just a record. And an order is not just a bag of numbers, either. Those are actual concepts in the business domain, and



OO gives us a way to express these concepts directly in code. Take a look at this small snippet here. As we can see in the code, we first create a customer, then an order for that customer, and after that we add products to that order. So even in a very small example we already have more than just data. We have actual objects representing concepts, and those objects are already working together. And that is why these examples matter. They're small, but they already show us the direction that we'll take in this course. Another important idea here is that objects are not just data holders, which is also something that you might be expecting them to be at this point in your learning. Instead, they actually combine state and behavior. They hold information, but yes, they should also do useful work, ideally close to the data they depend on. That is one of the reasons OO can make software easier to reason about, and behavior lives close to the relevant state, so data, the model, tends to be much cleaner. At the same time, no single object should try to do everything; again, something that might not be clear at this stage. Each object should have a clear role. So that brings us to the question, where do we, let's say, cut into a separate object? A system works well when those objects collaborate well. So OO is really about cooperation between parts, not just building one huge class that knows and does everything. So, if you remember only one line from this part, let it be this one: An object is useful not just because it exists, but because it has a responsibility. It's a simple sentence, but it's going to come back again and again throughout the rest of the course.

Working with classes and objects

With that in mind, let's now look again at working with classes and objects in everyday code. Consider this a short recap of the things we'll need to advance in this course. So, technically, you should already know these. I'm just including them here for completeness' sake. The distinction is basic, but it's still worth making it very explicit, and we have talked about it a lot in the introduction courses. Classes define structure and behavior, like the blueprint of a house made by an architect. Objects are the concrete instances created from these classes. So a class gives us the definition, but it is the objects that our application is actually working with at runtime. Here, for example, we have the Product class on screen, and what I want you to notice here is that this class defines the shape of a product in our code as we see in the snippet. It has a name, it has a unit price, and it has a constructor that lets us create a valid product with those values. Now, we'll come back to the valid part later in this course. So, this is the class side of the story. It tells us what a product is supposed to look like in code. Now, applications are not built out of class definitions alone. In fact, far from it. We ran the class once, but then we create objects from it. That's why I like this line here. It says classes define, objects exist. It's simple, but it's exactly right. Now if you look at this snippet here, we create two separate product objects. That's the important thing to notice here. It's the same class, same structure, but two separate instances. One represents the cherry pie and one



represents the apple pie, both real things, in fact. So it is the practical meaning of objects as instances. Now we're not talking theory here. We're literally creating multiple objects of the same class. And once we have the objects, we can start connecting them, as we see here in the code. We create a Customer, create an Order linked to that customer, and then add those product instances to the order. So, the objects are no longer isolated. They are related. And that is one of the nice things about OO when it is done well. The structure of the code starts to resemble the structure of the actual domain. And I will explain what I really mean with domain, don't worry. Take a look at this snippet theory. One object is asked to calculate the subtotal, and then a pricing policy is asked to apply pricing behavior to that subtotal. So the result here comes from multiple objects, each doing their part. This is a useful way to think about OO. Objects are not just stored and passed around; they are asked to do work. Let's make this a bit more concrete and go into the code. What I want to show you here in this demo is how these classes turn into actual objects, how those objects get connected, and how they already start working together in a very small example. I've opened the starter solution. You can find that with the downloads of this course. And it really does not contain any code just yet, but it does contain two projects, a domain project and a console project, and that in turn contains the programs, yes. There is also a reference between the projects so that the console project knows about the domain project, as we can see here. I will start in the domain project because that is where our concepts will live. I'm going to start by adding a Customer class inside a folder called Customers, and let me do that quickly here. I'm going to paste in the snippets. You can find all the snippets, of course, also with the downloads of this course. What we see here is a very small domain concept. For now, the customer just has a FullName and a flag indicating whether this customer is a loyalty member. That is enough for this first example. Next, I will add a Product class in the products folder. Again, what we see here is a simple concept, but I think already an important one. A Product has a Name and a UnitPrice. Again, that's enough for now. And now that we have a Customer and a Product, I'm also going to bring in an OrderLine, and that we'll add in the Orders folder. What I want you to notice here is that an OrderLine really connects a Product with a Quantity. So already we are moving beyond just storing loose values. We are shaping a little bit of a model. And since the Product class is inside a separate namespace, I also needed to bring that in, but Visual Studio does that now automatically. On that OrderLine, what we see is also a small method to calculate the line total. It's already a nice early example of behavior living close to the data that it depends on. The next, of course, is Order. Let me add that here in the same folder, too. This one is, well, already a bit more interesting because this is where things are starting to come together. What we see here is that an Order belongs to a Customer, and it keeps track of its own OrderLines. Since it again uses classes from a different namespace, I've brought in the correct using statements, too. Notice also that we're not building OrderLines all over the place in outside code. Instead, this Order already exposes an AddProject method, and it takes care of creating the OrderLine itself. That already gives the class a more



meaningful role. Then further down here, what we see is the `CalculateSubtotal` method. This is also a nice object-oriented moment already. We're asking the order to calculate something meaningful instead of doing that logic in some other place unrelated, that is. So these are some classes that already contain some basic data and behavior, and really this hasn't introduced anything new so far. Now let's switch over to the `Programs.cs`, which is in that console project. I'm going to paste in another snippet here. What you see here now is the object side of the story. First, we create a `Customer` object, then we create an `Order` for that `Customer`, and after that we create a couple of `Products` and we add them to the `Order`. We now have actual objects, and those objects are connected in a way that starts to resemble the real domain. Then I ask the `Order` here to calculate the subtotal and we write that to the console. So, in this small example, we already see the core OO ideas to appear. We have connected objects and we have behavior inside the model, and we have, well, meaningful concepts, meaningful classes. Let's run this quickly, and we can see the output. The subtotal is calculated based on the products we have added to the order.

The pillars of OO

In this next part, let us briefly revisit the classic pillars of OO. You have probably seen these before, but let's quickly line on them again. Encapsulation is about controlling access to state. So, often that is done using private fields and properties. Next is inheritance, which is about base and derived classes. Then, we have polymorphism, and that is about being able to use more specific objects to be a more general type. And abstraction is about focusing on the essential concept while hiding the unnecessary detail. So yes, they are important, and we will use all of them throughout this course. But again, just knowing the names is not really the goal here. What I want to focus on is understanding what these ideas help us to do in actual code and in actual models. And let's start with encapsulation. Encapsulation is about controlling access to an object's internal state. It helps protect data from careless changes, often from the outside of the class, and it keeps out the logic that belongs to the state close to the object itself. Let me do that. In other words, an object should not just expose everything and hope that the rest of the system behaves nicely. In practice, so far we have applied encapsulation by using private fields to encapsulate the data, and we wrap that with properties to control access to the data. Now that is okay, but there is more we can actually do. Here is a very basic encapsulation example. What I want you to notice here in the code is simply that there is an internal state that is the age and controlled access to that state. This is a small example, of course, but it already shows the idea that an object gets some say in how its own data is handled. So even here encapsulation is already about protection and control. Now, in this course examples, encapsulation becomes more meaningful. If you look at this order snippet, the interesting part is that the internal collection here lives inside the order, and the outside code does not just



take over that collection directly. Instead, the Order class exposes a read-only view. And if someone wants to add something, they have to go through the AddProduct method so the order stays in control. That is a much more practical, more real-world example of encapsulation. Again, don't worry if this is not fully clear yet. We'll come to it in a lot of detail. Next, let us recap inheritance. Inheritance creates a relationship between types. The idea is in a more specific type and builds on a more general one. So instead of starting completely from scratch, we can take that more general concept as a base and then create a more specialized version of it. That's also why we talk about base and derived classes. The base class gives us the more general concept, and the derived ones build on it. Here we have the most basic syntax example of that idea. As we see here in the code, Manager derives from Employee. In C# syntax, inheritance, so deriving, is indicated using the colon and then the name of the base class. So this is our basic base and derived class relationship in C#. Typically that derived class will also contain a more specialized behavior. We'll talk in a lot more detail about this later in the course, and be modeled through the use of inheritance. Now in this example, inheritance is again used in a more meaningful way. If you look at this snippet, the important things to notice are the relationship between a LoyaltyDiscountPolicy and DiscountPolicy, and also the fact that behavior is being specialized. The override is the interesting bit here. So this is not just structural reuse. This is a more specific type providing a more specific behavior. That is where inheritance starts becoming useful. Next, I want to take a quick look at polymorphism. This is one of those concepts that often sounds more complicated than it really is. The key idea is actually pretty simple. We can work with a more specific object through a more general type, and because of that, the same method can lead to different behavior. What really decides the behavior is the actual type of object behind the scenes at runtime. And in C#, this is typically where virtual and override come into play. They allow us to define shared behavior in a base type and then provide a more specific version in that derived type. Here's the basic example again. What I want you to notice is that both types expose the same operation, but the derived type provides a different implementation. So the setup here is really about a variation in behavior. The base class gives us the shared concept, and the derived class gives us the specialized behavior. That is what makes polymorphism possible. In this snippet, the variable is typed as the base type, but the object behind it is actually the more specific type, and when the method is now called, the more specific behavior is used. Now look at this example. The highlighted line is the most important part. We are always making the same method called via policy, so the Apply method, but the actual behavior depends on which policy object was passed in. So if this is a loyalty policy, that behavior is used. And if this is a no discount policy, well then, the other behavior is used. That is a nice practical example of polymorphism. Then we have abstraction. Abstraction is about focusing on what matters and hiding the unnecessary detail. It gives us the right level to think and work at. Here, for example, what matters is that this type exposes a meaningful operation, in this case, CalculatePay. So the cause can



work with the concept at the right level without needing to manage all the little details separately. That is already abstraction in action. Now that we have revisited the four pillars again, let's make these a bit more concrete and look at them together in a small example. I must start here in the Order class because this is where we can immediately see encapsulation. What we see here is that the internal collection lives inside the order. Outside code does not just take over that list directly. Instead, the order exposes a read-only view. And if something needs to be added to that list, then it goes through the AddProduct method. So this is a practical example already of encapsulation. The object, the order, stays in control of its internal state. Now let's bring in a small pricing hierarchy. I must start with the DiscountPolicy, which I will add inside the Pricing folder in the domain product, of course. What I've added here is a shared pricing concept. At this level, the policy just returns the subtotal unchanged, but the more important thing is that we now have an abstraction for this kind of behavior. So already here we are working at the level of a pricing concept, but not at the level of every little pricing detail. Next, I will add the NoDiscountPolicy. This one is a very simple concrete version of that shared concept. It does not change the behavior just yet, but it's actually the same code as we can see. But it does give us a concrete policy type. Next, I will add in the LoyaltyDiscountPolicy. Now this is where inheritance becomes clearly visible. What we see here is that a more specific type is building, let's say on top of the more general ones with a discount policy. The override is the interesting part here. This more specific policy provides, well, more specific behavior. The inheritance is being used for specialization. Now I want to go back to the Program.cs and use those policies. I've again pasted in a snippet that we can go through. What I want you to notice here is that the helper method works with the base DiscountPolicy. So the calling code does not need to know every detail of every pricing variation. Then comes the polymorphism part. What we see here is that we actually call the same helper method twice, but we pass in different policy objects. The call stays the same, but the behavior can change depending on the actual type of policy object that is being passed in. We have, in fact, in this small example already touched upon the four pillars. To recap, we've seen encapsulation in order, inheritance in the pricing hierarchy, polymorphism in the stable call with different policies, and abstraction really in the idea of the discount policy itself. The OO concepts are really helping us to shape our code more clearly. Let's quickly run this to make sure that everything works correctly. What you see in the output is the standard pricing and loyalty pricing go through the same overall flow, but the result changes depending on the policy object that was being used.

What these concepts enable and what they don't

With the refresher behind us now comes what I believe the more interesting part of this course. Let's dive in. OO gives us a number of things automatically, or I should say more naturally. It gives us a way to work with meaningful objects. It



helps us think in terms of responsibilities. It helps us structure collaboration between parts, and it helps us organize code around concepts instead of just loose functions and data. There are definitely more things here, so all of that is real advantages. OO absolutely gives us strong building blocks, but, and this is the important contrast, OO does not automatically give us a strong model. It does not automatically give us good boundaries. It does not automatically put behavior in the right place. It does not automatically make object creation safe. And it definitely does not automatically give us a maintainable design. That might be the worst, in fact. That part still depends on how we use the OO tools that we have. So that's really the transition I want to make in this course, not just asking are we using OO, but instead asking are we using it well enough to create a strong model? A class can still have the wrong responsibility. An object can still expose too much state. Behavior can still live in the wrong place. Inheritance can still be used poorly, so code can be perfectly valid OO code and still be awkward to understand or evolve. And it's one of the most important realizations to have at this point. We might think we're there. Well, we definitely aren't. Now, don't let that be a setback. This course will get you there. Look at this snippet again. Technically, as we have seen here in the code, there is nothing wrong with it from a language point of view. We have objects, we have method calls, and we have collaboration. We even have variation through the policy. So yes, OO, concepts are definitely present, but, and this is the more interesting question, is this actually the right model for the problem? Is the behavior in the right place? Are the responsibilities set correctly? Will this design still make sense when the business changes? Those are the questions we'll keep on asking. So good design is not about using more OO features, it's about using them deliberately in the correct place. That should be the message that you get from this introduction here.

Preparing for design-focused modeling

So from here on we are going to become much more design-focused. The OO recap is now complete, and that was mostly a syntax thing, as we have seen. From here on we're going to start with OO design. That will start from business problems and we will use OO as a way to create models that behave well and evolve well. So the foundation is there; now we're going to build on top of it. So the question is no longer, "Can I write this in C#?" It becomes, "Is this a good model for the problem?" That's the question we're going to keep answering throughout the rest of this course.

From a business case to an object-oriented model



Module introduction

Hi there, and welcome to this next module where we can start doing what I think is the more interesting part, taking a business case and turning it into a first object-oriented model. Up to now, the focus was on OO concepts themselves. We've refreshed the, let's say, theory about classes and objects. We've seen a small sample already being used, and then we saw how the four pillars of OO are applied, let's say, on a more language level. Now we are going to apply those ideas to a concrete business scenario. The focus now becomes building a first model around an actual problem. And it's an important change because from this point on, OO is no longer just a set of language concepts. It starts becoming a way to shape software around business meaning. And while that may seem a bit abstract, pun intended, that is actually what you'll be doing all the time when designing OO-based systems. Now, there is one thing I want to make very clear before we start. A good first model is allowed to be imperfect. In fact, it almost certainly will be. We are deliberately building something that is understandable and working. It does not need to be elegant yet. It does not need to be refined yet. And it is actually useful if this first model leaves room for improvement, because that's what the later modules will do. Otherwise, we would have a two-module course, right? So if you see something in this module and think, hmm, that feels a bit weak, then it's fine. That's part of the plan, really. We'll discuss the different improvements over the next modules. And so that is the loop we are going to follow throughout this course, which I think really reflects how we work also in the real world, designing a real system. We start with the business problem, then we build a first model, then we identify the weaknesses, and then we improve the design, and we repeat that over time. So what you see on this slide is not only the structure of this module, it is really also the structure of this course as a whole.

Understanding the business problem

So, let's begin at the right place, the business problem itself. The scenario for this module is still small, and that's a good thing. Bethany's Pie Shop needs a simple checkout flow. A customer places an order, and that order will contain one or more pies. The order can either be picked up or shipped. And at the end, we need to calculate the total. So the business story is not complicated yet, and it already has enough moving parts to give us something meaningful to model. If we look at what actually has to happen in that scenario, we can break it down into a handful of business actions. A customer has to be chosen. Then, products have to be selected. Quantities have to be added. A delivery method has to be chosen. And then the total has to be calculated. So, I really think that this overview is good because it turns a business story into something more operational. We are starting to see the flow our code will need to support. Now from that same



scenario, we can also begin identifying the main things involved. Imagine me doing air quotes when I say the word "things." There will be a customer, there will be products, there will be an order with order lines, and there will be delivery details, and there is pricing. That is typically how a first model starts to emerge. We listen to the language of the business and we start recognizing which things deserve to become concepts in our code. As developers, we are really translators of the business language into a code model. That's what we're doing here. And we can actually already go one step further and start spotting responsibilities for the identified "things," again, with air quotes. The customer will be placing an order. A product will provide name, code, and price. An order groups what the customer wants to buy. An order line connects a product to a quantity. And pricing logic calculates the subtotal, shipping, and total. So even before we had a single class, we are already doing something very OO here. We're not just collecting nouns. We are, in fact, starting to think about what each concept is responsible for. If the business keeps talking about customers, products, orders, and shipping, then our first model should probably start there, too. We do not need to begin with clever abstractions. We do not need to begin with patterns. We need to begin with the concepts that are clearly present in the model.

Identifying the core classes

So, let's identify the core classes in some more detail and make this concrete. Like I said, a first object model usually begins with the language of the business. We heard about customers, products, orders, shipping, and totals. So a very natural first move is to turn those business concepts into classes. Not because every noun automatically becomes a class. That would be too simple, definitely for more extended business domains. But because, in this case, these concepts clearly are central to the scenario we're trying to support. So, let's do that. Here are the core business concepts for this first version of the model. We have a Customer. That one will be placing the order. Then we'll have a Product, so that's a general name for something that Bethany's Pie Shop is selling. We also have the Order, which represents basically a purchase in progress. And then we'll have the OrderLine, part of the order, referring to a product with a certain quantity. What we also will have is then supporting logic in the form of an OrderPricingService, which then calculates the price for the order. That one is, in fact, worth noticing already. It tells us that not everything in the model needs to be a thing, like Customer or Product. Sometimes you also have supporting logic that serves that scenario. Keep that in mind for now. If we zoom in a little bit on the concepts, what we see is that Customer holds customer-related information and that Product holds product-related information. I've listed out the data that naturally belongs to each concept, and as we see, that looks like basic data, like a string for first name, and so on. I've done the same for Order which, as we can see, contains maybe some more complex items. It also links to Customer, contains the lines, and



contains also delivery-related details. And an OrderLine connects a Product to a Quantity. So what we see here is that the model is already starting to reflect not just isolated data, but the relationships inside the checkout scenario. Here we see a sample Customer class. What I want you to notice here is that it is not clever yet. It does not need to be, in fact. The point is that it reflects the business scenario and gives us a concrete starting point. That's exactly what a first model should be doing. It should help us represent the scenario clearly, even if we know that we're going to refine it later. And here we see part of the pricing logic in this first model. What we see here is a separate OrderPricingService class with methods to calculate the subtotal, the shipping cost, and the total. So for now, pricing behavior lives outside of the order itself. Keep that in mind for now. That is a design choice in this first version, but it's also very clearly something we are going to revisit later in the course. Now, one important thing here, not everything should go in one class. An order is not the same as a product, and a product is not the same thing as a customer. And even in a first model, things already get stronger when responsibilities are at least somewhat separated. So we're not going for perfection yet, but we're also not going to throw everything into one giant pipe and hope for the best. You'll see those boundaries appear. We'll talk about them later. So for now, not one big class. That's really the way forward.

Creating a first object-oriented model

So, now we are ready to create the first object-oriented model. In fact, we've already seen parts of it. And yes, I'm emphasizing the first here. As said, first means good enough for now. We want clear class names, understandable object relationships, correct behavior, and a working checkout scenario. What we do not yet need is perfect encapsulation, ideal responsibility placement, or refined modeling decisions. This model is not supposed to be the end result; it's supposed to be the first working step. So, what is the flow we actually want to support? We want to create products, create a customer, create an order, add order lines, calculate prices, and print a summary. That is the end-to-end part we want the model to support, and this is useful because it gives us something very concrete to test our design against. If the model can support this flow clearly, then it's already doing something useful. Let us already look at this briefly in code. We will see more details in the upcoming demo. You can see that a Customer is created and that an Order is created and linked to that Customer. So the code mirrors the business flow very directly. The code should already read in a way that feels close to the business story. Then we add products to the order. What matters here is that each OrderLine connects a Product to a Quantity. So, the structure of the model still makes sense at the level of the business. The order now contains the products that the customer wants to buy. That is a perfectly reasonable first design step. And now we calculate the pricing. As said, the pricing behavior is currently handled outside the order itself. I'm creating a pricing service and then ask you to calculate the subtotal, the shipping cost, and the total. So the scenario



works end-to-end, but the pricing choice is very visible. Pricing is external for now. That's exactly what I want us to notice at this point. This is a normal approach for solving this. So, let's make this a bit more concrete and go into the code. In the previous module, we used small examples to align on the OO concepts themselves. What we are doing now will be different. We're starting the actual model creation for the rest of this course, and for now, yes, we'll redo most of what we have done already, but what I want to show you in this demo is how the first version comes together from end-to-end. I have opened the starter solution again, which again contains really no code yet, but it does contain again the two projects we've had before, the console project and the domain project, that is. There is again a reference between the two so that the console project can use types defined in the domain project. Again, we'll start in domain project. That is where the model will be created. So again, let me bring in the Customer class in the Customers folder. What we see here is a very straightforward first version of the customer, slightly different from what we had in the previous module. That is, of course, intentional. We're now bringing in what we mentioned in the slides. It just contains the information that belongs to the customer concept at this stage, and that is FirstName, LastName, and Email. I'll do the same with the Product class in the Products folder. Again, what we see here is a very simple first model. A product has a Name, a ProductCode, a UnitPrice, and an indication whether or not it is active. So at this point, this feels still very data-oriented. That is exactly what I want to show you. That is typical for a first version. Now that we have a Customer in the Product, I'm going to bring in the OrderLine in the Orders folder. Notice here again that an OrderLine connects a Product with a Quantity. So we're moving again beyond isolated classes. We're starting to express a relationship that exists in the business scenario itself. And because this uses the product, we had to bring in that correct namespace again. I'll do the same with Order in the same folder. And as before, this is where things are starting to come together. What we see here again is that an Order links to a Customer, contains all the lines, and it also carries delivery-related details. Now, one other thing that should be striking you at this point, this class is very open. Everything is publicly acceptable. The collection is also exposed directly. Now, we're not going to fix that here yet, but I want you to notice it already because this is exactly the sort of thing we are going to improve on later in the course. Next, we'll bring in the pricing logic again in the Pricing folder, now through an OrderPricingService. What we see here in general is that pricing behavior is now currently handled outside of the Order class itself. It gives us a way to get the scenario working, but it is also a very visible design choice, and as said on the slides, this is something we'll need to come back later on. Let me quickly take you through the code. We see a few methods here. This one here, so the CalculateTotal, gets in an order for which it needs to calculate the total price as the parameter. In these two lines, it is then calling the two other methods also part of this class, so the CalculateSubtotal and the CalculateShippingCost, and both of these accept the same order as the parameter. So really what we see here is that the total is built up in two steps. First, we calculate the subtotal of the



order, then we calculate the shipping cost for that same order. And then at the end we simply add these two together to get the final total. If we quickly look at the `CalculateSubtotal`, you'll notice that I'm using something new here called `Sum`. If you haven't seen that syntax before, don't worry too much about the exact details. The important thing to understand here is that we are going over all the order lines and adding up their totals. For each order line, that is the unit price of the product multiplied by the quantity for that line. You could write this using a loop as well, but this approach is cleaner. This is using a LINQ syntax. There is a dedicated course on LINQ in this C# path. Then in the `CalculateShippingCost`, we first check whether the delivery method is `Pickup`. If it is, then shipping is simply 0, which makes sense, of course. If it isn't `Pickup`, then we again use `Sum`, but this time for something slightly different. Here what we are doing is adding up the quantities of all the lines, so we know how many items are part of the order in total. Once I have that number, then we apply a very simple shipping formula, a fixed amount plus a small extra amount per item. So again, nothing very advanced from a business point of view, but enough to support the scenario that we want to model. So, this service already gives us a working place for that pricing behavior. I think it keeps the flow clearer and it helps us get a working end-to-end version. But on the other hand, it is already hinting at a design question. Should this pricing behavior really live in this separate service, or should it perhaps belong closer to the order itself? That is exactly the question I want to solve later. For now, let's switch over to the program and already use this first model. I've pasted in quite some code, as you can see here. What you see is actually the full flow we talked about on the slides. First, we create some products using in this case an object initializer. That syntax should already be familiar to you, but let me quickly recap what I'm doing here. What I'm actually doing here is creating a new object and then immediately setting a number of properties on it. So instead of using a constructor, we create the object first and then we assign values to the properties inside the curly braces. Indeed, if we look again at the `Product` class, we do not have a constructor at this point, so using the object initializer seems like an obvious choice. I'm using the same for the `Customer` and after that for the `Order` here. Notice that the `Order` is linked with that `Customer`, so these two objects are now linked. Then we add the `OrderLines` on the order. What we see here is that we are creating two separate `OrderLine` objects, again, using object initializer syntax, and then I'm adding them directly to the `Lines` collection of the order. Each of these `OrderLines` connects the `Product` with a `Quantity`. So, 1 line is for the cherry pie with `Quantity 1`, and the other one is for the apple pie with `Quantity 2`. This is really a good example of how the code starts reflecting the business scenario. We're not just storing loose values somewhere. We have an order that contains all the lines, and each of those lines refers to a product and how many of that product the customer wants. After that, we create an instance of the `OrderPricingService`. On that object, we call the three pricing methods separately. Now, strictly speaking, if all that we cared about was the final number, we would not have to call all these three separately. But here I actually think it makes sense to do it this way because



I can also show you then the pricing flow. You clearly see that the subtotal and the shipping are two separate parts of the calculation, and the total is built up from those. Notice also that each of these is getting in the same order as argument here. Now, this does mirror the business language quite nicely because in a checkout flow, the subtotal, the shipping, and the total are all meaningful values themselves. And then over here we loop through the order lines. What we see here is that we go over each line that belongs to the order, we calculate the total for that line by multiplying the unit price with the quantity and then use that as part of the summary. And there's quite a lot of code, as you can see, but the important thing is that the flow matches the business story we started from. It is a really useful first attempt of the model. It supports the business scenario in a very understandable way. The code still reads quite closely to the story that we started from. But at the same time, what we also see is that the model is still very open. You are setting everything from the outside like here. You're adding directly to the exposed Lines collection, for example, and all the pricing logic lives in that separate service. That should be really the key message to get from this demo. This model works. That matters, but it is also leaving a lot of design questions open. Those are exactly the questions we will follow upon in the next modules. Let me quickly run this now to make sure that everything works. As we can see in the output, the full checkout scenario works from end-to-end. We get the customer, the items, the subtotal, the shipping, and the total.

Why this model works (for now)

And now that we have seen the first model in action, let's evaluate why it works, at least for now. This first model works because it matches the business, so it already knows about customers, products, orders, order lines, and pricing. And it supports a scenario we actually set out to implement. We can create an order, we can add products, choose delivery, calculate totals, and print a summary. So, from a first model point of view, this is a success. It is understandable and it works end-to-end. Before we improve the model, we need a model that actually supports the scenario in a clear way. But a model can be object-oriented, understandable, and working, and still leave important design questions open. That's important because it shows the difference between "this works" and "this is a strong design." That's what we're here for. We're here to learn how to create solid application code. So what we already have is named classes, collaborating objects, and a working scenario. But what we also have are public setters everywhere, exposed collections, pricing logic outside of the order, and very little protection against misuse. So yes, it's a real model. It's not fake OO, but it's clearly a weak model in a number of places. If you zoom in for a second on the older class, we can see some of these weaknesses already. What you see here is exposed state all over the place. The customers can be changed freely. The collection of lines is exposed directly. The delivery method is just a string. Shipping address is just a string. So from a readability point of view, the



model is still manageable. But from a protection and a design point of view, it's very open. Outside code has a lot of freedom to put the object into weak or inconsistent states. So, where are we now? Well, we start with the first model. Now we will need to identify weaknesses. Then, we'll need to refactor step-by-step to improve things and move to a stronger model. So, we're not throwing this away, we're going to improve it. That's an important mindset when designing a new system, and that's what we're doing in this course. We're not replacing one bad model with all of a sudden a magical, perfect model; we are learning to strengthen the model piece-by-piece.

From writing classes to designing domain models

Module introduction

Hi there, and welcome to this next module where we really will start moving from writing classes to designing domain models. And while we're at it, we'll also understand what a domain model really is. So, what will we change in this module? We already have a first working model, and that is a good starting point. We needed something to get started with and to talk about, right? But now we will start evaluating the quality of that model. We will move from asking, "Does it work?" to asking, "Is it actually a good model?" And those are definitely not the same questions. Something can work, but still be too open. Something can work and still change can be harder than it needs to be. That's what will change in this module. Now, working code is still important, of course, using the correct syntax in C#. That does not go away. But working code is, let's say, not enough anymore. Two classes can both compile, two classes can both produce the expected result, and still, one can be much easier to understand, much easier to protect, and much easier to evolve than the other one. And that is such an important step in learning OO very well. We will stop treating "it works" as the finish line and will start treating it as the beginning of a better design. I dare to say that domain modeling begins when we stop asking, "Can I represent this in code?" and actually start asking, "Am I representing the business problem well?" That is really the key line here for this module because once we start asking that second question here, we're no longer just creating types, we're no longer just mapping data into classes, and we are now starting to think about the business meaning about valid state, about what belongs together, and about which parts of the model should start protecting themselves.

Two classes: Which is better?

To begin the journey, let's start with that very practical question, then. If we look



at the two classes that both work, which one is actually better? Here we see the first Order class, and what we see here in the snippet on the right is something that is perfectly valid from a C# perspective. It stores order data and it exposes that data directly. So yes, it works. There is nothing technically wrong with it just because it is simple. It's also very open. Outside code can reach indirectly. It can decide what goes in that order and how it changes, and it actually means that the class itself owns very little. That's also why this is often called an anemic model or an anemic class. The idea there is not that the class is wrong because it is small, but that it is thin in behavior. It mostly holds data, while the real work and the real decisions tend to live somewhere else. So, already here the question is not whether it compiles, it is where this class is helping the model strongly enough. Compare that with an improved Order class we see here. Now, don't focus on the details just yet. What I do want you to notice here, though, is that it represents the same concept. It's still in order, but the design is starting to change. The internal collection is now kept private. Outside code does not just manipulate it directly anymore. Instead, we now have this AddProduct method, and that is starting to enforce some rules. So, this class is no longer just storing order-related data. It is beginning to take ownership over how an order should behave. That's a very different design direction. So, what is the real difference between these two, then? The anemic order, so the one we first looked at, exposes data directly, relies much more, therefore, on outside code, and protects its data very little. The richer one actually exposes less, owns behavior, and also adds some basic guardrails already. Both classes can and will work. Both can participate in a working application. But they do not guide the model in the same way. One will say to the first one, here is some data, and the other one starts saying, here is how this concept should be used. That's quite a big difference. Let's make that comparison a bit more concrete and go into code. What I want to show you in this first demo is the difference between an order that mostly exposes data and an order that starts owning behavior and protecting itself a little bit more. A small note to start here. I don't want to restart from zero here. We're going to continue building on the first model that we already have and that we're going to continue doing throughout the rest of this course. So, I have opened again the starter solution, which is basically the first working model from the previous module. We see here the same small checkout model we already know, customer, product, order, order line, and the pricing service. And we're now going to start changing the design. I'm going to start again in the domain project and for now I'll add a file called order design comparison for the comparison inside a folder, which is like a temporary folder. I'll call it design examples. Let me paste in the snippet as well. What we see here is really the comparison that we talked about in the slides as well. The AnemicOrders of this one mostly exposes the data. It has the public Lines collection, and outside code can do whatever it wants with that list. That's also why we call this anemic, not because it is invalid, no, but because it is thin in behavior. Most of the real work and the real decisions happen somewhere else. And yes, that is a problem. Next to it we have this, let's say, RichOrder, a



richer version. Now the internalist is kept private over here. Outside code now gets a read-only view that I am achieving with this type here that comes with .NET called the `ICollection`. If that type is new to you, the important thing to understand is simply that it represents a collection that you can read from, but not change directly. So outside code can still inspect the order lines, but it cannot just add or remove lines through that property. Now you may also notice this weird syntax here, this fat arrow syntax. This is just a shorter syntax in C# for returning a value from a property or a method when the implementation is very simple, like here. So in this case what we're really saying is that this `Lines` property returns the `_lines`, but it returns that, of course, as a read-only collection type. So this line is doing something interesting. It says, yes, outside code can still see what lines are in the order, but it does not get direct control over changing them. If something needs to be added, then that happens through the `AddProduct` method. And the interesting part is not just that there is a method, it is, in fact, that this method is starting to protect the model. As you can see, it checks whether the product we're adding is not. It checks whether the quantity is greater than 0. So this behavior is not just doing work, it is also enforcing rules, enforcing intent. Let's now switch over to the `Programs.cs` and use these two classes side-by-side. What we see here is that the two classes can be used. The anemic one works and the rich one works as well. The difference, of course, is in how they're used. When we look at the anemic version, we see that we reach into the `Lines` collection directly and so we're building up the order from the outside. When using the richer version, we go through the `AddProduct` method, and that means that the class, so the older class, starts to decide what a valid interaction with it looks like. Down here at the bottom, we also see a first small guardrail example, and we try to add a product with quantity 0 to the order. Then the richer class will reject it immediately. That is, in fact, a first attempt at creating a stronger model, a model that can help protect itself. What this comparison shows us is that even small differences in what a class exposes and what it protects can change the quality of the entire model. That's an important point because design quality does not always show up in huge, dramatic ways. Sometimes it is exactly these smaller choices that make the difference over time of what is private, what is exposed, where does behavior live, what can outside code do directly? Those choices shape the model much more than they may first appear. Now, and just to be clear, we are not comparing "good OO" with "bad OO." That's not what I want to do here. Both of these are valid OO approaches. The differences in clarity, data protection, maintainability, and so on. That is important because I do not want this to sound like there is only one legal OO style. That's not the point. The point is that some choices give us a stronger model than others.

What is a domain model?

So, let's take a step back and make that language more explicit and start really



implementing things this way. The first question I really want to answer is, what really is a domain model? That's a great question. I did mention the term already quite a few times without really explaining. Slightly intentional, by the way. The domain model represents the business problem in software. Well, that is the short version, but it's important to understand it. We are trying to reflect the business reality in code; not just technically, but meaningfully, so we want to get that meaning in the software. A domain model is not just a bunch of types that happens to exist. It is our attempt to express the relevant business concepts, rules, and relationships in a way that the software can work with. And the domain model itself is simply the part of the world the software is about. In this course, that includes things like customers, products, orders, pricing, delivery, and all the rules around these concepts. That's our business world, at least a part of it that the software cares about. So when we build a domain model, what we're really doing is building a representation of that little world in code. That is also why a domain model is more than just classes with properties. Yes, it does include structure. It includes data. But it also includes business meaning, rules, valid state, behavior. That's the big shift. We're no longer satisfied with just the technical structure. We want the model to start carrying some of the meaning of the business problem as well. A model should therefore reflect business reality, so the real world around the business problem. So, if we know, for example, that a shipping order should have a shipping address, or that an inactive product should not be added to an order, or that a quantity of zero should not be allowed, then ideally the model should help express these rules, these truths. These are business rules, and if we know those rules, then we'll want the model to start reflecting them. When our code allows these kinds of things too easily, the model is simply not expressing the domain strongly enough. That's the problem, then. The code may still compile and the application may still run, the happy part may still work, but the model is not helping us enough if invalid situations are as easy to create as valid ones. That is usually a sign that the domain meaning is still too weak in the code. Valid state matters. A stronger model makes valid objects easier to create, makes invalid situations harder to represent, and begins to protect the business rules more naturally, let's say. In other words, validity becomes part of the model itself instead of something that every call of the code has to remember all the time. That is really a huge improvement because it reduces the amount of accidental misuse the rest of the code can introduce. The more the model can guide calls toward a valid state, the stronger and safer it becomes. Here you see an example of invalid state that will make it real for you. The order is marked for shipping, but it has no shipping address, and still, the code allows that state quite easily. So what we see here is actually a weak model, not because it fails technically, the C# is correct, but because it allows something the business world would not really accept. That's exactly the kind of thing we want to start noticing more quickly now. Not only whether the code works, but also whether the code makes invalid business situations too easy to create. One important idea here is the idea of invariants. An invariant is simply a rule that should stay true for a concept. A shipping order



should, for example, have a shipping address. An order line should have a positive quantity. Those are examples of rules that are not supposed to be optional. If the model itself guards those rules, then outside code does not need to keep remembering them over and over again. The model starts helping. That is one of the real strengths of a rich domain model. At this point, it may not be 100% clear in terms of how valuable this is, but that'll come. That's a promise I'm making to you. That brings us to another important principle, behavior and data belong together. A stronger domain model does not just store business data. It owns some of the behavior and some of the rules that make the data meaningful. That is where objects start becoming more than just wrappers around properties, because if behavior stays completely outside and if all the rules stay outside too, then the object itself, well, remains quite passive. In the business model, that means that the design is weaker. If you look at this `AddProduct` example, what we see is that the code itself starts protecting the model. It checks for the null value, it checks for valid quantity, and only then does it add the new line. So behavior is not just acting on data here, it's also defending the rules of the concept that we're modeling here in code. That's exactly why behavior matters so much in a domain model. It gives the object a more meaningful role, and it helps keep the rules closer to where they belong. Let us make that a bit more concrete and jump back into code. What I want to show you in this next demo is how a valid state, weak guardrails, and richer behavior start showing up in the domain model. I'll start with two very small objects in the domain. First, I will add a `GetFullName` method on `Customer` so that the code reads a little bit more meaningful. And let me add that in. What you see here is a very small first step, but it is already hinting at something important. The class is starting to own a little bit more meaning instead of just exposing raw values. Next, I will replace the `DeliveryMethod` with an enum, and I'll update the `Order` class to use that. And here is now the updated `Order` class, which, as we can see, now uses that `DeliveryMethod` enum. And because the `DeliveryMethod` changed, I also need to update the pricing service as we can see here. So let me fix the error here. Here in the `CalculateShippingCost`, we're now checking what the `DeliveryMethod` is, and we're doing that based on the newly introduced enum and property that I've added on the `Order` class. So, we see here a slightly better technical representation for delivery because we now have a named concept instead of passing around strings like we had before. That alone, of course, does not solve yet the bigger design problem. Let's go back to the `Program.cs` and make that visible. I'm pasting the updated scenario. What we can see here is that the model still lets us do some things too easily. Even the first example, this cherry pie, isn't active, but we can still add an inactive product to an order without the model objecting at all. Nothing is really checking this business rule, so that'll go through without hesitation. The second example is possibly even more striking. Here I am creating a shipping order, but without a shipping address. If you look at the `Order` class, why wouldn't it go through? Nothing is in place to stop this from happening. That is exactly the point. The code still works and the application will run, but the model is not expressing what the business requires strongly enough



yet. So, even though that we have started making some small improvements, valid state is really not protected by the model at all at this point. Outside code still has far too much freedom. If we run this as said, and as expected, everything still works, that is really the problem. Invalid or weak states are still very easy to represent. That's the transition we need to make into invariants and stronger guardrails.

Where we're headed in this course

So, where are we headed from here? This module is really the first clear step in the right direction. We're not fixing everything yet, but now we're looking at the model through a much stronger design lens, I should say. Over time, what will improve is, well, quite a lot. Objects will start protecting themselves, read less exposed data, and more meaningful behavior. The creation we'll look at it, too, and that'll need to become safer as well so that we end up with fewer invalid objects. The relationship between the different objects will become clear in the model, resulting in clear, better ownership, and also better collaboration between objects. We'll also get better modeling of variation, and that will avoid conditionals and instead will introduce interfaces and polymorphism. So, not trying to transform the model all at once, we're going to do all these things step-by-step.

Designing behavior-rich domain models

Module introduction

Hi there, and welcome to this next module where we're going to start designing behavior-rich domain models. Finally, we're going to start improving that model by moving behavior inward, so into the objects where it actually belongs. That's an important step because this is where the model starts becoming more than just a set of classes holding data. So, what changes now? As said, at this point, the model still mostly stores data. Too much logic will actually live outside of the objects, and because of that, the model is still easier to misuse than it should be. We have seen that in the demos in the previous module. That is really the problem that we're going to solve in this module, not by adding complexity just for the fun of it, no, and also not by just sprinkling some more OO features around, but by moving the right behavior into the right objects. So, what are we then going to improve, you may ask? Well, we're going to move behavior where it belongs. We're going to let objects start enforcing more of their own business rules. And as a result, we are going to make the model harder to misuse. The objects will mostly stop being passive holders of data, and they will start doing more work that actually adds meaning to the model. That's work that, for example, was in the surface that calculated the totals, for example. That is



important because a logic outside of the domain objects tends to spread. Different parts of the system will start making decisions independently, so different callers basically will start having different rules. Different helpers may be stored collecting little pieces of business logic. Over time, the model becomes harder to understand and easier to break. That is really why behavior belongs inside of the model. Not because I just want to make my objects smarter, but because we want the model to stay coherent, and also we want to have all the code in one location where it can easily be modified in that one location and not in multiple ones.

The problem: Property bags and scattered logic

So, let's start with the problem itself, property bags and scattered logic. Let me first answer the question. What is a property bag? Well, a property bag is a class that mostly exposes data and also carries very little meaningful behavior. So, the class exists, the properties are there, but most of the real work and most of the real rules live somewhere else in the codebase. And when business rules stay outside of the object, then the rest of the system starts compensating for that, and that is where logic begins to spread. And once it starts spreading, it usually also does not stay in one place, either. So, once that happens, so that spreading of the logic, a few things tend to follow quite naturally. Callers will actually do more. So, from where the code is invoked, we'll see more code appear that needs to do the actual work, maybe in multiple places. The result is that services and helpers begin collecting business knowledge, which they shouldn't, since it's really not their business, pun intended. Rules become easier to duplicate. Indeed, we'll have them in multiple places. Hence, if something changes, we risk missing this in one or more places, and the model itself becomes easier to misuse because it's no longer guiding the rest of the code strongly enough. Here we see a typical property bag. What we see here in the code is that it contains the data, but the class carries almost no responsibility of its own. So, yes, technically speaking, there is an object there, but it's really not contributing much to the design. That is exactly my point. A class can be present in the model and still contribute a very little behavior or protection. The net result of building an application like that is, well, something like this. The object exposes data, the caller manipulates that data, business logic moves outward, and rules start to spread. I think this sequence is very good because it makes it visible, but otherwise just slowly creeps into your codebase. Once that outward movement starts, it becomes much harder to keep the model clean. This is what that looks like at the call sites or at the usage points. We've, in fact, seen this before. What we see here is that the caller adds the line directly. Caller calculates the total, and the object itself contributes very little. So, even though this works, the burden is now on the caller to do the right thing, to remember what to do, and also to keep the rules consistent. That's what you want to avoid. So, what I want to show you in this first demo is how scattered logic actually feels and why a



passive object tends to push too much responsibility outward. So I have again opened the starter solution for this module. What we see here is again the model that we ended the previous module with. And before we go into refactor, I want to point out a problem that we're going to fix in the upcoming demos. So, here is the order, and what we see here is a class that, as said, stores mostly data. The customer is set from the outside. The delivery method is set from the outside. The shipping address is set from the side, and perhaps most importantly, the lines collection is a public list as well. I think you get the message. So, what this class is, is really a classic property bag. The object is there, but it's not doing very much. It is just exposing data. If you look at the pricing service, what we see is that the order is not calculating its own totals. That behavior is living in this `OrderPricingService` to a separate class. That is, as said, okay, for our first model. It got us through the previous modules with working code. But what we see is also that the order itself does not own the pricing behavior. What we see here is that the caller, so this code, is doing quite a lot. It creates the order, it reaches into the lines collection, it creates the lines, and then it asks a separate service to calculate the totals. So yes, it works, but the burden is on the caller to do the right thing and remember the different steps. This isn't new. This is how we designed everything so far. The issue, I think, at this point is mostly where the knowledge lives. The order is exposing too much and the caller through the `Program.cs` is carrying all the responsibility for creating a right correct order. That's the problem we need to solve next. So to conclude, when the object does not own the behavior, the rest of the system starts making the business decisions for it, which is, by the way, a normal thing to happen. Once behavior stays outside, callers start compensating, and the model becomes weaker because of that. And that compensation often looks harmless at first. It is just a few lines here, a helper class there, a check maybe somewhere else. But over time I can assure you it adds up, like with everything.

Encapsulation as protection

So, let's now move into the next important ID, using encapsulation as protection. Encapsulation helps an object control how it is used and helps protect the rules that make it meaningful. That's a more useful way to think about encapsulation in domain design compared to what we saw with just C# syntax. We're not just hiding fields because OO says so, but we're using encapsulation to protect meaning. That to me is where encapsulation starts becoming valuable. Let me try to convince you here as well. So, what does encapsulation protect against? Without protection, any caller can change important state. They can bypass rules easily, and invalid combinations, well, they become easy to create. Once we add protection, the object controls its state. Important behavior will move inward, so into the objects, and therefore misuse becomes harder. So we're not just using access modifiers as syntax trivia, but we're using access control as a design tool. Here I'm showing you what



that looks like in code. You see that the collection is still visible. That's important because other code may still need to look at the older lines, but it is no longer directly replaceable or freely changeable from the outside. So what we see here is that the object now controls more of what happens to its internal lines. Outside code can still look, but it cannot just take over. That is a huge change. Protection also means controlling behavior, like as said. Take a look at this AddProduct method. We've seen it before. What we see here is that the object is no longer passively accepting anything. It will check for null. It is checking whether the product is active and is also checking if the quantity is valid. It even checks whether the line already exists, and if so, then it will increase the quantity instead of adding a duplicate line. So this method is doing more than just adding data. It's protecting the model and expressing business intent. That's exactly the kind of change we want and need in a real system. That protection can and should exist on the different types. At the order level we can control the line collection and guard product additions. At the order line level, we can guard quantity changes. At the product level, we can guard price changes and control active versus inactive state. This is important because when using stronger models, we are not having one single protective object do everything. Protection should exist where it makes sense inside of the model. Let me make it a bit more concrete and hop back to Visual Studio. So I'll start with Product. Right now, the UnitPrice and the IsActive flag can be set from the outside. We're going to tighten that a bit and move those changes behind methods on the class itself. Let me paste in a new version. What we see here is that the Product is still a simple class, but it is no longer just completely open. Notice that we now have private setters. Price changes now need to go through the ChangePrice method, and active state changes now go through the Activate and Deactivate methods. Before we had this change, outside code could just assign values directly, and the object had very little say in what was happening to it. Now for at least these two important state changes, the product itself is now owning the operation around it. That's important because changing a price is not just setting a decimal. From a domain point of view, that's a very meaningful action. And the same goes, of course, for the activating and the deactivating of a product. They're not just technical property assignments anymore. They are business-relevant changes to the state of the object. So what we see here is that the class is starting to move away from just being a bank of properties. It is not perfect, absolutely not. That is not what I'm saying. We can also see that name and product code are still public here, so the class is not fully protected yet, but that's okay for now. Next, I want to do something similar on OrderLine. Right now it is also just a bag of settable properties. Let me paste in a snippet for this one as well. Again, I've done a useful move. Outside code can no longer just change Product and Quantity anytime it likes. We see that the properties are also using a private set. And just as a reminder for that syntax, the idea is that the values can still be wrapped from outside the class with this get, but they can only be changed from inside the class itself. This gives us a nice middle ground. We're not hiding the data entirely because our code, outside code, may still need to inspect the



OrderLine, but we're also no longer saying that anyone can just change the data whenever they feel like it. That's important because an OrderLine is also not just a container of values, it represents something specific in the model, a certain product with a certain quantity as part of an order. Let's go to the order itself, because that, I believe, is the one where the most important change in this demo will happen. As we can see, the internal list of OrderLines is private, so the outside code gets only the read-only view. We already saw the same pattern before when we compared the anemic order to the rich order, so this may seem a bit familiar there as well. The important point was that the outside code could still inspect the order lines, but it no longer could just take over and start changing the collection directly. So I've applied that now on the order. The point of that type is that outside code read what is in the collection, so they can loop over the lines or count them, but they cannot add or remove items through that property. Instead, if something needs to be added, that now goes through the real AddProduct method. It is also protecting the model. It checks whether the product is known, checks whether the product is inactive, and it also checks if the quantity is greater than 0. So it is not just adding something to that collection, it is also enforcing rules. Let me now go to the Program.cs and make those changes there as well. As we can see, we have quite some build errors here. That's because the private set is now active, for example. So let me paste in a new version here, too. What we now see here is that the caller no longer just reaches into the collection directly anymore. The order controls how products are added, so using for that the AddProduct method to add products to the order at hand. The order is now in control. I am still keeping the OrderPricingService. That is intentional. At this point, we're focusing on encapsulation and protection first. The pricing behavior hasn't moved yet. That'll come later. Then further down, I'm also adding this small guardrail example, trying to add an inactive product. This is actually a very important aspect because it shows very clearly what changes once behavior starts moving into the object. Before this, the caller could more or less just build up the order however it wanted as long as the syntax, the C# syntax, that is, was valid. But now the order gets a say in that interaction. When I now try to add an inactive product, the order will check that state and reject that. That is because this code here in the AddProduct part of the actual order class, this will actively block this product from being added to the order, and that is a nice change if you ask me. Let's try running this. And as we can see in the output, the order still works, but it is already harder to misuse, and the Product class owns more of its own state changes. So when I try to add something that is not allowed, the order pushes back. That is throwing the error here. Now this is also why this is much better than just thinking about private fields, which is how we thought about encapsulation earlier. Private data is not the goal itself. The goal is protecting meaning. If an object controls access, but does not protect its rules, then basically we have weak encapsulation. The value comes when protection and behavior work together. That is the important nuance.



Moving behavior where it belongs

So now that I've convinced you, let's look at moving behavior where it belongs. Before, outside code could add products, calculate the total, and keep more of the business rules in mind. After the refactor, Order owns order behavior, OrderLine owns line behavior, and Product owns product state changes. That's a very useful way to think about the refactor. We're not randomly just moving methods around. We're moving code into the classes themselves. That is the first sign of that better design. Here is that Order class again. Now the order total is calculated by the Order class itself. The caller no longer needs to assemble that total externally. That means that the object now owns more of its own meaning. It's a small change in code, but a big change in design, because it changes who carries that knowledge. Where is that knowledge inside of my code? The object is indeed now becoming smarter. That is just one example. Let's jump back to the code so I can show you more of this. In this demo, I want to show you how behavior will move into Order, OrderLine, and Product, and how that changes the shape of the model. Let me start with OrderLine because this is a nice, small example of moving behavior inward. Let me paste in a new version. What we now have is an OrderLine that owns line level behavior itself. The line will now, for example, know how to calculate its own total because of this GetLineTotal method that I've added that returns the total for that specific line. It is also now controlling how quantity changes should happen. I've added two methods directly on the OrderLine class here, the IncreaseQuantity and the ChangeQuantity. Both of these contain small, but important business rules. For example, rejecting values that are less than or equal to 0. So this is another good example of moving behavior inward. OrderLine is, again, no longer just a little data holder with Product and Quantity. It is now carrying some of the meaning and protection that belongs to the concept of an OrderLine. Now again, let's go back to Order because there we need to do quite a big change. There is quite a lot of code in here now, but I'm not going to show you every line. The important thing is to look at where behavior now lives. I'll take you through the important parts of the code. First, we see that Order still has the same core state. It has a Customer, a DeliveryMethod, a ShippingAddress, and internally a list of order lines. But around that state we've now added a number of methods, again, on the class itself. We can see that Order now owns adding products, removing products, calculating subtotal, calculating shipping, and calculating total. That behavior is now no longer spread across the caller, so the Program.cs, or that separate service, the OrderPricingService. The AddProduct, we already had that, that is still the entry point inside the Order class to change the contents of the order. It again checks those rules. That code is definitely protecting the model. There's some extra code in here too. First, I want you to look at this part here where we see this FirstOrDefault call. If that syntax is new to you, and the simple idea is that we are now looking through the existing order lines to find the first one that matches this product code to the one from the product we're



passing in. If no line is found, well, then the result is just the default value, which in this case null, meaning that no line was found. That then connects to this is not null right after it. This is C# syntax to check whether we actually found an existing line. So, these two pieces are saying, do we already have a line in this order for that same product? If the answer is yes, then what we see here is that we do not create a duplicate line. Instead, we call the `IncreaseQuantity` method, which is the method we added on over the line. So really think of this as another improvement. Order delegates the line-specific behavior back to the `OrderLine` class itself. And then we call `return`, so we stop essentially the execution of this method. And then if there is no matching line, then in that case we do create a new old line and add it to the internal list. So this is essentially already doing quite a bit. It prevents duplicate lines for the same product and it uses behavior on the `OrderLine` instead of changing the line itself. Then below that I've also added `RemoveProduct`, which gives the order a way to remove a product from the order. So again, outside code no longer has to reach into the collection and change it directly. The code is very similar to what we had with the `AddProduct`. And now finally I've added `CalculateSubtotal`, `CalculateShippingCost`, and `CalculateTotal`. I've now moved the pricing behavior into the `Order` class itself. The order now knows how to calculate values that belong to that order. This is an important change. The order is no longer just storing data. It is behaving like an order, an entity that knows how to behave as an order. Because of the fact that the order now owns the pricing behavior, I actually do not need the pricing service in this version anymore. So I will remove that class here. That will cause some errors in the `Program.cs`, and I'll fix those in the next demo. For now, notice that we have now done a serious refactoring to our model, moving functionality into the classes and removing the external service code, like the `OrderPricingService`. So we can start seeing much more clearly where behavior now lives. Inside order, we have things like add products, removing products, calculating subtotals, calculating shipping, calculating total. Inside the order line, you can calculate line totals and manage quantity changes. And inside product, we can change price and manage the active state. So, the model is definitely getting richer. That should be clear now. A behavior-rich model does not just store business data. It carries business operations close to that data. That is the key point here.

Assigning responsibility to objects

And that will lead us automatically to assigning responsibility to objects more carefully. And what changes at the call site is very important. Remember, the call site is from where we use the objects. Before, callers could manipulate data directly. They could calculate things themselves and carry in general more business knowledge. After the refactor, so with the changes that we are now starting to implement, callers use meaningful operations. They work together basically at a higher level, and objects carry more business meaning. That is a



very practical payoff because now the rest of the system has way less to remember. And less to remember also means less things can go wrong. That might be even more important! Take a look at this snippet here. We've seen this one before. The caller is doing all the work. It is adding the line directly, calculating the total, and the object just exposes data, but contributes very little behavior. So the object is present, but it's not helping out very much. And here we see the after version of the call site. The caller can use meaningful operations. The object owns more of that behavior, and the code reads much closer to the business intent. That is exactly where we need to go. The caller should not need to remember all the little details if the object itself can guide the workflow. A stronger domain does not just hold data. It gives the rest of the system a better way to work with that data. I really like this sentence here because it captures the improvement that we've done in this module very well. It's not just about moving code around for the fun of it. It's about improving the way the rest of the system interacts with the model. Let's make that visible once more in the code. So at this time I want to show you the difference at the call site because that is really where this change is becoming very tangible. I've now updated the Program.cs, and I'll take you through it. First, of course, we still have the code that creates the different objects it needs to the products, the customer, and the order itself. That part hasn't really changed. Now, once the order exists, the important difference is that the caller no longer is doing the real work. There's a lot less code now in the Program.cs. We now add products through the AddProduct, so the call is no longer reaching into the collection itself. I'm even adding the cherry pie twice, as you can see here. The caller no longer needs to care whether or not that should create a second line or update an existing one. That decision is now owned by the Order class itself. Then the order, and that's probably the most important part, is now calculating its own pricing. You can see that I'm calling the order CalculateSubtotal, CalculateShippingCost, and CalculateTotal directly on the order itself. That's a big difference compared to where we started. Before this, the caller had to rely on that separate pricing service. Now the pricing behavior lives inside of the order where it actually naturally belongs. We also use the RemoveProduct here, which is a good example of the same idea, really. The caller no longer manipulates the list internally. It asks the order to remove the product. So the usage code, this program says is much simpler. And why? Well, because the object model is already a lot stronger. The call still sets up the scenario, but the order-related behavior lives on the order itself.

The benefits: Before and after

Now let's close off the module by looking at the benefits again in a simple before and after comparison. What we see here in the code is that the richer model exposes more meaningful operations. The caller does less work, less low-level work, let's say, and the object itself carries more of the business behavior.



That makes the code easier to follow and easier to trust. Clear usage is one of the best signs that the design is improving. In our different entities, we've made several changes. Order now owns adding and removing products and calculating subtotal, shipping, and total. OrderLine owns a line total calculation and a quantity protection. And Product protects price changes and owns the active versus inactive state. Those are all concrete changes we've now made in the model. Now, before you get your hopes up, we're not there yet. Better behavior does not automatically mean, for example, a valid creation. Indeed, we have methods, better protection, but we haven't spent time talking about the actual creation. Behavior placement is better. Encapsulation has improved. Basic guardrails or basic business rules are in place. But the object can still be created in a questionable state. Important data can still be optional. Creation, in other words, is still too open. So yes, the model is better, but no, it is not strong enough. So that means we have more fixing to do.

Creating objects that are always valid

Module introduction

Hi there, and welcome to this next module. I'm happy. Our model is quite a bit better already. We have moved behavior into objects, the objects started protecting themselves more, and the usage became clearer as well, so that was an important step forward. But we're definitely not there yet. Our next problem is object creation. Like I said, we move behavior into objects, yes, but objects can still quite easily be created in an incomplete or even an incorrect state. So, while the model became stronger during use, it can still start weak right at the beginning, and that matters quite a bit because if the creation is weak, the rest of the model very often starts from a weak place as well. How an object is created is part of its design. That is really important in this module here. If creation is weak, the model starts weak as well. So creation is not just set up code around the object, it is really part of the object's design as well. That's what we look at here. So in this module, we are reintroducing constructors, but in a very different way than just C# syntax. Because the interesting question is no longer what is a constructor and how do I write it in C#; we already know that. The interesting question now becomes what can we enforce when the objects are being created? That's where constructors become much more interested in the domain model. What we want constructors to do here is really require essential data, reject invalid input, and help the object start life in a valid state. So what we'll get to is that constructors really become gatekeepers. They can act as a gatekeeper. They can make the model safer from the very first moment that the object exists, and that is a very different role than just a place where some values get assigned, which is what we have used constructors for so far.



The problem: Objects created in an invalid state

The problem we're really trying to solve in this module, then, is objects being created in an invalid state. Let's see how we can start fixing that. What I like to refer to as weak creation looks like this. The class allows an object to be created before or without essential information being present. So, at that point, the object will already exist, but it may still be in an incomplete state. And that may seem handy at first. In fact, in simple applications, it often does look quite convenient. But the price you pay is that the risk moves, well, further in the system. The object exists, but other code must now also remember what is still missing. As you may have expected, that's where the trouble will start. Here we can see a very typical, but probably weak creation style. What we see here is that the product already exists before it is fully ready. Important information is assigned later one step at a time, so there is a small window here where the object exists, but it is still incomplete. And that is really the issue. The syntax is not illegal, of course not. What we see here is valid C#, but the issue is that the model allows the object to exist before it is really in a state we would consider acceptable. And it could be that we are running code on it before it is really initialized correctly, and that causes many problems, sometimes even hard to track ones. Here we see another version of the same problem. The order can be created. Yes, that works. But if the `DeliveryMethod` is shipping here, then essential information might be missing. So again, what we see here is an object that begins its life in a questionable state. That needs to be solved. If objects can start in an incomplete state, then callers, users of the code, must remember what still needs to be assigned. And once that happens, again, other code will start checking for missing information. We need to add a validation, and that starts spreading outward. And correctness depends, well, again too much on the discipline, that is, discipline from checking that the object is correctly initialized. That is never a strong place for the model to be in because as soon as correctness really depends on everyone remembering the same thing everywhere, the mistakes become more likely. Believe me, they will. If a class allows partially initialized objects, it is really telling the system, "You make sure this becomes valid later." That's really not what we want. We want to model itself to help much earlier than that. We want the object to begin its life in a better state already and not to rely on the rest of the code, hoping it all gets fixed afterwards. Hope is definitely not a strategy here. So in the first demo, let's make that weak creation visible in practice. What we see here is the model on the end of the previous module. Now, I want to make a weakness visible here. The model became stronger while we are using the objects, but object creation is still too open. Thus, objects can still start their life in a weak state. Let me switch to the `Program.cs`. I'm not going to change anything just yet. I just want to show you what the current creation style still allows. What we can see here is that the product exists already on this line here before it is really ready. We create a new product first, and only after that do we assign the name and the product code. The price is also added later to this `ChangePrice`. Now this works, of course.



This is a normal C# code. But from a domain modeling perspective, there is a small window where the product object exists without the information that is probably essential for the product. The customer uses the same style. We created here with an object initializer and assign values to the properties. Convenient indeed, but the object is not really controlling its creation just yet. And an order, well, that makes it even more clear. I'm creating a shipping order, but I'm not providing a shipping address. So the object does exist and the application can run, but from a business point of view. This is a questionable state, let's say. That is the problem we're going to fix next.

Constructors as gatekeepers

Now, constructors should become gatekeepers. That is really the new way I want us to start thinking about constructors going forward. And let's see how we can do this. A constructor can require what is essential in terms of data. It can require the information the object really needs, so the object cannot even be created without it. That also means that the object starts its life with the correct data instead of hoping that the data gets assigned later. So already here what we see is a much stronger design than create now, fix later. You may see a sample and probably a valid constructor for product. What I want you to notice here is that the constructor is not just assigning values, it is also validating them. Product code must be valid. Name must be present. Price must be greater than 0. So the object is effectively only created if those essential conditions are satisfied. If not, then exceptions will be thrown in the creation process, and that is basically blocking the new instance of being created. And it also means that once a product does exist, the rest of the code can trust a bit more that it at least started its life correctly. As I mentioned, this will really help our code and application become more robust. Weak creation basically says, well, let's create the object first, assign important data later, and hope that it becomes valid afterwards. Constructor-based creation says let's require the essential data up front, validate that immediately, and start in the end with a valid object. A constructor is therefore not just initialization code. It is the moment where the model can say, "Yes, this is a valid object," or, "No, the object should not exist like this." This is an important change in how we think about construction because once we start thinking that way, constructors become part of the domain model and not just technical plumbing anymore. That's where we need to go. So, let's make that concrete again and look at this in Visual Studio in our code. So what I want to show you in this demo is how constructors can become gatekeepers. So instead of creating the object first and hoping that the required values are assigned later, we're going to require that data up front. And we'll start with the product. Right now, product can be created empty, and that's exactly what I want to stop. Let me paste in an updated Product class. What we can now see is that the Product can no longer be created without a productCode, a name, and a price. Those values are now constructive parameters, and that means that they



are required at the moment that the object is created. Also notice that the properties for `productCode` and `name` now only have a `get`, so outside code can read them, but it cannot just change them after creation. The price can still be changed through the `ChangePrice` because that is a meaningful product operation that we already introduced earlier. The constructor also validates the input with this code here. So, the constructor is not just assigning data, it is deciding whether this product should exist like this. In the case we have invalid data, we're throwing an exception, an argument exception, that is, which really means to the caller that we're trying to pass something in that isn't possible. So it basically needs to try again with other hopefully correct data. Next, I'll also update the customer in the same way. Again, the same idea applies. A customer now needs a first name, a last name, and an email address when it is being created. So we are removing the empty customer scenario from the model. And, of course, we also see a similar validation logic here in the constructor. Next, let's update `OrderLine` as well. Here the constructor now checks that the product is present, that it is active, and that the quantity is greater than 0. We had some of these rules already in the `AddProduct`, but `OrderLine` itself should also not be weak. If an `OrderLine` represents a `Product` and a `Quantity` in an order, then it should not be possible to create one without a valid `Product` and a valid `Quantity`. So it is better to have these checks also close to the source, and notice that `Product` only also has a `get`. That means that the `Product` reference is basically fixed once the `OrderLine` has been created. The `Quantity` can still change, but only through the methods that we have added here, that is the `IncreaseQuantity` and the `ChangeQuantity`, since it has a private `set`. It just can't be changed from the outside. Now next, let's go to the `Order` and do the same thing there. I've pasted in quite a large snippet. There's quite a lot of code in here, but the main change is again the constructor. An `Order` now needs a customer and a `deliveryMethod` when it is being created. And using these lines here, we are checking that if the `DeliveryMethod` is shipping, then a `shippingAddress` is required as well. So there is effectively quite some business logic to check the validity here in the constructor of the `Order` class. So the invalid state that we saw in the previous demo is now blocked at this creation point. We no longer can create a shipping order first and hope that an address gets assigned later on. The constructor is explicit. It says if this is a shipping order, then the address is essential. Notice that the customer only has a `get` here as well, and the `DeliveryMethod` and the `ShippingAddress` have private setters, again, blocking values from being set from the outside. Now let's head back over to the `Program` class, which as you can see is now giving us quite some errors. That's normal. We've added parameters in the different constructors that we're using here. I'll bring in a snippet so that we can prove that these constructor guardrails really do work. What we see here is first that I'm trying to create a few weak objects, a `Product` without required data, a `ShippingOrder` without the shipping address, and an `OrderLine` with a quantity of zero. I'm using try catches here and so an exception is actually a good result. It means that the model is pushing back. The invalid objects do not get to exist and travel in the system. That



will avoid a lot of runtime bugs. Down below here I'm creating the valid scenario. Notice how this code now reads. A product is created with the product code, the name, and the price immediately. A customer is created with the required customer information immediately, and their shipping order is created with the shipping address in one go. Let's run this to test it out. And as we can see, the invalid creation attempts have been blocked and the valid order workflow still works. What constructors improve immediately is quite a lot. Required data becomes obvious. Invalid inputs are rejected early. That's a lot safer for the application to avoid runtime errors. The rest of the code can trust the object much more. And creation becomes part of the model. It is not just set up code around it. As said before we went into the demo, the constructor is not just purely technical anymore.

Multiple constructors, one valid state

Now, of course, a class may still need more than one valid way to be created, and that's actually a very normal thing to have in your code. Not every object has only one possible scenario in which it can exist. So, the question here is not where the multiple creation scenarios are allowed. The question is how do we support those different scenarios without weakening the rules of the model again? As said, a class may need multiple entry points. And with that, I basically mean multiple ways of creating an instance of a class. That is not a problem in itself. The problem would be if those different entry points produced different standards of validity. Then one constructor would create a strong object while another one would quietly allow something weaker. That is exactly what we want to avoid. Different scenarios can be valid, but the object should still end up valid in a consistent way. When we talk about having multiple ways of creating an object using constructors, I'm referring to what is known as constructor overloading. So just like with methods, we can have multiple constructors next to each other, as long as the parameter list is different, either in the number of parameters or in the types. That's what we see here in this Customer class sample. Two constructors, both for creating a customer, but with a different parameter list. So yes, that is the syntax side of it. This is how C# lets us support multiple creation parts. But the important thing is not the syntax. The important thing is what we're going to do with it in the design. So, constructor overloading in C# basically means more than one constructor is possible. They differ in the parameter list, and that allows us to support multiple creation scenarios. It's useful, but only if those scenarios are all valid ones. So this is a language feature, and we will connect that to the actual design goal because that's more important for what we are learning here. So, overloading is really the syntax. Valid creation is what we need to get to. What we see here is a shorter Customer constructor delegating to the fuller one. That is where the `this` syntax comes in. This syntax is new to you. Basically what it means is that one constructor is calling another constructor in the same class. So, in this case, the short constructor is not doing all the work



itself. Instead, it forwards to the fuller constructor together with a default email address. That's a nice design move because what we see here is a shorter way of creating a customer, but that shorter path still ends up going through the same main creation path, so the object still ends up complete according to the current model. That's much better, and that's a very good use of overloading rather than duplicating the same creation logic in multiple places. Then in the full constructor, the main validation stays in one place. That's really important because it keeps the rules consistent. Like I said, we don't want multiple constructors all validating differently or forgetting some rules entirely. That would make the creation story much weaker. So what we see here is not just using multiple constructors. What we see here is using multiple constructors with only one real validation path. The shorter one supports a different valid scenario, but actual checks will still happen in one place. Let us head back to our sample and see how we can support multiple overloaded constructors in our design. So I'll start here in the Customer class again. At the moment, the customer, the only one we have, requires a first name, last name, and an email. That's normal for the customer. But let's say that we also want a guest-like scenario, where we only have a name for now. Let me paste in a second constructor. What we now see here is constructor overloading. We have two constructors with different parameter lists. This one here is now added to be clear. The important part is this syntax here, the `:` this. If this is new to you, the idea is that this constructor is calling another constructor on the same class. The short constructor is not doing its own separate validation. It delegates to the fuller constructor and provides a default email address. And to be clear here again, it is calling the constructor with the expected list of parameters. It could be that one in turn calls another one, and so on, but not limited to, let's say, calling just one level deep. Also, we can create as many overloads here as we want, or need, that is. At least from a C# language perspective, that is possible. Adding this second extra constructor now means that the guest-like scenario is supported, but the main validation part that is still centralized. We're not copying the same checks into multiple constructors, and we're not creating a weaker constructor that skips the rules. Copying business rules in multiple places is rarely a good idea. You'll need to update them in several places, and when this happens, well, you're bound to forget an occurrence here and there. I'll also update the order with a short constructor for pickup scenario, and I'll add controlled methods for changing delivery later. Here, again, this shorter order constructor delegates to the full constructor. That means that a pickup order cannot be created without a shipping address, which is a valid scenario. But if someone tries to use that same shorter path for the shipping order, then the fuller constructor will still check it and reject that. Notice here also that the `shippingAddress` parameter here is optional. I've added a default. This is a good practice when that parameter isn't required for object construction. It means, or should mean, I should say, that there won't be any checks in the constructor code for this. I've also added these two methods here, the `ChangeToPickup` and `ChangeToShipping`. Those are, of course, not constructed to the regular methods, but they follow the same idea. Since delivery



method and shipping address are no longer settable from the outside, we need controlled operations for changing the delivery of an order. `ChangeToShipping` requires an address so that we make sure that the order doesn't become invalid later, either. Let's update the program and show how we can use those overloads. What we see here now is that the caller has two valid ways to create a customer. For a full customer, we do pass in the email address, but for the guest-like scenario, with a shorter constructor, and the class supplies the fallback email. The important thing is that both paths still create a complete customer according to the current model. The shorter part is, as said, not allowing us to bypass validation, it goes through the same main constructor, and that same idea is visible with the pickup order. Pickup order does not require a shipping address, so the shorter constructor makes sense here, but the validation rule for shipping orders still remains in one central place.

Object initializers: Convenient, but not always safe

Now let's talk about object initializers because they are convenient, absolutely, but they're not always safe. I'm not saying that object initializers are bad. They are useful, and we all should use them. The question is simply whether and how they fit in a richer domain model. Here we see what an object initializer looks like. It's a commonly used syntax. We've used it already in earlier courses. It looks like a constructor very much, but it is not a constructor. Basically, instead of using brackets, we're using a set of key-value pairs pointing to the different accessible properties on the class, and those are listed in a comma-separated way in between a set of curly braces. What we see here is that the object is being created first, and the values are only assigned after the construction. That little detail matters quite a lot in a domain model. They are convenient, absolutely. They're easy to read, they're easy to write, and very familiar in everyday C# code, and they fit well when objects are simple, when data is mostly optional, and when there is little domain protection required. So, I'm not saying that object initializers are bad. It is more about saying that they are not always a great fit for the kind of model we are trying to build here. The issue that I see is that in a richer domain model, some values simply aren't optional. If the object can exist without those values, then the model is weaker. The object may technically exist, but from a business point of view it may not be valid yet. That's, again, where things become risky. So, with object initializes, the object exists first, and required values may come later. It is possible, of course, that the parties do get their values, but nothing is checking this. With constructor-based creation, required values are supplied up front. Validation happens immediately, and the object will start in a safer state. So basically, the risk with object initializers is something like partial initialization. Here we see the same object in those two creation styles. On the left, we see that the initializer version allows the object to exist before the required data is complete. While on the right, we see the constructor version, which requires that essential data is available



immediately. Well, time to head back to Visual Studio and see what we can do effectively with object initializers. For this demo, I don't want to refactor the domain model again. Instead, I want to look at the call sites of the Program.cs and compare the old object initializer style with the constructor-based style we now have. Let me paste in a short comparison here in the Program.cs. First, I have the usage of the object initializer syntax. As we can see, that is currently giving me an error, both on the customer and the deliveryMethod. The syntax, though, for using object initializers looks fine. We're using a set of curly braces after the call to the constructor, which allows us to use those key-value pairs. Why is it failing then? Well, let's head back to the Order class. And I see that we don't have a set on the customer and we're blocking the call to it. If I add it quickly again, then we can see that we can use it from the outside again. For now I'll comment out the DeliveryMethod, which also wasn't callable because of that private set. Now still, things aren't compiling since we don't have a parameter-less constructor anymore. Only when I bring that back can I use object initializer syntax again. As said, there is nothing wrong with the syntax per se, but notice what I have now introduced again, the ability to create an invalid type of order in the system by bypassing all the rules, even omitting to add the DeliveryMethod which was previously blocked by the regular constructor. So yes, using it from a C# language perspective is fine, but we will end up with a domain object, an order in this case that can be trusted less. So although this works, I will undo the changes that I've done before. As said, object initializers are still useful in C#, but in this kind of domain model for these important entities, constructor-based creation gives us much more protection. The object starts its life in a safer state, and the rest of the code can trust it.

Primary constructors in context

We've seen and used the normal way of using constructors, but since a more recent version of C#, a new syntax was added to C# classes called primary constructors. Definitely the syntax is interesting and useful, but again, the real question is not can we use it, which, of course, we can. The real question in OO design becomes where does it fit well? Here we see the basic primary constructor syntax. The parameters are declared right in the class header, and they are available inside the class body. So the syntax is indeed shorter. No longer do we need to add a full constructor. We just have these parameters, and we can use them inside the class. This compactness makes the syntax quite popular nowadays. With regular constructors, we define the parameters directly in the constructor, and we use the initialization logic also there. With primary constructors, we declare them in the type pattern, so the initialization can be shorter and, well, less repetitive. That is a real advantage in some places. We all like efficiency, right? Here we see a real example of using a primary constructor, and what we see here is exactly the sort of case where this works nicely, a small helper-style type with straightforward data flow and very little, if



any, validation logic. So, again, this is not me saying never use them, not at all. It's about placing them in the right context. That's really why primary constructors are useful. They fit well in small helper-style types, simple data-carrying types, and cases where there is a little validation logic in general. They can be really elegant in those places. But on the other hand, they are often not the best fit for rich domain entities because rich entities may need multiple valid creation parts, heavier validation, clearer guard clauses, and more explicit construction logic. This is much closer to the kinds of entities we are working towards in this course. So that is why I would be a bit more careful applying this syntax too quickly to the heart of the richer domain model. So, let's keep primary constructors in simpler cases. Let me show you in the application where I suggest we use them. I don't want to use primary constructor for our main domain entities. Product, Order, Customer, and OrderLine now have explicit constructors with validation and guard clauses. That explicit style is very useful here, but primary constructors can be a good fit for small helper-style types. I've already added a file here under the DesignExamples folder. Let me paste in a snippet. What we see here is now primary constructor syntax. The parameters are declared directly in the class header, and that is where the syntax is different. In OrderSummary of this one, that means that customer name and total are available inside of the class body, and we use them to initialize the properties. That's what we see here. So indeed, inside the class body these values, these arguments, are available. So, for a small helper-style type like this, this syntax is actually very nice. It removes some of the boilerplate and keeps the type, well, very compact. Indeed, we don't need a constructor for this anymore, which was without primary constructors the only way to assign values. And now let's use those helper types in the Program.cs. What we see here is that the helper types are very easy to create and easy to use. In fact, from a syntax perspective, at the call site, over here, there's really no difference. Once they are created, the order summary can now describe itself. The shipping quote can calculate a very simple shipping quote. And that is what I really want to touch on primary constructors. As said, they're useful when the type is small and straightforward. But for rich domain entities where we need multiple valid creation parts and more explicit creation and validation, a regular constructor is usually clean.

When constructors start to hurt

Constructors are great, but should we always use them, or are there alternatives? We've seen that constructors solve a real problem, absolutely. They help us set up initial data up front, and they make invalid objects much harder to create. That's what we wanted, so they are the perfect place to place this logic in, like we've seen. But they can become heavier as the creation story becomes more complex. So, how would constructors then get heavier, but more required inputs may show up, more valid scenarios appear, more overloads may need to be



added, and more rules may be needed to stay consistent, all problems that will start appearing when we design a larger applications for sure. Now the problem is not that constructors, well, let's say, enforce too much. The problem is that object creation can become more complex than a constructor can really express. So what do we need then, you may ask. We still want valid creation, we still want centralized rules, and we still want strong guardrails. And we've seen that C# constructors can give those to us. But on the other hand, we also want clarity when calling the constructor. We want more expressive creation and better handling possibly of multiple creation scenarios. So some things we'll absolutely want to keep, but some things we also may need to improve. That's exactly the next problem we are going to solve.

Encapsulating object creation with factory methods

Module introduction

Hi there, and welcome to this next module where we are going to look at object creation with factory methods. Constructors solved a real problem back in the previous module. They helped us require essential data upfront, and they helped us block invalid creation. So I want to be very clear from the start. We are not undoing that work. In fact, we're going to build on it, but what we also started to see is that constructors are not always the clearest way to express what kind of object we are creating, that is really what this module is all about. Don't get me wrong, constructors remain the default, that is still a baseline. Factory matters, on the other hand, come in when a named creation part makes the model clearer. So like I said, this module is really not about replacing constructors everywhere. It's about adding another tool for the cases where the constructor call starts asking the reader to infer too much. As we'll see, there are places where this change will matter quite a bit, and as said, we are not going to undo all of our constructor usage here, far from it. So we may be asking then what problem are we then going to solve. Sometimes a constructor call is technically valid, but may be still a bit awkward to read. Yes, the outcome, the object may be valid, but the creation intent, let's say, is not obvious enough when it is being used. Named creation methods can help with that. You'll see that soon because they make the scenario, well, visible directly, which will improve the readability of the code. Here we can see a very good first look at the difference. Don't worry about the details just yet. What we see here is that both approaches can create a valid object, so this is not about one being correct and the other one being wrong. The difference is in how much guessing needs to be done when the constructor is being called. With the factory version, the one on the right, that is, the scenario is named directly, so as we can see here, create shipping order. Shipping order



means, well, shipping an order, and pick up order means pick up order, so you no longer have to decode that from a parameter combination, which is what we see on the left with the constructor-based approach. This, as you probably see yourself, becomes easier to understand more quickly. That is a good rule to keep in mind for this module. We will use factory methods where they improve the design. We're not going to introduce them where they only add ceremony. So again, this is not a factory methods everywhere module. Alright, time to start understanding everything in some more detail.

Recognizing constructor pain points

So let us start with recognizing the pain point that leads us here, and that is constructor pain at the call site. The issue is that some constructor calls can become harder to read and harder to interpret quickly. The object is valid, but the creation intent is not obvious enough. That's important because reading code is an important part of development, even more so in the time of AI where we'll need to review more and more AI-generated code. If a creation call makes the reader stop and decode what is happening, then the model may still be valid, but it may not be as expressive as it could be, so the problem is, well, hidden intent. You can see that in a real code, both of these calls on the right are valid, but still, the meaning depends on interpreting the arguments correctly. You have to look at the delivery method and then connect that to the shipping address and infer the scenario yourself. I hope you see where I'm going with this. The constructor call can sort of hide the intent of the code, even while it's technically perfectly fine. That's exactly the kind of problem we'll often encounter when writing this sort of code. So as already mentioned, the problem we have is that the constructor call may be correct, but the code still asks the reader to guess too much. So, let's make those constructor pain points a bit more visible in code, so we'll jump back to Visual Studio for this. So here is the starter solution for this module. Like before, this is what we ended the previous module with. The objects are valid, but some constructors are starting to ask the user to interpret too much, so what I really mean is that, as said, the code is not really expressive enough, while of course technically being perfectly valid code. Let me go here in the program and paste in a scenario. What we see here is that some constructor calls are still perfectly fine, so these ones here, product is still clear enough. We pass for the code, the name, and the price. Customer, same thing, pass the first name, last name, and email. There's really not much guessing there. But with order, and the constructor is well, doing a bit more. We pass a customer, then the delivery method, and sometimes a shipping address. So if I read a new order with delivery method shipping and an address, I can understand that this is a shipping order, but I have to, let's say, interpret that combination of arguments here, and the same is true for a pickup order. It is valid, it works, but the creation scenario is not named directly. I need to look at the enum of delivery method, that is, and then infer the meaning. That is the constructor pain point we are focusing on



here. The model is not invalid and the constructor is not broken, but the call site is not really as expressive as clear as it should be. That I would like to improve. Now, to be clear, most objects are still perfectly clear and fine with constructors. A product with code, name, and price is still easy enough to read. A customer with a first name, last name, and email, well, same thing, those are the ones you see on the left here. But an order with different delivery scenarios is already starting to become less clear. Like I said, this is not a problem we will have with every constructor. The problem is specific cases where the constructor stops being expressive enough, so we don't need a new creation approach for every class. That's absolutely not what I'm seeing here. We need it when the meaning of creation should be more obvious than the constructor call can make it. That's really, I'd say, the decision point. It's not, can we have a factory method here, but does the model become easier to understand if we do.

Introducing static factory methods

So let's now introduce static factory methods properly. And perhaps first, the most urgent question, what is a static factory method then? Well, great question. In fact, it is simply a named method on a class that creates and returns an instance of that class, and the useful part is that the method name can express the creation scenario directly. So instead of relying only on a constructor signature, we now get a named entry point that can tell us more clearly what this thing is doing. That's the real benefit here. A factory makes the scenario more explicit. The creation intent is visible in the name. The caller no longer has to infer the scenario from the constructive arguments. And because of that, the code just reads closer to the business language. Yes, this is nothing difficult, far from it. It is just a static method that we're using. What static means that we can call it on the class, instead of creating an object and calling it on that. Here, we actually see what I've shown in code already. The constructive version on the left works perfectly fine. But the factory version of the right is, in my opinion at least, more expressive. That's the most important takeaway here. We're improving the readability and the intent of the code, not validity that we already covered when we looked at constructors. So this is a new improvement, but it is building directly on the one we already have made. A good factory method name reduces guessing at the call center from where the construction is being done. That by itself is a real design improvement because when the creation story becomes clearer, the rest of the code becomes easier to follow as well. It is one of those changes that may look small in code, but has a nice effect on readability, and something we commonly use in C# code. Hence, it also is getting its own module here. One important nuance here, the factory method still can and should enforce the rules like we discussed in the previous module. The named method absolutely is not removing validation. It should not weaken the model. It simply gives the creation scenario a clearer entry point. Under the surface, we are still making sure that the object is created correctly. That's what we did in the



previous module, and that is still applying here. So time to look at factory methods in the model and make it, well, concrete in our code. To do this, I will start in the Order class because, as I said, this is where the creation scenario is a bit less obvious. I only want to introduce factory methods where they make the design clearer, so let me paste in an updated Order class. The important change we see here, that is the create pickup order and the create shipping order. These are now static factory methods, so those are the ones that we talked about in the slides. Static means that we can call them on the order itself, so `order.CreateShippingOrder`, for example, without first creating an order object, and they are indeed factory methods because they create and return order instances. They manufacture objects, hence the name factory objects. What we now see here is that the creation scenario has a name. `CreateShippingOrders` is exactly what we're doing, and that's what I mean with the code being more expressive. `CreatePickupOrder` also says exactly what we're doing, so the caller no longer has to interpret the delivery method argument to understand the scenario. Now when working with a large code base, you'll really be thankful if you can find these factory methods that really say what we are doing. We just have a few parameters that we need to interpret, but in real applications, you'll see a lot more, and making things more clear will absolutely help, but then also that the factory methods still call the constructor internally. We're not throwing away the constructor validation from the previous module. The constructor is still the central place where the object is initialized and where the shipping address rule is checked, so this is the first important design change in this module. The factory method improves the clarity, and the constructor still protects the validity. Here, we now see the difference at the call site. Product still uses a constructor. That's fine. Customer still uses a constructor, also fine. We're not turning every constructor into a factory method just because we can, and let that sink in. I'm absolutely not saying replace every constructor with a factory. But for order, the creation scenario is now named. Instead of a new order with the `DeliveryMethod` argument, we now call `order.CreateShippingOrder` or `order.CreatePickupOrder`. That reads much closer to the business language, and when reading the same code in a few months, this will still make sense. Over here, I still have that guardrail example. I'm trying to create a shipping order with an empty address. The important thing to notice is that the factory method doesn't weaken the rules. It still ends up calling the constructor, and the constructor will still reject an invalid shipping order. So indirectly, this is doing the same thing. We're still going through the constructor and that's what we need. Let's run this now, and as we can see, the valid shipping and pickup orders are created, and the invalid shipping order is still locked, so factory methods are not replacing validation. They are giving us just a better public creation approach. So what have we improved already? Well, the creation is easier to read at the call site. These scenarios are named now explicitly in the model, so the business intent becomes clearer both in the code that uses the model and in the design of the model itself. So even before we go further, we already have a benefit here. The model reads closer to the language of the business. That's always a good sign



when designing your model.

Private constructors and controlled creation

Now, let's take the next step, private constructors and controlled creation. So maybe first, why does this matter? Because as long as the constructor stays public, callers can still bypass the name creation methods. So even if we add a clearer path, we're still relying on convention, that is them using that factory method. But by making the constructor private, we gain control over how instances of a class are created. So far, all constructors were always public, and now take a look, now we're going to make it private. That's what we see here. That means effectively that the constructor can't be called from outside anymore, so we'll introduce a factory method alongside with it, and only that can be used to construct, therefore guiding the user. What I want you to notice here also is that the validation scenario is still there in the constructor. So again, the private constructor is not about removing the rules. It's about keeping the real construction logic in one place while taking away the bypass path from outside code. And I said, these factory methods then become the public entry points like we see here. This is now a nice combination. The public API is very clear because it gives us named creation paths, and the actual construction logic is still centralized underneath that. So indeed see that this code, in turn, is using the constructor, which is then private. Of course, this works since it is calling it from the same class. What we are getting is that factory methods improve clarity. The private constructor still protects validity, and those two things fit together very nicely. I think that is one of the stronger design combinations in this whole course so far because both paths reinforce each other. So let's make that controlled creation visible in code. So right now, we have `CreateShippingOrder` and `CreatePickupOrder` as the factory methods, but the constructor on the `Order` class is still public. That means that the caller can still ignore the clear factory methods and create an order using the constructor. So we have improved the design, but we are relying on conventions, saying that we should use the factor methods instead. But the convention is just as weak as the next developer not knowing about it and just going ahead and using the constructor. So let's go here and make that a private constructor, and the main thing that changed is small, but it is important. In fact, I've just changed one keyword here. The constructor is now private. Just as a recap, private means that outside code can't call it anymore, so this constructor can't be called from the outside, but code inside the `Order` class can still call it. That's exactly, of course, what our factory method is doing. So indeed, `CreatePickupOrder` and `CreateShippingOrder`, they're still public. They are now the public and only creation pats, but the actual construction logic that still remains centralized, but now in a private constructor. That gives us a nice combination. The caller gets those clearly named methods, and the class just keeps one validated construction path. A code outside can't just bypass that by calling the constructor directly. In the



program.cs, as we can see, we can't call the order constructor anymore. Instead, we need to use the `CreateShippingOrder` for a shipping order and the `CreatePickupOrder` for a pickup order. So from a design point of view, this is what has changed. Callers now see named creation methods that improves the clarity. Callers can no longer bypass the intended creation path that improves control, and all creation still goes through one validated constructor. That gives us consistency. So what we now have is not just a nicer call site, we also have stronger creation boundaries in the model. Again, I want to stress this, private constructors are not for every class. They are useful when creation should happen through specific named paths. In other words, when creation intent really matters.

A quick look at factory objects

In the final part of this module, I want to take a quick look at factory objects. I want to stress the quick look part here because the goal is not to suddenly open a whole new topic in depth. What I want to do instead is show you the next issue that can appear once we start taking object creation more seriously. Note that this is really still part of the same story. First, we use constructors to make creation valid. Then we introduced static factory methods to make creation clearer, and now we're looking at what can happen when that creation process itself starts needing a little bit more help. Static factory methods work well when the creation logic is self-contained, and what I mean by that is, well, pretty simple. The method gets all the information it needs through its parameters, validates what needs to be checked, creates the object, and then returns it. That's exactly the kind of thing we have been doing so far. But sometimes creating an object requires another collaborator or dependency. If those words sound a bit abstract, but the practical meaning is very simple. It just means another object that helps with the creation process. So maybe creating an order also needs something that generates an order number, say, or something that looks up a setting, or something that helps coordinate an extra part of the creation flow. And once creation needs that kind of help from another object, static factories are not always enough, basically, because the creation process is now becoming a bit more involved. So what is a factory object then? Well, good question. Let me explain. A factory object is simply a normal object whose job it is to create other objects, and because it is a normal object, it can hold collaborators. In other words, it can keep a reference to another object it needs in order to do its work. So if creating an order needs help from something like the order number generator, a factory object can receive the generator once, keep it basically as a state, and then use it whenever it creates a new order. That is the important difference here. The equation logic is still kept in one clear place, but now that place can also hold what it depends on. Here we see the difference in code. On the left, we have a static factory method. That is called directly on the type itself and that works very well as long as the method



does not need anything beyond the parameters it receives. On the right, we first then create a factory object, and when we do that we pass in an `orderNumberGenerator`. The latter one here is the dependency, or if you prefer, the helper object the factory needs. That is really the practical difference. A static method is just a method on the class. It is not an object you create and keep around, so it does not have its own field where it can store something like a number generator. A factory object, on the other hand, does. Here is an example of a factory object. What I want you to notice is that the `PreviewOrderFactory` receives an order number generator in its constructor and stores it in a field, so that generator is the dependency or collaborator in this example. It is the extra object this factory needs during creation. Then, when `CreatePickupOrder` is called, the factory first asks that generator for the next number, and only after that does it continue creating the order. So the important point here is not the order number itself. The important point is that the creation logic now needs help from another object, and the factory object gives that logic a proper place. Instead of having a method to perform the creation, we now have an entire class. Factory methods are useful since they can hold collaborators. They can coordinate more complex creation, and they can still return fully valid objects at the end. So the goal does not change at all. We still want valid creation. The only thing that changes is the mechanism. When creation becomes more complex, a factory object can carry that complexity better than a static method can. So I really see this as the next step on the same line, not as a completely different technique. Now what you are seeing here definitely leans towards dependency injection and containers, a common concept you'll often encounter when building real-world applications with .NET and C#. Because the moment a factory object receives what it needs from the outside, instead of creating it itself, we are already very close to the core idea behind dependency injection. Now later on in a larger application, so-called dependency injection containers can help wire, let's say those dependencies together automatically. Now that is not the focus here. That would take us too far, but know that this is, in fact, what we are talking about. So what I want to show you in the final demo is not a full new design and definitely not a whole dependency injection setup. I just want to show you the moment where static factory methods start to feel a bit too limited. Static factory objects work well when creation is self-contained like we had, but if creation needs help from another object, then a factory object can be a better fit. I will go to the design examples folder again, and I'll bring in a file called `FactoryObjectPreview`. But the first tier is a small order number generator. It's very simple. It just keeps a number internally and returns the next number when the `GenerateNext` method is called. Then we have the `PreviewOrderFactory`. This is a factory object. Unlike the static factory methods on order, this is a class we will create an instance of. The important part is the constructor of the `PreviewOrderFactory`. It receives an order number generator and stores it in a private read-only field. That generator is the collaborator or dependency. In practical terms, it is simply another object this factory needs in order to do its work. So when this `CreatePickupOrder` is



called, the factory first asks the generator for the number generator, and then it continues with the actual order creation by calling the `Order CreatePickupOrder`. The order number itself is not really important. The important thing is that the creation process now needs help from another object. A static method cannot hold on to that collaborator as instance state, but a factory object can. Let's now update the `program.cs` to use this preview factory. What we see here is that we first create the collaborators of the dependency, and that is the `OrderNumberGenerator`. Then we pass that into the preview order factory. That is the difference, the important difference, let's say, between this and the static factory method. The factory object can now hold onto the generator, and it can use a generator each time it creates a pickup order. This is also closely added to dependency injection by giving an object what it needs from the outside, instead of having it create all of its helpers internally. In larger applications, dependency injection containers can help wire those objects together, but this is already the core idea for that. This object depends on another object, and the dependency is passed in.

Modeling relationships with composition

Module introduction

Hi there, and welcome to this next module where we are going to look at modeling relationships with composition. Let us dive straight in. So far, we've had a lot of focus on the actual individual objects. We have made behavior richer, we've made the creation safer, and we've made the creation clearer as well. But of course, real domain models are not made of isolated causes. They're made up out of objects that relate to each other, collaborate, and depend on each other. So the next thing we'll need to solve is this, how do we model those relationships well? So relationships, yes, because objects rarely solve business problems on themselves. Orders will need customers. Orders need delivery information. Orders need collections of line items, and the way these objects relate, of course, affects the clarity and the correctness of our domain model. So even if the individual objects are already improving, the model can still stay weak if the relationships between them are not modeled correctly. That brings us first to composition, which is often a safe or safer default. For now, I want to keep that idea very practical. Very often, one object should contain or collaborate another object rather than inherit from it. In other words, instead of forcing a type hierarchy, we simply model the relationship directly. That's really the direction we're going to take here in this module. We're going to make the has-a relationship much more, let's say visible. As a sort of preview again, what we see here is that the order owns the delivery details and it also owns the order lines. That's the important design idea. Those relationships are no longer just implied somewhere in the code. They're becoming intentional part of the model,



and that's what we'll create next.

Understanding object relationships

So let's start by understanding object relationships a bit more clearly. There are two common relationship types we're going to look at here. A one-to-one relation means that one object owns or works closely together with one other object, so with the emphasis on one object working with another one. One-to-many means that one object owns or manages collections of related objects. These are the ones we'll mostly use in the model we are building. We will use a set composition, composition we've already touched on, and it means that one object has or contains another object. We're composing that relation. So what we see here is the relationship being modeled directly through fields or properties. This is often what we mean with a has-a relationship. You can see that one-to-one relationship in order. An order has one customer. That is the normal relation I think we can all see. An order in a typical ordering system typically belongs to one customer. Also, an order will probably have one delivery details. So instead of storing delivery-related information as a couple of loose values directly on the order, we now model that as a real related concept. That is already a real improvement because the relationship becomes more explicit. Instead of raw field, we now have a concept in the model that says what those values actually mean together, and here we see the one-to-many site. An order owns multiple OrderLine objects, so this is no longer a list somewhere. It is a real relationship in the model, and that collection is part of the order's internal state that immediately raises interesting design questions like who owns that collection, who can change it, and where should the related behavior live. Not all relationships are the same. Some relationships are just simple references. Others imply strong ownership, and some relationships affect who can create or change the related objects, so we do not want to treat every relationship as if it were identical. That would make our life a bit too simple. We'll see the different relationship types in this module. When an object really owns another one, a few things start to change in how we design things. The first one is access. The owner will often control that relationship more carefully so that outside code does not just get to reach in and change things however it wants. The second one is change over time. The owner may become responsible for how the related object is created, how it changes, and sometimes even whether it is being replaced at all. And the third thing is behavior. Once we have ownership, the logic around that relationship starts settling in a more natural way. Some behavior belongs with the owner, and some will belong with the related concept itself. I may be thinking, how do we recognize ownership? Well, by asking ourselves a few practical questions. Who is allowed to create a related object? Who gets to expose it to the outside world? And who is allowed to change it? If the answer to those questions points to mostly one object, then you are probably looking at a stronger ownership relationship. And once you see that, it becomes much easier to decide where



the relationship should live in the model. So, let's now look at those relationships in the current model in code in Visual Studio. Here is our project like we left it at the end of the previous module. So now I want to look at things from a different angle. Not only are the individual classes getting better, but how are these classes actually connected together? Let me first open the Order class again. We can see here that order is, in fact, already connected to other classes. It has a customer, it has delivery information, and it of course, also has the collection of order lines, so there are already relationships present in the model. Now the design question is whether those relations are, let's say, explicit enough and whether the right object actually controls them, so order, in this case, that we'll need to investigate. The customer relation is already visible as a property. That one is fairly easy to understand. An order belongs to a customer, and the Order class exposes that directly. Then we have delivery. At this point, delivery is still represented by two different pieces of information, that is, the delivery method and the shipping address. That works, but those two values clearly belong together. If the delivery method is shipping, then the address matters, and if the delivery is picked up, then the address should be empty. So what I just said is actually a good hint that delivery may deserve to become its own related object. We'll talk about that soon. And then of course we have the OrderLine collection over here, and that relationship is stronger. The order does not just point to an order line. It owns a collection of order lines. That is why we already have this private backing field here, so `_lines` and the public `lines` property which we exposed as a read-only collection. So already in this class, we see two different kinds of relationships. One order has one customer. One order has one delivery information, but one order also has many OrderLines. Let's also look at OrderLine again. OrderLine is also not just a random object. It belongs inside the order. It represents one order and the quantity for that product in that order, and that matters because if the order really owns its lines, outside code should not be the main place where line objects are created and changed. The order should control the relationship. So these are the relations we already have currently in our applications code. What we have seen is that the quality of the domain model does not only depend on whether the individual classes are done in a decent way. It also depends on whether the links between those classes are modeled in a way that makes sense. When a relationship is well modeled, responsibilities become clearer and misuse will become harder, and the model starts to be more like the actual business scenario we are trying to support. So composition is not just a coding mechanism anymore. It is really a tool that we will use to make the model, let's say, more coherent.

Modeling one-to-one relationships

Let's now zoom in on the one-to-one relationship first. A one-to-one relationship means that one object contains or closely works with exactly one other object like an order that has one delivery details. So instead of thinking of delivery



information as just a few loose values, we now start treating it as an actual related object in the model, even modeling it through composition, not inheritance. What we see here in the code is that the relationship lives directly inside order, and from there, we can see that it is connected to a Customer and to DeliveryDetails. So the structure of the code itself starts telling us something about the domain. That's important because once the relationship becomes visible like this, it becomes much easier to understand what belongs to the order and what not. Now this only becomes truly useful once delivery details then is more than just a bag of values, which it clearly isn't. That's what you see here now. This related object carries delivery-specific behavior as well. It knows about shipping versus pick up. It can validate the shipping address. It can protect the delivery-specific rules that belong to that concept. So we're not just going to extract data into another type for the sake of it. We're creating a related concept that can also carry meaning and behavior, so data and methods. This is where ownership starts affecting behavior placement very clearly. So where we place the code that has to work with the data of the object. It's clear that the order owns the relationship. It decides that delivery information is part of the order, but the delivery details object itself owns the logic that is specifically about the delivery, like shipping changes, address validation, and calculating shipping cost behavior. That is a very useful split because it means the order does not need to absorb every single detail. Composition does not mean pushing everything into one giant parent object. It means giving a related concept their own place in the model so that they can carry the behavior and the rules that belong to them. In C#, it's typical to create small self-contained classes, which is exactly what we are doing here. That is really one of the main design benefits of using composition. So, let's head back to Visual Studio and explore the one-to-one composition in more detail, and now I want to improve the one-to-one relationship around delivery. In the current code, order has a delivery method and the shipping address as we just explained, and that works because those values are really one small concept together. They, of course, describe how the order should be delivered, and we'll bring in a new class, that is delivery details, and this will be then the class that will carry delivery-specific state and delivery-specific behavior. Here, we can now see that delivery details now owns delivery-specific rules here in the constructor. It has a delivery method and a shipping address, but those values are no longer just loose values on order. They are now part of this new class. The constructor is private again. That's intentional. We do not want the outside code to create delivery details with any random combination of values. Instead, we expose two named creation methods for pickup and for shipping. This is the same ID that we used before with static factory methods, and so the factory methods create a valid DeliveryDetails object for that scenario. The validation stays in the DeliveryDetails constructor because this is now the object that understands delivery. That is a great improvement. I've also included ChangeToPick up and ChangeToShipping. It's important to mention because this object is not just data. It can also protect how delivery changes over time. Change to shipping still checks that the address is



not empty, and `ChangeTo Pickup` clears the shipping address because a pick up order should not keep an old shipping address hanging around. So the business logic that is related to this entity is all now really inside this class. Finally, we also have the `CalculateShippingCost` method. `ShippingCost` is a delivery-specific behavior, so of course it fits naturally on delivery details. We're not pushing every piece of behavior into order anymore. We are giving the related concept its own place. And now let me update order so it uses this new object. The class, as you may notice now, is a lot shorter since we have moved indeed a lot of the code into the other class. Again, there's quite a lot of familiar code, but the important change is now here at the top. The order no longer exposes the delivery method and the shipping address. Instead, it has one `DeliveryDetails` property, that is composition. The order has delivery details. It's, of course, not inheriting from `DeliveryDetails`. Absolutely not. It is not trying to become a delivery object. It simply owns a related object that represents the delivery part of the order. Owning is the key word in that sentence. The constructor has also changed. It now receives a customer and a `DeliveryDetails` object, and we can see that both are required, so the order cannot exist without related objects it needs. Notice the syntax that I'm using here customer and then two question marks and then throw new `ArgumentNullException`. That is a compact way of saying if the customer is not null, use it. Otherwise, throw this exception. It keeps the constructor short without changing the rule. The factory methods are still the way to create orders. `CreatePickupOrder` now creates pick up `DeliveryDetails` and passes it into the private order constructor, and it uses for that the `ForPickup` method. `CreateShippingOrder` does the same for the shipping `DeliveryDetails`. Notice that internally they are using the factory methods on the `DeliveryDetails`. How nice it is now. The one factor method is calling the other, resulting in what I think is a very readable code. Further down in the `Order` class, the delivery changing methods also delegate to `DeliveryDetails`, so order still exposes a useful behavior like `ChangeToShipping`, but the delivery-specific rule itself lives within the `DeliveryDetails`. That is a very great improvement. And, by the way, `CalculateTotal` also changed. The order still calculates the total because it owns the whole order, but for the shipping part, it asks the `DeliveryDetails` to calculate the shipping cost, so the larger object coordinates and the smaller related object handles the details that belong to it. Let me now update the `program.cs`, so we can see the relationship at work. What we see now in this scenario code is that the order exposes delivery as a real relationship. I can ask the order for its `DeliveryDetails`, and then from there, I can see the delivery method, the shipping address, and the shipping cost. It is much more expressive than having loose delivery values directly on order. The code now tells us that these values belong together. Then I call `order.ChangeToPickup`. Notice that I'm still going to order here in the call sign, so in the program. The caller does not need to reach into the `DeliveryDetails` and manipulate its properties directly. In fact, it can't because those setters are private, that's even safer. So order now owns the relationship and delivery details owns the delivery-specific rules. That is the balance I wanted to get to here.



Modeling one-to-many relationships

Now, let's move into one-to-many relationships. In this relation type, one object owns or manages several related objects. In our model, that means that the order is responsible for the collection of order line instances. That's, of course, the classroom example to understand this sort of relation, and it is really more than just stoning a list on the order. Once the order owns those lines, then we need to ask ourselves how much control should it keep over that collection. This code shows one answer to that question. Let's take a look. The owner keeps the actual list private, and the rest of the system only gets to see a read-only view of it, so outside code can inspect the lines, but it cannot just take over and manipulate the collection directly. That's an important design change because it lets the relationship stay visible without giving up ownership, a big change. And if the order really owns that line collection, then the order should also control how that collection changes. That means deciding how products are added, how quantities are adjusted, and how lines are removed. The outside world should not bypass this rule by editing the collection directly. The Order class will have rules to manage what goes in and out, and that's where we need to go. Ownership in a one-to-many relationship should show up in the behavior, not just in the data, and that's what we see here. Order becomes the place where those changes are coordinated. Instead of letting outside code edit lines directly, it exposes behaviors like `AddProduct`, `RemoveProduct`, and `ChangeProductQuantity`, so the collection is no longer the public editing surface, meaning that it is not a thing that we can directly interact from the outside. The order itself is now that surface. At the same time, `OrderLine` still remains a meaningful concept with its own behavior, so it's a type that also contains business logic. That part is important because otherwise the lesson could sound like the parent controls everything and the child does nothing. That is not what is happening here. `OrderLine` still knows about its own quantity-related logic. The difference is that the creation and imitation are restricted so that the order still governs the relationship as a whole. Like we did with the one-to-one relationship, let's now return to Visual Studio and explore this more in code, and the important idea here is ownership, and order owns the order lines. That means that the outside world should not be the main place where old lines are created and changed. The order should be the object that controls that collection. So I'll start in here in `OrderLine` and tighten that class a little bit. What we now see is that `OrderLine` still owns its behavior. It still knows how to calculate its line total to `GetLineTotal`. It also still knows how to increase its quantity and change the quantity, but there is an important access change that I've made. The constructor is now set to internal. Increase quantity, as we see here, is now also internal and change quantity same thing, that is an access modifier that we hadn't looked at before. Internal basically means that the member can be used from inside the same project, more precisely from within the same assembly, but not from outside of it. So the domain project can still create and change order lines internally, but



the console application cannot just say new order line anymore. That is a small, but useful restriction. We are saying OrderLine is part of the old relationship inside order. Outside code should go through the order behavior instead. That is a lot safer and cleaner. Notice that the quantity also has a private set. We've used that before, but I think it's worth calling out again because this is exactly the kind of thing that supports that ownership that we're looking for. Outside code can, of course, still read the quantity, but it cannot assign a new quantity directly. If the quantity changes, it has to go through behavior, and the quantity methods still contain business rules. IncreaseQuantity rejects amounts less than or equal to 0. ChangeQuantity does the same, but for the new quantity. So this is not just access control, the object still protects its own small piece of behavior. And now I'm going to update order, so it really becomes a public editing surface for the OrderLine collection. Let me show you how we can see who controls the relationship. The actual list is private. That was already the case. Outside code only sees the lines collection as this IReadOnlyCollection. AddProduct is still the main way to add something to the order. Inside that method, we use first or default to look for an existing line with the same product code. That's that link method. We've encountered it before. That is why we can then say existing line is not null. This is a readable way to check whether we actually found a matching line, and if we did, we increase the quantity on that line and return it, and otherwise, we create a new OrderLine. I've also added the ChangeProductQuantity. This is now new behaviors callers can use when they want to change a quantity. The method looks up the line by product code. If the line is not found, the order rejects the operation, but if it is found, the order tells the line to change its quantity. So the order is coordinating the collection while the line still protects the quantity rule. I've also added the GetProductQuantity and ContainsProduct methods. These are small created methods, and they let the caller ask useful questions without exposing the real list as the editing service. That is what I wanted to avoid. In the GetProductQuantity, you can see this line here. That means if the line is not null, return its quantity, otherwise just return 0. It is a compact syntax with a very common null check. Let us now update the program.cs again to show this relationship from the outside. So what we now see here is the use of the one-to-many relationship. We create an order, we add two products, and then add the cherryPie again. The results, of course, that adding the same product twice does not create a duplicate line. Then we ask the orderer a few questions. Do you contain a cherry pie? What is the cherry pie quantity? What is the subtotal? We're not reaching into the internalist to figure it out ourselves. We're asking the order because the order owns the relationship. After that, we call the ChangedProductQuantity. The caller does not grab the OrderLine and assign the quantity directly. It asks, instead, the order to perform the change. The order will find the right line, and the line applies its own quantity rule. The output shows one OrderLine for cherry pie, not two separate cherry pie lines. It also shows that the quantity is changing through the order behavior. That is the point of one-to-many ownership. The collection exists, but the owner controls the collection changes.



Constructor injection: Passing dependencies

Up to now, we have mostly looked at relationships inside the domain model like the one with order and delivery details, but objects can also relate to other collaborators they need in order to do their job. So relationships in a model are not only about domain entities owning other domain entities. They can also be about an object depending on another object to perform its work. That is where a constructor injection comes in. So we're looking here at an object depending on another object in order for it to do its work. Now what does constructive injection mean in practical terms? It means that if an object needs another object in order for it to do its job, the dependency is passed in when the object is created. We have touched on this already before. Think of the number generated being needed for the order creation, or maybe a logger being required to write orders to a log file. So instead of the object secretly creating what it needs itself, the dependency is made explicit upfront, and it'll be given read injected from the outside. We receive that in code. The CheckoutApplicationService receives a catalog in its constructor, so the service is not responsible for creating that catalog. It simply says, I need this in order to work. It is explicit about this. The dependency becomes visible in the type's setup. You can look at the constructor and understand immediately what the service depends on. At the call site that is also visible, that's important. The caller decides which catalog to provide and passes it to the service, so the collaboration itself stays explicit. The service is not hiding where its products come from. That makes the code easier to understand, and it also usually makes the object easier to test because what it depends on is already clear and externally provided. If the object cannot do its job without the collaborator, which is in other words, required for the object's setup, then that collaborator should probably be required. If the object is perfectly fine without it so that it's optional, then forcing it through the constructor might not be the best idea. Keep in mind, constructive injection is just a design pattern for making required collaborators explicit. It does not require a DI container. It does not require some DI framework. It is simply a way of saying this object needs this other thing in order to work. So, let's see how we can pass collaborators through the constructor, and the scenario I'll use is a small application service. This service will coordinate a checkout flow. It will not take the domain rules away from the domain model, that's important. The order still will add the products, calculate its totals, and owns its relationships, but this service can orchestrate that use case. First, I'll add one small supporting cost for this count calculation. This will not be the main constructor injection example yet, but it gives the checkout flow something practical to do. I will add the Order Calculator class in the pricing folder, and let me paste in the code for that. What we see here is a small Calculator class that receives a loyalty discount percentage in its constructor, so the percentage becomes part of the calculator setup. The constructor even also validates that the percentage is between 0 and 100. Then the CalculateTotalWithDiscount receives an order and uses the order's own calculate



method. That is important. This calculator is not rebuilding the order pricing from the outside. It still relies on the order to know its total. Now I will add two small classes in the console project inside of an application folder. What I've added here are records which are useful for small data carrying shapes. Records are very similar to classes, and we'll learn about them more later in the course. For now, see them as a simple type of class. The `CheckoutLineRequest` represents what the caller wants to add to the checkout, a product and a quantity. The checkout summary represents the result we want to return after checkout. Now we're just adding these to keep the `program.cs` cleaner. Instead of passing loose values everywhere, the application flow gets small named objects that describe the request and the result. Now, let's add the actual service finally. Here, we now can learn more about constructor injection. At the top here, the service has a private readonly field called `catalog` is `_catalog`. The catalog is a dictionary of product by product code. It is private again, meaning that callers can not manipulate it directly. I'm also using `readonly` here, which means that the field is assigned during construction and then not replaced later. An important part is the constructor. `CheckoutApplicationService` receives an `IEnumerable Product` called `catalog`, so the service does not create a catalog for itself. It does not new up the products internally. It says, if you want to create a service, give me the product catalog I need, and that is constructor injection. A required collaborator is passed into the object when the object is created. Then inside the constructor, the `ToDictionary` turns the sequence into a dictionary, so a key value collection. The key is `p.ProductCode` so that we can look up products quickly by code, and the second parameter here means that the lookup ignores casing. Then in the `CheckoutShipping` and `CheckoutPickup`, the service creates the right type of order using the factory methods we already built. It also calls the `AddLines` method, and `AddLines` resolves each product from the catalog and adds it to the order. Notice where the responsibilities are. The service is coordinating the checkout flow. It knows how to translate the product code request into real products from the catalog, but does not take over the order rules. `AddProduct` still belongs to `Order`, `CalculateSubtotal`, and `CalculateTotal` also still belongs to `order`. Shipping cost still belongs to the delivery details. That is the important distinction. Constructor injection makes the dependency explicit, but it does not mean that we move everything into a service. Now let's update the program so we can use the service from the call site. What we see here is the call sign for constructor injection. First, we create a small catalog with two products, then we pass that catalog into the new `CheckoutApplicationService` over here. So the caller decides which catalog the service will use. That makes the collaboration clearer. If we later wanted to use a different catalog in the test or a catalog coming from another source, the service already supports that chip because it receives the catalog from the outside. Then we create a customer and run a shipping checkout. The checkout request contains two lines, and we also pass in a loyalty discount percentage. The service then returns a checkout summary and `program.cs`, then prints the result. After that, we run a pickup checkout as well, so the same service can coordinate both checkout scenarios while the domain



model still owns the domain rules. Let's run it and see that everything works, and there we go. The ShippingCheckout, which is the injected catalog to resolve the products. The order calculates subtotal and total, and delivery details contributes the shipping cost. The discount calculator also applies the configured discount, so the important takeaway here is this. When an object needs another object to do its job pass that dependency in through the constructor and make the collaboration explicit.

Composition vs. inheritance: Making the choice

Now for the final part of this module, let's look at composition versus inheritance. It is something I get asked about often. What I want to talk about briefly here is really about making the right choice for the kind of relationships we are modeling here. There are two very different questions at play here with these two approaches. Composition asks, does this object have or own another object. Inheritance asks, is this object a more specific kind of another object. That's an important difference. Our current model is clearly using composition. An order has a customer, it has delivery details, it has many OrderLine objects. So what we see here is a very direct has-a relationship. That is exactly what composition is meant for. We are modeling collaboration and ownership, not pretending these things are all versions of the same kind of type. Inheritance is different. It is useful when one type truly is a more specific kind of another type that can absolutely be a valid design tool. But it's a stronger commitment, let's say, and a riskier one if the relationship is not really that kind of specialization, and for the relationships we looked at in this module, it is simply not the right fit. So this is why composition is usually safer. It keeps the collaboration visible, it keeps the relationship explicit, and it allows the objects to work together without forcing them into the same hierarchy, and it usually leaves the design more flexible over time. That's a very strong reason to treat it as a safer default for domain relationships like the ones we are modeling here. Order is not a kind of DeliveryDetails. Order is not a kind of OrderLine. It works with the objects by owning them, and I think that's a very good place to be in right now because what this module taught us is that object-oriented design is not only about designing individual classes, it's also about modeling the relationships between those classes in a way that makes ownership, collaboration, and responsibility clearer.

Replacing conditionals with polymorphism

Module introduction

Hi there, and welcome to this next module where we are going to look at



replacing conditionals with polymorphism. So the next problem that we'll tackle is variation. Let me explain because it may seem a bit cryptic. We have already improved how objects relate and collaborate through composition, but it does not solve every design problem. Objects may still behave differently depending on the kind of thing they are, and one of the easiest ways to model that is with conditionals, so ifs or switches. The problem is easy does not always mean strong, and as usual in this course, we are interested in what happens after creating the first working version. A variation in behavior is very easy to model with flags and conditionals. It is basically, let's say, the default approach, and that's exactly why it becomes a design issue very often because the first version usually works fine. A delivery method gets stored somewhere. Then we use an if statement that branches on it. If delivery method equals pickup, for example, and the result is correct. So the issue is not that the code will fail. The issue is that the design starts, well, let's say, centralizing too much variations in one place, several if elses if you wish, and that will create some design pressure later on. This is where polymorphism comes in. We've already seen polymorphism, but it is not just a language feature that you are supposed to recognize and work with. It is instead a design tool. It gives us a way to move a variation into the model itself instead of keeping it trapped, let's say, in a growing set of conditionals. That is the design improvement we're going to make in this module. So to be clear, what problem are we going to solve? Well, what we see here is a typical conditional solution. The method checks the delivery method and then chooses the shipping cost. Yes, that'll work, but it also means that this one class is going to be deciding on the delivery variation. It knows the cases and it also just chooses the behavior. That is what we're going to focus on next. So instead of asking an if flag, let's say what to do, we'll let the object decide. If the behavior varies because the object is different, so like the delivery type, then very often the object itself should own that difference that we need now to make part of the model.

The problem: Conditionals everywhere

So let's start by looking at the problem a bit more closely. As said, the problem is conditionals everywhere. A conditional solution feels often very reasonable when there's only one or two cases. That is often how things start, right? It is still important to say this because otherwise it may seem like we're criticizing a normal code too quickly. So in the beginning, it all feels fine. One class will store the delivery flag, one method branches on that flag. The problem is how easily this style grows as the model starts doing more. This is what the conditional design looks like in code, and this is the normal code, to be clear. One method decides on the behavior for the different delivery types. The variation is centralized in this conditional. An object is still dependent on a flags or an if to know what it should do. And once you start seeing that, you can already feel that this isn't perfect. The method is not just doing one thing. It is also acting as the place where multiple cases are interpreted. That can be a red flag. So the real



problem is not that the code isn't working. Shipping can be calculated. Pickup can be calculated. So from a purely functional point of view, good. But the design is weak because one class owns too much variation, and new cases will grow the conditional, and class, or related logic, often stay coupled to that same flag to that same conditional. So the weakness is really an architectural weakness, and conditionals spread very easily. Once one flag enters the model, it often becomes the first of many. One branch appears in one method, then another method needs the same distinction, and then a third place starts carrying about the same variation. So what began as one harmless conditional can now slowly become, let's say, a repeated way of thinking about the model. That's where this becomes a design concern. That is why when one object keeps checking a flag to decide what it is, the model is often asking for a different design. That does not mean that every if is wrong. Again, not what I am saying. It means that when behavior is varying based on the same distinction again and again, the distinction probably belongs in the model itself, and for that problem, we could actually use polymorphism. So as we've done before, before we improve on things, let's make the conditional delivery model visible in code. So I'll add a small comparison class in the design examples folder again called conditional delivery comparison. What we see here is the, let's say pre-polymorphism style. Conditional delivery details stores a delivery method string and then calculates shipping cost, checks that string to decide what to do here. So if delivery method is shipping, it returns 5, otherwise it returns 0. That is simple and it works. This is not bad because it would fail. It is weak though because one class is now responsible for deciding every delivery variation. We just have two variations here, but it can be a lot more in a real system. Also, notice that the delivery method is just a string here that makes the problem even more visible because the object is depending on the value that has to be interpreted, but even say it would be an enum, the same design issue would still exist. The object would still be checking a flag to decide which behavior to run, and that is exactly the part we're going to improve in this module. Let's take a look at the program.cs and update it so that we can see the behavior before we refactor things. Here, we are now creating one shipping delivery and one pickup delivery using the old conditional model. Then we can CalculateShippingCost on both. The output will be correct. Shipping costs 5 and pickup costs 0. So again, we're not proving a functional bug here. I'm proving a design smell. The interesting part is also that both objects are instances of the same class. The only thing that makes them behave differently is the delivery method string inside the object. So this class has to ask what delivery method am I, and then it chooses the behavior. Let's run this now. As expected, the numbers are correct, but the observation is that one class decides every delivery variation through a flag check. If we add express delivery, international delivery, local courier delivery, or anything else, this will push us towards more branches in that same place. It can be a small problem now, but it can easily become a much bigger one. More delivery types that get added mean more branching. More branching probably means more duplicated decision logic, and the model becomes harder to extend without touching the existing code over and over



again. So instead of each delivery type knowing how it behaves, one class keeps checking which delivery type it is dealing with and deciding thus what to do. That is that centralized variation, and that's the weakness we need to fix.

Introducing abstraction

The solution to this problem can be found in the concept of abstraction. An abstraction is a shared concept we can program against. It gives us one common ID while still allowing different concrete cases to exist underneath it. That is the direction that we're moving in now. We're looking for a shared concept that can represent the concept of delivery in our case in a cleaner way. So, instead of having one class decide all the cases, we introduce a shared base concept. That base concept represents what the cases have in common. That is moving up a level, so that is abstraction. Then, more specific classes represent the actual variations. So instead of having one class decide all the cases, we will need to introduce a shared concept. That base concept represents what the cases have in common, that is moving up a level. That is the abstraction. Then, more specific cases represent the actual variations. So in our example, pickup and shipping are both kinds of delivery. And from a C# and modeling perspective, we are turning one class with branching logic into a shared concept with specific cases. Here we see a regular base class version of that ID. Now, a base class is just a parent class that other classes can inherit from. In this case, delivery becomes that shared concept. It defines the members that derived classes can reuse or specialize. And for that, we also see the virtual keyword being used here. If that keyword is new to you, it simply means that the behavior can be replaced in a derived class, replaced by a more specific version, that is. Here we then see the first derived class, PickupDelivery. A derived class is just a more specific class that inherits from the base class. We indicate that this class derives from that base class using the colon and then the name of the base class. Then, the important keyword here is override. That means that this class is replacing the inherited behavior with its own version, with specifying indeed a new version, an override if you will of that base method. Here, in this case, that is they CalculateShippingCost. So the pickup version is still a delivery, but it now provides its own specific implementation. And here we can see another derived class, the ShippingDelivery. So again, it is still a delivery, but again with a different behavior. That's important because it makes the variation explicit in the model itself. The differences are no longer hidden in a conditional. They are owned by the different concrete types, and that is a big difference. And we do want to make the keywords clear before we move on. Virtual basically says that the class allows derived clauses to replace that behavior. Override is what the derived class uses to actually provide a different implementation. And used together, they let the same method called behave differently depending on which concrete object is behind it. So let's make that regular base class approach visible in code. I will start with the regular base class. This is useful as a start because it lets us use



virtual and override before we go any further. I will add the Delivery class in the Orders folder. We see here a regular class, and it's called Delivery. This is the shared base concept. It has the DisplayName property and the CalculateShippingCost method. The important keyword here is virtual. Virtual means that the derived class is allowed to replace this behavior. So the base class provides a default version, but it also says a more specific delivery class may override this. At this point, the base class can still be called directly. We'll come back to that in the next demo because this is not the best final design. But for now, this makes the inheritance mechanics very visible. Now, I'll add a pickup delivery type in the form of the PickupDelivery class. PickupDelivery inherits from Delivery, and we can see that through the colon syntax, so PickupDelivery : Delivery. That means that pickup delivery is a more specific kind of delivery. And then we see the override. Override means that this derived class is replacing the virtual behavior from the base class. So DisplayName becomes Pickup, and CalculateShippingCost returns 0. That is already different from the conditional model. Pickup behavior is no longer one branch inside the generic Delivery class. And then the shipping version too, so I'll bring in the ShippingDelivery class. ShippingDelivery follows the same idea. It also is a delivery, but it behaves differently. It has a ShippingAddress, it validates that address in the constructor, and its shipping cost is 5. So now we have two concrete delivery types. And now, I will update the Program.cs to call this through the base type. What we see here now is the first polymorphic usage. The variables are typed as delivery, but the actual objects are different. One is a plain delivery, one is a pickup delivery, and one is a shipping delivery. The PrintShippingCost method is really important. It does accept a delivery. It does not accept a pickup delivery or a shipping delivery specifically. So it can call DisplayName and CalculateShippingCost through the shared base type. And because the derived classes override that behavior, the same method call can produce different results. The output shows three lines. Generic delivery gives the generic default behavior, pickup delivery gives pickup behavior, and shipping gives shipping behavior. So that seems to work. We have a base class. We have derived classes. We have virtual behavior, and we have override behavior. But the generic delivery object, well, that is still a bit strange. In our domain, there is no useful thing called just delivery with some made-up default behavior, so delivery is a shared concept. The real objects are pickup and delivery. So we should definitely improve on this. Now, this is also the point where we should correct the design a little bit more. We actually do not want a generic concrete delivery type with some made-up default behavior. That suggests that there is a catch-all delivery type. And in this model, that is not what we mean. We want a shared concept, absolutely, but not a generic concrete default. That's where abstract is actually the better choice. So, what does abstract into the abstract class mean? An abstract class is a shared concept, but it is not meant to be created directly. So it cannot be instantiated. It can't be newed, that is. It can define members that derived classes must complete, but it does not pretend to be usable as a concrete version itself. So in this design, delivery becomes a shared concept



only. You can work through it, but you cannot create a generic delivery. That fits the domain much better. That is also why abstract is better here than a regular base class. A regular base class can imply that a generic concrete version exists. And since it is a regular class, it can be instantiated, which might not make sense since it is modeling an abstract ID. An abstract base class, on the other hand, says this is a shared concept, and a concrete behavior belongs in the specialized types. It doesn't allow for instantiation and thus requires that inheriting concrete types complete the behavior. That is, in fact, a better fit for this delivery example because the idea of the shared concept is real, but the generic default instance isn't. Let me show you this in the next demo in code. The regular base class helped us understand the syntax, but it also allowed us to write a new delivery, and that is not really what we mean in the model. We want delivery to be a shared base concept, not a concrete catch-all object. It needs to change to something that exists in real life. So, I'll change delivery into an abstract base class. What we now see is that delivery is abstract. Abstract means that this type represents a shared concept but cannot be created directly. So we cannot say new delivery any more. The DisplayName property is abstract as well, and the CalculatedShippingCost is abstract too. That means that the delivery is saying every concrete delivery type must provide these members. The base class defines the contract, but it does not invent a fake default. That fits our domain much better. There is no generic delivery behavior in our model. There is pickup behavior and shipping behavior, and there might be other delivery types as well. I've also added two static factory methods in here, ForPickup and ForShipping. That gives us name creation methods on the abstraction, so callers can ask for delivery for pickup or delivery for shipping without newing up the concrete classes directly. Now that part is not really related to polymorphism, but it keeps the creation story consistent with what we have been doing in the previous modules. The pickup and shipping classes, well, they do not need any big change here. They already override the members. The main difference is that they are now completing the abstract contract instead of replacing a default implementation. Now, let me update the Programs.cs. Now the code no longer creates a generic delivery as we've seen. It is not even possible to new it up. Then, here in the code, we create the pickup and shipping through the factory methods. Notice the variable types. Both variables are still typed as Delivery. That is important. The caller now works with the abstraction. The concrete object behind the variable supplies the behavior. So, when we call PrintShippingCost over here, the method still accepts delivery. It does not care whether it receives a pickup or shipping. It just calls the members only abstraction and a concrete object response. And the output shows pickup and shipping behavior, but without the generic default case. So now, delivery is a shared concept and just that. Pickup delivery and shipping delivery are the concrete behaviors.

Replacing conditionals with polymorphism



Now, we are finally ready to make the real design move, and that is replacing conditionals with polymorphism. The idea is the following. We'll work with a shared base type, while concrete objects decide how the behavior is actually being carried out. The call can stay the same, but the result will differ depending on which concrete type is actually behind the abstraction. That is really what polymorphism means in practice. We use the same call but on a different concrete type. We get different behavior, therefore, based on the concrete type, that is, and we can thus remove the conditional. This is our before version again. One class is deciding on all delivery behavior. The variation literally lives in this conditional. The object checks the state to figure out what to do. So, this decision happens centrally. That's the thing we're trying to remove. The model still asks one place to decide on all the cases. And this is where we're going. Now the shared concept stays in one place, but the varying behavior becomes part of the abstraction. The concrete delivery types provide the actual implementations I showed us next. So instead of having one class that decides everything, the different delivery types own their own behavior. So here we see the inheriting types concretely. We have `PickupDelivery` and `ShippingDelivery`, and they both respond to the same call, that is the `CalculateShippingCost`, but they do it in a different way. The method really does not change, but the call does not need to branch out. The object itself decides. That is polymorphism doing the real design work. It's a cleaner model of working. Notice, indeed, we're using the colon and then the override keyword here as well. So, without polymorphism, we ask what kind of delivery this is, branch on that answer, so add that conditional, and then choose the behavior. Once we introduce polymorphism, we work through the shared delivery type and let the concrete object respond. That is a much better design because the variation moves inwards into the object. And this is also where the runtime behavior then comes in. The variable may be typed as a delivery type, but the actual concrete object at runtime might be a shipping delivery. So when the method is called, the shipping version is running. C# and .NET will pick the right one for us. That is, in fact, all that runtime polymorphism really means here. The abstraction is what the caller sees. The concrete object is what actually is responding. That is why the same call can behave differently. So, let us refactor the delivery logic and make that visible in code. So far, we have built a delivery abstraction next to the model. Now we need to use it in the real `Order` class. This is the actual refactor we'll do here to show a couple of things. Now, at this point, `Order` used delivery details, and delivery details still calculate the shipping cost by checking the delivery method. That was better than loose properties, but it kept the variation in this one object. Now we'll let `Order` work with delivery directly. I'll paste in a new version, and I'll take you through the important changes. Here we can see that `Order` now has a `Delivery` property, and the type is indeed the abstract base type `Delivery`. So not `PickupDelivery` and not `ShippingDelivery`. That is what allows the order to work with any delivery implementation that fits the abstraction. I've also changed the constructor. It now receives a delivery object. And just like before, the constructor protects its required collaborators. `Customer` is required, and so is



Delivery. The factory methods now create the right delivery object. `CreatePickupOrder` uses `Delivery.ForPickup`. `CreateShippingOrder` uses `Delivery.ForShipping`. So the creation scenario is still named, but the delivery variation is no longer represented by an enum and a conditional. Then look at the `ChangeToPickup` and `ChangeToShipping`. Then I replace the delivery object with a concrete delivery object. So if the order changes to pick up, it receives a pickup delivery. And if the order changes to shipping, then it receives a shipping delivery with the new address. And finally, `CalculateTotal` is perhaps the most important one. It returns a subtotal plus the `Delivery.CalculateShippingCost`. There is no if statement here anymore. Order is not asking are you a shipping or a pickup or delivery? It just asks the delivery object to calculate its own shipping cost. That is really the refactor in one line there. Order stops interpreting the delivery type and starts calling behavior through the delivery abstraction. Now since the `CheckoutApplicationService` uses delivery information too, I'm getting some errors here, so I'll update that next. The important change in this service is actually pretty small. The summary now uses the `order.Delivery.DisplayName` and `order.Delivery.CalculateShippingCost`. Again, no delivery flag check, no type check, and that is really the sign that this is paying off. The polymorphic design is not only cleaner inside the order, it also makes the calling code cleaner. Let's now also update the `Program.cs` with another scenario. What you here now see is that the public order API did not become harder to use. We still create a shipping order and a pickup order through the named factory methods. We add product side before, and then we print the delivery type and the total. The important difference is inside the model. Shipping total includes shipping cost because the order has a shipping delivery object, and pick up total does not because the order has a pick up delivery object. There is no branching here anymore. Also, in `Order`, there's no branching anymore. The delivery object supplies the correct behavior.

Polymorphism at the call site

Now, let's look at the polymorphism at the call site, so from where this is being used. This is really where the improvement becomes visible outside of the types now. Now polymorphism becomes useful when the calling code depends on the abstraction, on the shared abstraction, rather than on a specific concrete type. That way, the same call can work with different concrete objects without needing to change. The call depends on the contract, not on the exact implementation. Here we see a good example of that. The method accepts `delivery`. Notice not pickup delivery nor shipping delivery. So the method does not care which specific delivery type it receives. It just uses the shared contract. That's merely the power of abstraction. The caller becomes simpler because it no longer needs to know every concrete case. The interesting thing happens at runtime. The variable is typed as `Delivery`, but the actual object may be a `ShippingDelivery`. And when that method runs, the shipping implementation



is the one that will respond. So the call stays stable but the behavior varies. Let us now look at this in code. So from where we are using this. The caller can work with the base type and does not need to know every concrete case. So let me update Program.cs with a full scenario. There are a few things going on here, but they all point to the same design improvement really. So at the top, I still kept the pre-polymorphism example using the conditional delivery details, and that gives us a contrast. Then, we run the real checkout flow through the CheckoutApplicationService. The service creates a shipping and pick up orders, but the service does not inspect the delivery flags either. It uses the order, and the order uses the delivery abstraction. The most important part is here. We create a shippingDelivery and a pickupDelivery, but both variables can be passed into the print shipping cost. The method accepts a delivery. It does not have overloads for pickup delivery and shipping delivery. It does not use an if. It does not use a switch. It simply call the Delivery.DisplayName and the Delivery.CalculateShippingCost. And at runtime, the correct concrete object will respond. That is polymorphism at the call site. As we have seen, callers can stay stable while behavior varies behind the abstraction. That is a very strong improvement in our design because it means we can add or change behavior without forcing all the usage code to become more complicated. We don't need to add more conditionals when using our types. That is the real payoff, the real benefit of this refactor.

Protecting your abstraction with sealed

As a final topic in this module, let's look at protecting the abstraction with sealed. So first, what does the sealed keyword do? Well, good question. Let's start with that. A sealed class is a class that cannot be inherited any further. The class is still usable like a normal class, but inheritance stops here basically. So if sealed is new to you, the practical meaning is this. The concrete type is finished as an inheritance endpoint, meaning no other class can inherit from this class. Now you may be thinking, why is this useful? Well, good question. Without sealed, other code can extend the behavior in ways that the original design did not intend for. And over time, that can weaken the model. So sealed is a way of saying this concrete behavior is complete, and inheritance should stop here. You can see it like this. Keep the shared abstraction open while a variation still belongs there. But seal concrete types whose behavior is already complete and should not be extended any further in the model. That gives us a nice balance between flexibility and protection. We keep the right places open, and we close the parts which are finished. I think that our delivery model is a good example of that. ShippingDelivery already expresses a complete delivery behavior inside of our current model. So sealing that class makes sense. We're not expecting more subclasses of ShippingDelivery inside this design. So here, sealed is helping us to say something meaningful about the completeness of the type. It's a design statement really. This leads me into the idea of invariants. An invariant is simply a



rule that should always stay true in the model. It is something that the design is supposed to keep consistent. And if later code can break that rule, well, then the model becomes weaker again. So the reason this comes up here is that inheritance can sometimes reopen behavior that we intended to complete. That is why sealed can help protect important invariants. If other code cannot subclass this finished concrete type, then completed behavior cannot quietly be changed through inheritance. That deduces extension by mistake and helps protect important rules the model depends on. So in that sense, sealed is not just a keyword; it is a design protection mechanism. It helps preserve the meaning we already had built into the model. So let's make that visible as well. Using sealed in our model is the last demo for this module. And I want to protect the concrete delivery types a little bit more. We now have the right abstraction. So delivery is abstract, and the concrete types like PickupDelivery, they provide the behavior. But that does not mean that every concrete type should remain open for more inheritance. The ShippingDelivery already represents a complete behavior in this model and so does PickupDelivery. We're not designing subclasses of PickupDelivery or subclasses of ShippingDelivery here. So, I'll add sealed to the concrete delivery classes. So the only key word I want to focus on here is sealed. PickupDelivery is still a normal class in the sense that we can create it and use it. It still inherits from Delivery. It still overrides DisplayName and CalculateShippingCost. But sealed means inheritance stops here. No other class can derive from PickupDelivery. That is useful because PickupDelivery is already a complete pickup behavior in this model. If later code wants a different kind of delivery, it should probably create another delivery implementation, and it should not secretly extend PickupDelivery and change what pickup means. I've also done the same with ShippingDelivery. It's also sealed, but it can still be used as before.

Defining capabilities with interfaces

Module introduction

Hello there, and welcome to this next module where we're going to look at defining capabilities with interfaces. Interfaces are a very important and very commonly used concept in C# and .NET. So this module, where we will learn about their importance in designing a good domain model, is a very important one. Let's dive straight in. So far, we used abstractions and polymorphism to move variation into the model. That's what we have seen in the previous module. But we will also need objects to be able to collaborate through capabilities without forcing them into the same inheritance tree, and that's different compared to what we had in the previous module. What I mean applied to our little domain here is this. Delivery, pickup, and shipping really were part of one shared delivery concept. But with discounts, we may not need that



same kind of hierarchy. We may just need something that can apply a discount but have this shared capability. How can we solve that? So why do we move beyond base and abstract classes? Abstract classes are useful when we have one shared domain concept, when some shared implementation belongs in the base type, and where the types are really part of a hierarchy. But like I said, sometimes we need something different. We need a shared capability without a shared implementation this time and without forcing the unrelated classes into the same inheritance chain. That is where interfaces will come into play. Interfaces are typically described as a contract that defines indeed what objects can do, what the effect must adhere to. That we will use here in our domain as well. An interface does not define the full identity of a type. It defines a capability that the rest of the code can rely on because it is in the contract. So instead of asking what exact kind of object is this, which is what we did in the previous module, we can ask can this object do this thing I need here? Let me quickly start with a small introduction on interfaces from a syntax perspective again first. Here, the interface is now called `IDiscountPolicy`. It declares one member, `ApplyDiscount`, which receives a total and returns a decimal that is the discounted total. It does not contain an implementation. That is typical in interfaces, although since the more recent version, it is possible to also include this from a C# perspective. But it's possible I won't use it here. So, what we see here is not a full discount class. It is the contract for a capability. Any class that can apply a discount can implement this interface. And here we see a class implementing that interface. A loyalty discount implements indeed `IDiscountPolicy`. The syntax is very familiar to what we had with inheritance because we can use the colon and then the name of the interface we are implementing. But the meaning is different. We're not saying that loyalty discount is a more specific kind of base class. We are saying it promises to provide the behavior required by the interface. So the class fulfills the contract. Let's make that first interface a visible in code as a means of recapping interfaces from a syntax point of view. What I want to show you in this demo is just the basic mechanics first, how to declare an interface and how a class can implement it. Now I want to start very small here. Let's just review the syntax as to the basic ID. I'll add an interface for a discount capability. So it's not a full checkout flow yet and also not multiple discounts yet. So just first a contract and I'll add a new file under pricing for that, and that'll be the `IDiscountPolicy`. So what we see here is an interface called `IDiscountPolicy`. By convention, interface names in C# often start with an I, and that we can see here. So `IDiscountPolicy` basically means this is the discount policy capability. The interface also declares one member. That is the `ApplyDiscount`. It receives a total and will return a decimal. There's no implementation here, so this is not saying how a discount is to be calculated. It's only saying any class that implements this interface must be able to apply a discount to a total. So that is the contract part. The caller can rely on the fact that this method exists without needing to know which concrete class is behind it. Now, let me add a first implementation. I'll bring in the loyalty discount. Here is now the `LoyaltyDiscountPolicy`, and it implements the `IDiscountPolicy`. The syntax



uses a colon, just like inheritance, but the design meaning really is different. We're actually saying it supports the discount policy capability. And because it implements the `IDiscountPolicy`, it has to provide an implementation for `ApplyDiscount`. Here, I'm keeping logic fixed at 10%. So, now we have the interface, and we have a class that fulfills the contract. Let's now use that in the `Programs.cs`. What we see here is the important idea around the usage of it. The variable is now typed as an `IDiscountPolicy`, but the actual object is a `LoyaltyDiscountPolicy`. So, the caller is no longer talking directly to the concrete type. It is talking to the capability, let's say. The only thing the caller needs is the `ApplyDiscount`. And that is exactly what the interface promises. The important thing is that the calling code can now work with the discount capability is separate from the concrete discount class, and that is the interface `ID` really. Define what an object can do, and then let one or more classes provide that behavior. Now, apart from being a contract, interfaces really are about capabilities. That's how I want us to look at them in this module. An interface describes something that an object can do as part of the contract, that is. So it's not just a technical C# thing. We'll rely on it in the design of our model. and the design idea is this capability matters to the model, and so we will give it a name. So, we'll use interfaces when you want to model what objects can do and not just when you want another layer. I'll explain in more detail what I mean with the what they can do. It's an important warning because interfaces are very easy to overuse. Creating an interface for every class does not automatically make the design better. Sometimes, it makes the code more indirect. So we want to use interfaces where the capability means something, where it makes sense to use one.

Interfaces as capabilities, not data contracts

So what we need to do is looking at interfaces as capabilities, not just data contracts. That's an important distinction because it's very easy to create an interface that technically works but does not really help the design very much. That's exactly what we're after in this course, right? So in this part, I want to look less at the can we write an interface and more at what does this interface mean in the model? Let's take a look. An interface can definitely improve the model, but it can also add very little. If an interface only says like these are the values you can read, then the rest of the code still needs to decide what to do with those values. So yes, we then have an interface, but we may not really have moved responsibility again into the model. That's not what we're really after. In a domain model, an interface becomes much more useful when it describes behavior that the caller can rely on. Here we see an example of what I'd call a data-shaped interface. It tells us something has a discount percentage. And that may be true, of course, but it does not tell us what this object can do. So some other piece of code still has to read that percentage, calculate the discount, and decide how the rule is applied. In other words, the interface



exposes data, but the real work lives somewhere else. It's not adding any extra behavior. That's why this interface is not very strong as a domain concept. Now, let's compare it with this one here. Let's call this one a capability interface. Here the interface says this object can apply a discount. That's more useful because the caller does not need to ask for the percentage and then perform the calculation itself. No, the caller can simply say apply the discount. And so what we see here is a different kind of interface. It is centered around behavior. It is something the object can do. It adds a real responsibility. That is more valuable in a domain model. So really, the difference is in what the interface means. The data-shaped interface mostly says here are some values that you can read, but it often means that the caller still has to assemble the behavior somewhere else. Capability interface says here is a responsibility this object can perform. And that's the goal again. The behavior stays with the object at hand, which is much stronger because the caller can work with the behavior directly. In a domain model, an interface is strongest when it describes a capability that a model can rely on. We're not creating interfaces because we like to create extra layers. Some people do, but that's not what we're after here. We are creating an interface when the model needs to depend on the behavior without caring about the exact concrete class since, indeed, we'll be working with the interface again. Let's make that difference visible in code. In this demo, I want to compare the data-shaped version with the capability version. And what I want you to watch for is not the interface keywords. Watch what the interface makes the caller do. So I'll add a small design example again with a data-shaped interface and a capability-shaped interface. For that, I will bring in a file called `CapabilityInterfaceExamples` under the `DesignExamples` folder. There are two different ideas in this file. The first interface is the `IDiscountData`. And as you can see, it exposes a percentage property. So it tells us basically what data is available, and that can be useful in some code. But notice what it does not do. It does not apply a discount. It does not contain the behavior. It only gives another piece of code the value it needs to do work somewhere else. Then, I added this percentage data discount class here now in the same file. It is just there so we can instantiate an example and later on make the comparison. Then, over here, I have the `IDiscountCapability`. That interface exposes the `ApplyDiscount`. So instead of saying here is a percentage, it says this object can apply a discount. That's a very different design approach. And then, the `PercentageDiscountCapability` class implements that behavior. It stores the percentage internally and applies the discount itself. So one interface exposes data, and the other one exposes behavior. Let's use that and look at the difference. What we see here now is that the data shaped interface forces the caller to do more work. We get the percentage from discount data, but then `Program.cs` to calculate the discount amount and subtract it from the total. That means that the behavior has now moved to the caller. So yes, we've used an interface, but we did not really improve the responsibility placement very much. Then, we do the same thing, but now through the `IDiscountCapability`. The caller does not ask for the percentage. He just asks the object to apply the discount. So the behavior stays with the object that owns the discount rule. So



really, the design meaning is different. The data-shaped interface tells the caller what values exist. The capability interface gives the caller useful behavior.

Designing a capability interface

Now, let's design a capability interface in this next part. Say we are building out the discount object before we start modeling every detail of the discount object, and let's ask a much simpler question. What does the checkout flow actually need from a discount? In this case, it needs something that can take a total and return a discounted total. That is useful behavior. So that is what our interface should contain. And yes, that's a very simple way to approach things. If you look at this interface, it's very easy to understand what it does. It's a contract that does not describe every possible thing a discount object might know. It only describes the capability that the caller needs. The surface area, therefore, is small, and that's a good sign because the interface is focused. Now we need a class that fulfills that contract. A NoDiscountPolicy is the one we'll add here, and that may sound a bit strange at first. Because of the word no discount, it might sound like nothing is happening. But it is actually useful to model it this way. It means that the no discount case still fits the same capability. It simply returns the same total unchanged. So instead of special casing no discount in a lot of places throughout the application, we can still work with an object that implements the same interface, and then another class can implement the same interface differently. Here we have now the LoyaltyDiscountPolicy. It still has the same method, ApplyDiscount. But this implementation actually calculates a discount and returns the reduced total. So we have two different classes with the same capability but different behavior, and that's exactly where interfaces are useful. The rest of the code can just say apply a discount, and the concrete object decides how that happens. From what we've looked at here, I see a few practical design rules. A strong interface is focused. It has one clear capability, a small surface area, and it's easy to understand and implement. A weaker interface often starts collecting too many members. Often it also mixes responsibilities. And in that case, it often becomes harder to name well because of the mixed responsibilities. That is actually a warning sign. If an interface feels vague, it might be doing too much. As said, the goal is not to wrap every class in an interface. That's a trap, in fact, we want to avoid. More interfaces do not automatically mean better design. Sometimes they just mean more files and more indirection, and that's not what you want. Instead, the goal is to introduce an interface where the capability matters to the model. In this demo, I want to connect IDiscountPolicy to the actual checkout flow, so we're no longer looking at an isolated interface example. We're looking at how the model can use this capability. The checkout flow actually needs a discount capability, so this is a good place to introduce that IDiscountPolicy now for real. So, here is the interface again. I've said it before. It is intentionally small. It has one method and one responsibility. ApplyDiscount receives a total before discount and returns a



total after discount. Now, I will add two implementations, and NoDiscountPolicy will be the first one. Here is NoDiscountPolicy. It may look a bit strange at first because, of course, no discount sounds like it's doing nothing. But that's exactly why it's useful. It lets the no discount case still fit in the same capability. So instead of having a special case null or a special case no discount everywhere, we can now represent it as an object that does implement the IDiscountPolicy and simply returns the total unchanged. Now let's look at the loyalty version. Here we see an updated version of the LoyaltyDiscountPolicy we already created earlier. This version is definitely a bit more realistic. Now the discount percentage is passed into the constructor. The constructor checks that the percentage is between 0 and 100, and then the ApplyDiscount uses that percentage to calculate the reduced total. So again, the caller does not need to know how a loyalty discount is calculated. It only needs the capability to apply a discount. Next, I will update the order so it can also use this interface. Most of the order is familiar from the previous demos, so I want to focus here on the new overload here near the bottom of the file. We now have a calculated total with an IDiscountPolicy parameter. That means that the order can calculate its normal total first and then ask the discount policy to apply the discount. But notice what the order does not do. It does not check whether the discount is a loyalty discount or no discount or a seasonal discount. It does not branch on the discount type. It only depends on the interface capability of applying a discount. Also notice we're still guarding against a non-discount policy. Now I'm going to update the checkout flow as well. So I'll need to add a few updates here. First, let's make a change to checkout summary. The summary now separates total before discount and total after discount. That'll make the result clear for our demo because we can see what the discount did. Now, let's focus on the CheckoutApplicationService again. I've made some changes, and the most important one here is the method signature of the CheckoutShipping. It now accepts an IDiscountPolicy parameter with a question mark, and that means that the parameter can be null. So the caller can provide a discount policy, but does not have to do so. Inside the method, we calculate the total before discount. Then we calculate the total after discount. And if discount policy is null, the total stays unchanged. Otherwise, the order calculates the discounted total through that policy. This is now where the interface is useful. The service does not need to know which concrete discount class it received. It just knows it has something that can apply a discount. The same idea is also applied to the CheckoutPickup, so both checkout parts can use the same capability. Now, let's go to our program with another checkout scenario. What we now see here is that the checkout code can pass in different discount policies. This one call here passes in the NoDiscountPolicy, and the other call passes in the LoyaltyDiscountPolicy. The service method is the same. The parameter type is the same. But the concrete object differs, and that is really the win from a design point of view. The checkout flow does not become full of discount-specific branches. It just relies on the IDiscountPolicy and lets the policy object provide the behavior.



Implementing multiple capabilities

Let's now look at implementing multiple capabilities next. A class is not limited to just one interface. That's an important difference compared to inheritance. With inheritance, a class has one base class chain. But with interfaces, a class can say, I support this capability, and I also support this other capability. That is useful because real objects can indeed play more than one role sometimes. And interfaces let us model these roles without pretending they all belong in the same inheritance hierarchy. Here we see the syntax for that. The class lists multiple interfaces after the colon separated by commas. That means the class is promising to fulfill all those contracts. So if it implements the `IDiscountPolicy`, it must be able to apply a discount. If it also implements the `IDiscountDescriber`, it must also be able to describe the discount well. So one class, multiple roles. That is why separate focused interfaces are, in fact, useful. Applying a discount and describing a discount are related, but they are not the exact same responsibility. Some code may only care about applying the discount; other code may only care about showing a description. By keeping these capabilities separate, we let calls depend on what they actually need. That keeps the design cleaner, which is what we're after. This also, again, explains that interfaces and inheritance are far from the same thing. With interfaces, a class can participate in multiple roles. The roles stay small and explicit. There is no need to force a fake is-a hierarchy just because several classes share a capability. With inheritance, the class can just have one base class, and the relationship is more about the type identity, not just a capability. It can be right, but it is, as said, a stronger choice. Interfaces are lighter when we only need a shared behavior. So, back to Visual Studio. In this demo, I want to show you one class being used through multiple interfaces, through multiple capabilities, let's say. Technically, this is one of the big practical differences with inheritance. With the base class, a class belongs to a one-base class chain. But with interfaces, a class can participate in several roles if it supports several capabilities. Let me add a second interface called `IDiscountDescriber`. So `IDiscountDescriber` is just, again, another small capability. It does not apply a discount. It describes one. That is a different responsibility, so a different interface. Keeping that separate is useful. Some code may need to apply discounts, and other code may only need to display or log discount descriptions. Those calls should not depend on more behavior than they need. And now let's update the `LoyaltyDiscountPolicy` with this new interface added. Now we see that `LoyaltyDiscountPolicy` implements, in fact, two interfaces, so the `IDiscountPolicy` and now also the `IDiscountDescriber`. The interfaces are separated by a comma after the colon. That means that now the class must provide both capabilities. It must implement the `ApplyDiscount` and the `DescribeDiscount`. So the same object can be used in now two different roles. If a caller needs to apply a discount, it can treat the object as an `IDiscountPolicy`. But if the caller only needs a description, it can treat the same object as an `IDiscountDescriber`. Now let me bring in another implementation, the



SeasonalDiscountPolicy. The SeasonalDiscountPolicy also implements both interfaces, but the calculation is different. It subtracts a fixed amount instead of a percentage, and the description is also different. So now we have two classes with the same capabilities but different internal rules. Let's now use those separate capabilities from our program again. First, I create here a LoyaltyDiscountPolicy and a SeasonalDiscountPolicy. Then, I intentionally assign the loyalty object to two different interface variables. One variable is of type IDiscountPolicy. And through that variable, I can apply the discount. And the other variable is of type IDiscountDescriber. And now through that variable, I can describe the discount. Keep in mind it is the same underlying object, but the caller only sees the role it asked for. Then, I even create an array of IDiscountDescribers. That array can contain both the loyalty policy and the seasonal policy because both implement that interface. And then I loop over the describers and call the DescribeDiscount method. Let's quickly run this. The output assures the applied discount and then the descriptions.

Polymorphism through interfaces

Now we saw polymorphism in the context of abstraction, but also with interfaces, we can apply this. Let's take a look. Basically, this is the same general idea we saw in the previous module. But now, the shared type is an interface instead of a base class. So the caller can work with the IDiscountPolicy, while the actual object at runtime can be a NoDiscountPolicy, a LoyaltyDiscountPolicy, or a SeasonalDiscountPolicy. The important part here is that interfaces can also be used as the abstraction that callers depend on. So the concrete type will only be known at runtime. Here in the snippet, we can see that the method accepts an interface. That means that this method, the PrintDiscountResult, does not care which concrete discount policy it receives. It only cares that the object has the apply discount capability, which is what the contract offered. The method is not tied to one discount class. It is tied to the behavior it actually needs. Now, we can call that same method with different implementations. The shape of the call stays the same. We pass a label, a discount policy, and a total. But the behavior changes because the concrete object behind the interface is changing. So this is interface-based polymorphism in practice. We use the same method. But because of interfaces, we get different concrete behavior. At compile time, the caller knows it has an IDiscountPolicy, and that is the contract. That's what the compiler can check. But at runtime, the actual object may be a NoDiscountPolicy, a LoyaltyDiscountPolicy, or a SeasonalDiscountPolicy. That's polymorphism at work in our model. And the right implementation runs for the actual object that was passed in. So the interface gives us a stable way to call the behavior while still allowing different implementations behind it. So, let's look at interface polymorphism at the call site. This is, as said, the same as we saw with abstract base classes, but now the share type is an interface. I've updated the Program.cs, and I'll take you through the important parts here. There are a few



pieces, but the key idea is that the checkout flow and the helper method work both through interfaces. First, we create three discount policies, `NoDiscount`, `LoyaltyDiscount`, and `SeasonalDiscount`. These are, of course, different concrete classes, but they all implement the `IDiscountPolicy`. Then, the checkout service uses those policies in the shipping checkout flow. The service accepts an `IDiscountPolicy`, so the method signature does not change for the discount type. We just pass in a different object, and the behavior changes because the object implements `ApplyDiscount` differently. Then, I'm calling this `PrintDiscountResult` method a few times. You can find it here at the bottom of the file. Its parameter type is an `IDiscountPolicy`, so it does not care what the object is. It just calls the `ApplyDiscount`. That is polymorphism through an interface. At compile time, the method knows it has an `IDiscountPolicy`. And at runtime, the actual object decides how the discount is applied. Finally, I'm also using, again, the `IDiscountDescriber` that shows, well, the same idea. The call can depend only on the description capability when that is all it needs.

Interfaces vs. abstract classes

I may have used both interfaces and base and abstract classes now. But I want to take a moment here to compare them and to show which to use when. I suggest you use an interface when you want to model a capability. In other words, when the important question is what can this object do, then an interface is a good fit. The caller only needs the behavior, and the implementing types do not need to be part of the same inheritance hierarchy. That's what we saw with the `IDiscountPolicy`. Use an abstract base class when you have one shared domain concept. If the types really belong together under the same common idea and maybe some shared behavior or structure belongs in the base type, an abstract class can be a better model. That is what we did with delivery. `Pickup` and `shipping` are both delivery types. That is the real hierarchy in our domain model. In code, we can see this happening. The interface expresses a capability contract, `apply a discount`. The abstract class expresses a shared domain concept, `delivery`. You will need to choose the construct that best matches the relationship in the model when you're building out your own domain. Interfaces can help avoiding unnecessary hierarchy pressure. If unrelated classes can share capability, they do not need to be pushed in the same base class chain. And if one class supports multiple capabilities, interfaces make that possible. Abstract classes can still be right, but only when the types truly belong in one hierarchy. Interfaces give us flexibility when we only need shared behavior.

Built-in interfaces as design inspiration

In the final part of this module, I want to look briefly at some built-in interfaces in .NET as design inspiration really. Interfaces are really everywhere in .NET, and



that's for a reason. The framework uses them to model capabilities very clearly, just like we need to do in our own domain model. So, when you see interfaces like this, do not think about them as technical framework types. Think about what capability each is representing. For example, the `IEnumerable` of `T` means that something can be enumerated. `Comparable` means that something can be compared. `Disposable` means that something can be disposed. And `IFormattable` means that something can be formatted. Those interfaces do not describe the whole object. They describe one useful thing an object can do. That's the exact same idea we used for the `IDiscountPolicy`.

Using inheritance carefully and deliberately

Module introduction

Hello there, and welcome to this module where we are going to look at using inheritance in a careful and deliberate way. Just like the title of this module says, inheritance is powerful but also very easy to overuse, and that is the knowledge you'll walk away with after watching this module. So yes, we are adding inheritance more deliberately now. We've already looked at inheritance earlier in this course. We've used base classes, derived classes, and abstract classes. The question I want to answer here is where do we really need a type hierarchy in a given part of the model? That's the shift we're going to make here. We're no longer going to ask can I inherit from this class? We're going to ask instead does this inheritance relationship say something true and stable about the domain? Inheritance is not the default tool for reuse. We've, in fact, mentioned that already before, but it's important. It's a modeling decision about identity and substitutability. If we use inheritance just because two classes share a few members, we can easily create a hierarchy that, well, looks convenient at first but can become misleading later. To go with inheritance in the domain model, we need to see a few things in the model that we are working with. We need to recognize when a subtype is really a more specific version of a base concept. We also need to recognize when that relationship is stable enough to belong in the model. And finally, inheritance should improve clarity. We shouldn't introduce inheritance just because we want to reuse a bit of code. From a domain modeling perspective, if the hierarchy mostly exists to make the implementation easier, well, we should be cautious, and it might not be the right choice. Here we see a snippet from the current model. If a collection typed as `IEnumerable` in deliveries and inside that collection we can place delivery objects coming from both shipping and pickup orders, that works because both are really deliveries. So here, the hierarchy expresses a real and stable domain idea. Shipping delivery and pickup delivery are different, but they are still both delivery concepts. So in other words, it is okay to use inheritance when the subtype really is the base concept and will remain so as the model grows. It is not



enough that the relationship sounds okay today. It also needs to stay that way when more behavior, more rules, and more details get added later on. That is what separates a, we'll say, useful hierarchy from one that only looks good at first, and that's what I want to show you here in this module.

What inheritance really means in a domain

So, let's explore what inheritance really means in a domain. The classic shortcut for inheritance, which I confess I also taught you in the introduction course on object-oriented programming, is the is-a. But in a domain model, that has to mean more than these things look related. It is very easy to say a shipping delivery is a delivery. And in this case, it's actually true. But it's also easy to say similar things too quickly just because two classes have similar data or similar methods. To model a real domain, we're going to have to be stricter. The subtype should really be a more specific version of the base concept, not just related, but instead really a meaningful specialization. When using inheritance in a domain, we are really making, let's say, a strong claim. It says that the subtype is of the base concept. It says that the relation belongs to the domain itself. And it also says that the hierarchy should keep making sense as the model grows. So inheritance is not just a mechanical code relationship, like what we've treated it thus far when we talked about it. It's a statement about meaning, that is meaning in the domain model. And that is why we should be careful with it. We should be able to make that claim true. As said, a real good inheritance relationship is about stable identity. ShippingDelivery is still a delivery. PickupDelivery is still a delivery. That is very different from inventing subtypes only because they share some code or because one type has one extra optional feature. That's not what we should be doing when modeling a domain like we're doing now. Those may be real concerns, but they're not automatically new identities. Now, another important aspect in the context of inheritance is substitutability, quite a difficult one to pronounce, I must say. The practical idea is this. Code that expects the base type should still make sense when it receives any subtype. So it must be possible to substitute and still work in the same way. If stated a method expects a delivery and we pass in a pickup delivery, the code should still be meaningful. If a subtype, however, breaks the meaning of the base type, then the inheritance relationship is weak, not what we want. Here we'll see the idea in code. Both shipping and pickup delivery objects can be treated as delivery. The calling code can work with them through the base type, and that still makes sense. That's a good sign. It means the hierarchy is not just technically valid; it also behaves correctly from the perspective of the domain. Let's head to Visual Studio for a demo about this. What matters is how the calling code works with the different inheriting types. So I'm here in the Programs.cs, and I'm going to paste in another snippet with a very small focused scenario. What we see here is that we create two delivery objects through orders. So we're using the factory methods here again. One comes from shipping order, and the other one comes from pickup



order. But the connection is typed as an `IEnumerable` in `Delivery`. So the calling code is saying I do not need to know whether each item is a shipping or pickup. I can treat them as delivery. That's already a good inheritance test. If the calling code becomes, let's say, weird when it receives one of the subtypes, then the hierarchy is probably weak as well. But here, it still reads well. We can loop over deliveries. We can print the display name and ask each delivery to calculate its shipping cost. Notice, again, this uses the `IEnumerable` entity that is one of those built-in interfaces we talked about in the previous module. So the method does not need to care about the exact collection type either; it just needs something it can enumerate. So the important inheritance point is this. Both concrete objects can stand in for delivery without breaking the meaning of the code.

Valid inheritance: Stable identity

I've mentioned stable identity in the inheritance tree earlier. Let's dive in a bit deeper. `Delivery` is a stable domain concept. It is not just a place where we happened to put shared code. The business really has the idea of delivery, and pickup delivery and shipping delivery are both meaningful versions of that concept. That is why the hierarchy is meaningful. It starts from the domain, not from the desire to avoid duplicated code. That would be a purely technical reason that we have moved away from. Here we see the base types of delivery again. The important modeling design point here is that the delivery represents a shared concept. It defines what all delivery types must provide, a display name and the ability to calculate shipping costs. So the base type is not random here. It captures what is common to the delivery concept. Here we can see the concrete subtypes, `PickupDelivery` and `ShippingDelivery`, and they both specialize the concept without breaking it. They provide different behavior, but they still remain deliveries. This is exactly the kind of inheritance relationship we want. The behavior can vary, but the identity still holds. So the hierarchy is helping the model speak clearly. We could say that to be a valid hierarchy from a domain modeling point of view, three concepts must remain. We have a stable base concept of delivery, we have stable subtypes, and the meaning of the call side stays the same. When these three are checked, we have a valid hierarchy. Also, a good model should stay honest as the model grows. If the shipping delivery gains a shipping address that does not break the idea, then it is still a delivery. If pickup delivery keeps different behavior, that also does not break the idea. So we can add domain detail, and the relationship still feels true. That is another good sign that the hierarchy is valid. Here we see why substitutability works in practice. This method works with an `IEnumerable` in delivery. It does not need to know whether each item is a pickup or a shipping delivery. It simply asks each delivery for its display name and shipping cost. And because each subtype behaves meaningfully as a delivery, the method still reads cleanly and correctly. That is inheritance doing its thing. In the next demo, let's see that the hierarchy is real



and useful. So what I want to look at here is to evaluate why this inheritance relationship is valid. What we see here is an abstract base class. `Delivery` is not a concrete thing we create directly. It represents that shared domain concept we talked about earlier. The members on it are also important. Every delivery has a display name, and every delivery can calculate its own shipping cost. So the base type is not just the place where we dump ____ code. No, it expresses what all the delivery types must be able to provide. That's what is important. Then, we have the `PickupDelivery` type. `PickupDelivery` is a concrete delivery type. I've also made it sealed, so it is a finished concrete behavior in this model. It overrides the `DisplayName` and the `CalculateShippingCost`, but it still keeps the promise of delivery. It does not break the idea of delivery; it specializes it. Here is then `ShippingDelivery`. It's also a delivery, but with different details. It has a shipping address, validates that, and returns a shipping cost of 5. So the two subtypes differ in behavior and state, but they remain true to the same base concept. That is why this is a good hierarchy still. `PickupDelivery` and `ShippingDelivery` are not just related; they are meaningful versions of delivery. So inheritance is fine when it says something true about identity, but it becomes risky when it is only used because two classes share some code or because one class has an extra feature, which, as we can see here, isn't the case here.

When inheritance becomes a design smell

Now we've have seen the positive side, but let's now look at the other side as well. When inheritance might become a design smell, probably even more important to understand. Now the smell is not a we have inheritance that exists here. Inheritance itself is not a problem. The problem appears when the subtype is not really a more specific version of the base type. Like we saw already, that often happens when inheritance is used mainly for code reuse or for attaching a feature to something. Here is an example where this is the case. I have a hierarchy growing around product features. And as we can see in the snippet, we have `SeasonalProduct`, `ShippableProduct`, and `FeaturedProduct`, all inherit from `Product`. And at first glance, you might say that doesn't look terrible. They're all related to products after all, right? But the problem is that these names mostly describe features or situations. Seasonal, shippable, and featured are not necessarily stable product identities. They are more like extra details or roles a product can have. So if the subtype name describes a feature, a status, or a convenience detail, the hierarchy might be lying about the domain. That may sound a bit strong, I know, but it is a useful test. If the subtype is only there because one product has an extra discount or one product is part of a campaign, then maybe it should not be a subtype after all. Maybe that detail should be modeled in some other way. What we can do here is a sort of an identity test. A good subtype is still clearly the base concept in a stable way. A weak subtype adds one feature or state and then usually leads to more siblings for other features or states. And over time, the hierarchy stops feeling like real



specialization. It starts feeling like a drawer where we keep more and more labels. So, let's look at bad inheritance in practice in the next demo. So I'll create a separate file to show you this, and it'll be the `InheritanceExamples.cs`. In here, I'll bring in a product hierarchy that looks plausible at first but is weaker from a domain modeling point of view. Let me paste in the code first. What we see here is a base product console, this one here. And then I have three subtypes, the `SeasonalProduct`, the `ShippableProduct`, and the `FeaturedProduct`. At first glance, this may not look terrible. They are all related to this product, right? A `SeasonalProduct` is still a product, and a `ShippableProduct` is also still a product. And of course, the same goes for the `FeaturedProduct`. But that is exactly why the simple is-a test is often not enough. The issue is that these subtype names describe features or states and not really stable identities. `SeasonalProduct` is a temporary business concern, `FeaturedProduct` is a marketing state, and `ShippableProduct` sounds more like a capability or a shipping detail than really a fundamentally different product identity. This kind of hierarchy tends to grow badly. What happens when a product is both seasonal and featured? Do we create a featured seasonal product? What if it is also shippable? Do we then create a shippable featured seasonal product? That is the hierarchy telling us it is solving the wrong problem. So we have a print method that is then going to print out some information to the screen, and it will actually make it a bit more visible. Let me run this now. So when we print, we see the campaign name, the seasonal discount amount, and the shipping surcharge. So those are real details, but they're not good reasons to create product subtypes. If the subtype mainly describes a feature, a status, or an optional detail, inheritance is probably not the right model in choice. The hierarchy may be technically valid, but it is weak from a domain point of view.

Inheritance vs. composition revisited

Now we talked about composition being a good approach earlier. So let's contrast that now with inheritance. It's a good point to bring back also composition, but with a slightly different angle than before. Earlier, we used composition to model relationships. Here, we use it as an alternative when inheritance would stretch things too far. The inheritance version we have looked at makes the model heavier. The product is still a product. Marketing and shipping are extra details around it. And they matter, but they do not necessarily create new product types. So if we turn every extra detail into a subtype, the product hierarchy becomes heavier and heavier and less clear. It will be less of a match with the real-life situations with the business. That's not what we came here for. Here we see the composition-based alternative. `ProductPresentation` keeps the product as the core concept. Then, `MarketingDetails` and `ShippingDetails` become related objects. That is a much better fit because the product identity stays stable. We're not pretending that a product with marketing details is a fundamentally different kind of product. Those classes are regular classes as we can see here, so modeled



directly. So these concepts are no longer forced into fake subtype identities. It is really the power of composition here. It lets us represent additional concepts without distorting the inheritance hierarchy. So, why is composition safer in this case? Well, the product remains the core concept. Features stay separate, so separate classes, so that optional or changing concerns do not become subtypes. And the model stays more flexible because new related details do not make us consider rethinking the hierarchy. So composition keeps identity cleaner and leaves more room for change. So the room for change is important since the model will change over time quite often, typically in the beginning. So in general, if you're adding features, states, or optional details, composition is usually a better fit than inheritance. Not always, but usually. As always, it'll depend, but it's something to keep in mind when building out your model. So, time for the final demo in this module. Let's make the composition-based alternative visible in code. We'll see that we can still model marketing and shipping details clearly without creating new product subtypes. The product should stay the core product concept, but marketing and shipping details can become related objects. So I'll add composition-based types in that same example file here, and I'll remove the inheriting products. So, what we now see here is that these extra concepts have their own place. Like `ProductMarketingDetails`, for example, it represents now the marketing concern. It has a `campaignName` and validates that the name is not empty. `ProductShippingDetails` represents the shipping concern, and it has a `shippingSurcharge` and validates that that is not negative. Then we have the `ProductPresentation` that composes these concepts. It has a `Product`, optional `MarketingDetails`, and optional `ShippingDetails`. That is now the important shift. We're no longer pretending that the product with the campaign is a new type of product. The product stays a product, and the marketing details can be modeled as related information. That'll give us much more flexibility. A product can have marketing details. It can have shipping details. It can have both or neither. We do not need separate subtypes for each possible combination. Let's use that from the `Program.cs`. What we now see here is that we have a product and a product presentation that brings together marketing and shipping details. The product identity, that stays stable. Cherry pie is just a product in this example, but the campaign and the surcharge are extra related concepts. They are not new product identities. So we've now modeled things in a much more flexible way, and that is why composition is usually more safer for optional details, feature data, or state that can change over time.

Modeling value objects and immutability with records

Module introduction



Hi there and welcome to this next module where we are going to look at value objects and immutability and especially how records in C# can help us to model these concepts, well, better. Up to now, many of the concepts we have worked with were entities. They have identity, they have behavior, and they could change over time while still being the same thing. But not every important concept in the domain model works like that. Some concepts are not important because of who or what they are. Instead, they are important because of the value they represent. An entity is something we usually recognize by identity. An order is still the same order even if we add another line to it. And a customer is still the same customer even if their email address, for example, changes. It does have a lifecycle in the application and can, as said, it will change over time. A value, well, it's different. A value is normally defined by what it contains. If the content changes, we really do not have the same value anymore. We have another value. And in code, we typically treat them as immutable. More on that later in this module. If something in the model is truly of value, then we should not force it into the same design style we use for entities. Otherwise, we end up with objects that look object-oriented, but they really do not behave like the domain concept they are trying to, well, represent. Here you can already see the kind of distinction that I mean. Customer is still an entity. That makes sense because a customer is something we may track over time. But email address, your shipping address, and money, well, they are different. They are not really things we identify separately in the same way. They are meaningful, absolutely, but they are meaningful because of their content. And value-like concepts are very easy to model badly. One common approach is to leave them as loose primitives, so just as strings and decimals, everywhere. Another common approach is to create small classes for them, but then make those classes fully mutable. Both approaches will work technically, but they can also make the model weaker because the meaning of the value is not properly protected. That is where records come in. Records are not the entire story, and they're not magic. But they are useful in C# when we want to model value-like concepts. They help us with two things that matter a lot for value objects, that is immutability and value-based equality. And those two ideas are going to prove very useful in our domain modeling task.

Entities vs. values

So, time to dive in, and let me start with making the difference between entities and values clear. Knowing what exactly is what can help us, of course, choosing the right tool for the problem at hand. Then, we'll see how we can use that C# feature for these values. So as said in the introduction, an entity is something we track over time. The order can start empty, then get all lines added to it, then get a delivery detail, and then eventually be confirmed. During all of that, we still think of it as the same order. That is what identity really means. It lets us change the object internally while still being recognized as the same conceptual



thing. So, like I already said, entities have an identity. They have a lifecycle, and they are not interchangeable just because their data happens to look the same. Two objects that contain the same values of all the data fields are still different objects. Behind the scenes in C# and thus in .NET, they are indeed two different objects on the heap. The values, on the other hand, are different. They just represent content. If two values contain the same meaningful data, then we often want to treat them as equal. And if a value needs to change, we often create another value. Replaceability is really one of the most important clues here. If the shipping address changes, we usually do not think the same shipping address object has evolved over time. We just say use this other address instead. But if an order changes, we do not usually have a completely different order. We say that the same order was updated. That difference actually tells us a lot. In our model, order, customer, and product are good examples of entities we've seen that already. Money, EmailAddress, and ShippingAddress are good examples of values. This prevents us from designing every class in the same way. Different concepts, well, they need different design rules. Let's see this already in our application. What I want to show you here is not only what the types look like, but how they behave differently. An entity can keep its identity while its state changes. A value should feel more like a safe, meaningful piece of information that we can pass around without worrying that it will unexpectedly change or without us really noticing. An order that is an entity, a customer, same thing, but a shipping address is much more value-like. If two orders use the same address detail, that does not make them the same order. It just means they share the same value. Now to make that visible, I will add a small design example again. It will create two real orders and compare what is identity based and what is really just value data. So I'll add a file here, ValueObjectExamples, and I'll add some code, and I'll do that in the DesignExamples folder. So, what am I doing here? Well, first, I'm creating one customer object, and then I'm using that same customer to create two separate orders. Then, notice the shipping address. I'm using the same one. So even though both orders are created in a very similar way, they are still two different order instances. They are really two different objects in memory. Now, to check this, I'm using the ReferenceEquals down here. That method is new to you. But it is built in, and it checks where the two variables point to the exact same object instance in memory. In other words, it is not asking whether two objects contain the same data. No, it is asking whether they are literally the same thing in memory. So when I compare firstOrder and secondOrder, the result should be false because they are two distinct orders, although their values seem identical to us. Now, that fits the idea of an entity very well. Two orders do not become the same order just because they look similar. And then, I do the same kind of check, but now with customer. There, the result should be true because both orders refer to the exact same customer object that I created at the start. So we can already see an important distinction here, different orders by the same reused customer entity. And now, finally, I compare the shipping address values. At this point in the model, ShippingAddress is just still a string. So here, I'm simply checking whether



the address text matches. And if it does, it should also return true. So let's test this code from the Program.cs. So I'm just going to call this method from the Program.cs. All right, let's run this. So together these three checks show us the distinction we are after in this module. The two orders are different entities because each order has its own identity. The customer is the same reused entity in both orders. And the shipping address, that behaves much more like value data. We're not treating it here as something with its own tracked identity. What matters is just the address content itself. So some parts of a model are about identity over time; other parts are much more about the value they represent. And once we recognize that difference, well then we can start designing those concepts differently as well.

The hidden cost of mutable values

Now let's look at one of the biggest problems with all the modeled values, and it is immutability. A mutable value can be created once and changed later. That's pretty convenient at first. Technically, it's indeed easy. We can create a money object and update its amount, or we can create an address object and update a street, for example. But the question is not whether the C# allows it. The question is whether that behavior matches the kind of concept that we're modeling. For values, it often does not. The problem appears when the same object is being used in multiple places. Here we see a small example of a MutableMoney object that is reused by two ShipmentQuotes. That is already, well, a little bit dangerous. We may think we are just sharing the same value because both quotes happen to start with the same shipping cost. But because the value itself is immutable, we also share the ability to change it, a sort of shared state really. That is the subtle part. So now, if we change the amount on the shared object, both ShipmentQuotes will get this change. That is really the bug. We did not go to both quotes and update them. We only changed one object. But because both quotes refer to the same mutable value, the change leaks into both places. That is probably not what you want. The code themselves were not changed directly, but they still behave differently after the shared value is changed. That means we have created a hidden coupling. One object can affect another object through a value that looked harmless. And for a domain value, that's exactly the kind of behavior we usually want to avoid. So the solution, or at least how we should think about it, is this. A value should carry meaning. It does not carry hidden mutability. If it passes around a money value or an email address or a shipping address for that matter, we should be able to trust that value. It should not quietly change because some other part of the code still has a reference to it. And let's look at that mutable value bug in the demo. Now I want to show why mutable values can be a problem. This is one of those things that looks harmless at first because the code is very simple. We create an object. We change a property. The job is perfectly happy with that. But values often get passed around and reused. And if the value itself can be changed, then



different parts of the model can accidentally share not only the same data but also the same changeable object. So let me first extend the value object examples file with a `MutableMoney` type and a small shipment quote type. There are a few things to note here. First, `MutableMoney` is intentionally a class with a public `settable` amount. So after an object has been created, outside code can still change the amount. Secondly, a `ShippingQuote` then receives a `MutableMoney` object in its constructor and keeps it around. So a quote has a shipping cost. That sounds perfectly reasonable. But then, in this method here, I'm creating one `sharedShippingCost` object and pair the same object to two different `ShipmentQuotes`. That means that both quotes now point to the same `MutableMoney` instance. That's where we'll get the bug. When I later on change the `sharedShippingCost.Amount`, I'm not changing the `morningQuote` directly, and I'm not changing the `eveningQuote` directly, but both quotes will observe the new value because both of them still refer to the same mutable object. Let's now add an extra call in the `Programs.cs`. I'm going to call that method that shows us the bug here. Before the change, both quotes show a shipping cost of 5. Then we change the shared value to 7.5. And after the change, both quotes now show 7.5. That's the key thing. We did not ask both quotes to change, but the change leaked through the shared mutable value. That is why value objects are usually much safer when they are immutable. A value should carry meaning. It should not carry a hidden mutability that can surprise another part of the model. So if immutable values are the issue, then we should go to immutable values. If a value cannot change after it has been created, then sharing it is less dangerous. Multiple objects can share the same value, and we do not have to worry that one of them will modify it for everyone else. No surprises, in other words. That makes the model more predictable. Predictability is good. It will definitely be easier to follow how things are happening in our application. That's a plus.

Designing immutable value objects

So, let's now look at how we can design immutable value objects. A value object often gets reused. We've seen that if we have a shipping address, we may pass it into an order. Or if we have a money value, we may pass it into a product. So if those values can be changed from the outside, we have to be much more careful. If they are immutable, well, then the model becomes more solid. To be fully clear, let me explain immutability quickly. Immutability simply means that once that object has been created, it cannot be changed, just like the word basically says. So if we need a different amount of money, we need to create a new money value. If we need a different address, we need to create a new address value. You get the idea. That may sound like a bit extra work, but it actually makes a model easier to reason about and, in the end, the application safer. That's definitely worth it. This also means that the value object should start its life in a complete state. We do not want to create an empty money object



and then fill the amount in later. That would leave a window again where the value exists, but it's not really value yet. Instead, the required data is passed in when the object is created. The value is complete from the beginning, in other words. That's what we see here being done using a regular constructor. That will help with creating a valid value again. Now, I want to introduce another keyword here, another C# keyword, namely the `init` since it is relevant in this context. Here we see the `init` keyword being used inside a property, the way we would normally expect a `set`. An `init` property can be assigned while the object is being initialized, but it cannot be later on changed by outside code. So it gives us a nice middle ground here. We can still initialize the object clearly. But after the creation, the property is no longer assignable. That fits well for value objects. The difference between `set` and `init` is important. With `set`, outside code can assign a new value later. So the object remains open to change. With `init`, the assignment is limited to the initialization part. Once the object exists, outside code cannot just say, okay, let me change the amount now. That helps us keep the value more stable. Validation still matters, of course. Immutability, of course, does not mean that we accept any value and then freeze it. That would just give us an immutable invalid object, so not useful at all. Validation still needs to happen when the value is being created. Here we can see that if the amount is negative, the object should not be created successfully. Invalid values should never enter the model. I'm going to make that concrete by designing two trustworthy value objects. In this demo, the important part is to look at both sides together. We do not only want the value to be immutable, we also want it to be valid when it is created. We're going to add immutable value objects for the concepts that really behave like values in the model. I will start with money. That is the clearest one. Money should not just be a decimal value floating around the model. It is a domain concept. It should be created in a valid state. It should not accept negative amounts. And once it exists, outside code should not casually change it. So, I'll add a `Values` folder in the domain project and create `money.cs` and let me paste that in. There is, as you can see, quite a bit of code here. Let me take you through the important parts first. I'm using a sealed class for money. Sealed again means that other types are not allowed to inherit from money. For a small value concept like this, that is usually a good constraint. Secondly, the `amount` property uses `init` instead of `set`. That's important. `Init` means that the value can be assigned while the object is being created, but not changed later from the outside. So after a money object exists, other code cannot just assign a different amount to it. Notice that validation, of course, also still happens. If the amount is negative, we throw an exception. So immutable does not mean blindly accepting whatever value comes in. It still means that we validate during creation. And then, if the value is valid, it stays stable after that. Then I have these operator methods, so the rest of the model can still work with money in a very natural way. We can add two money values, we can subtract them, and we can multiply a money value by a quantity or a percentage. If the term operator overloading is new to you, well, this is simply a C# feature that lets a type define what operators such as plus, minus, the



asterisk should mean for that type. That is what these methods are doing. They make the expressions such as price plus shipping cost or unit price times quantity possible while still keeping the logic inside the money type itself. Now, let me add email address in the Values folder too. Here we have seen the same pattern but, of course, with a different domain meaning. EmailAddress wraps a string, but it is not just a string anymore in the domain. It has a name in the model, it has validation, and it stays stable after creation because of the init. Next, let me bring in ShippingAddress too. This one has some more data, but it is the same ID that applies. A ShippingAddress is not identified separately like an order. It is defined by its parts. So street, city, and postal code together define the value. So now I'll update my Program.cs and see how we can use this. So what we see here now in the usage example is that creating the values is still very straightforward. A money object gets an amount, an email address gets the actual string, and a shipping address gets the address parts. I also added a guardrail example here. Creating money with a negative amount is again not allowed. That matters, of course, because it shows that these are not just pretty wrappers around primitives. They're protecting the rules of the value. Let's run this to make sure that everything works. So here we see the output of the three new values and then the validation example. So now we have value objects that are complete when created, validated when created, and stable after creation. That is the combination we should have. But there is still one thing missing, and that is how equality should behave.

Equality and value semantics

Now, let's move on and look at equality and value semantics. This is the, let's say, second part of value object design. Immutability protects us from unexpected change. Value equality makes sure values compare in the same way the domain expects. And for value objects, the default class behavior is often not what we want. Let's take a look. If, let's say, two money objects represent about \$15, then from the domain point of view, they should normally be equal. We care about the meaning they carry. That is very different from entities where identity usually matters much more than matching data. In .NET and C#, a reference equality asks a very specific question. Are these two variables pointing at the exact same object in memory? For entities, that can make sense in some situations. But for values, it is typically not the right question. For value, we want to ask, do these two objects contain the same meaningful value? That is called value equality. Do they contain the same value? So, reference equality and value equality are really asking different questions. Reference equality is about object instances; value equality is about their content. A value object should not usually behave as if its memory identity is the important part. It is the content that is the important part. That is not how C# works with regular objects. More on that in just a second. Here we see code that is comparing two different money objects that represent the same amount, so the object the same amount and the unit



price from the object in the array using the default here in C# using two equal signs. For a normal class, unless we implement equality ourselves, these will not compare the way we want. The runtime may treat them as different because they are two different instances. That is the normal way in C#. But for the value object, we want equality to compare the amounts. Let's look at equality, but in the way that the domain thinks about it in the next demo. So now that we have immutable value objects as ordinary classes, let's look at equality. That is the other big part of value object design. With a normal class, two different objects usually compare by reference unless we implement equality ourselves. In other words, C# asks whether both variables point to the same object in memory. And for entities, that can make sense. But for values, it's not the right question as we saw. So for money, for example, if two objects both represent 15, I want them to compare as equal. They're not the same object, but they represent the same value. So let me show you what happens right now with these class-based value objects. I've added a few lines here in the Programs.cs. First, price and sameAmount are two different money objects. I created them separately, but they represent the same amount. So domain-wise, we would like them to be equal. I then add both to a HashSet. In .NET, a HashSet keeps only unique values, and that means it depends on the equality as well. If equality is wrong for value objects like these, collections like these will also behave in surprising ways. Finally, I do the same kind of check with ShippingAddress. I create another address with the same street, city, and postal code. Again, these are two different objects, but they represent the same address content. Let's run this and see the result. This is now the important part. With regular classes, these comparisons do not yet behave in the way we want for value objects. The two money objects are treated as different objects. The HashSet is keeping both, and the two addresses are treated different as well. It is not a bug in our code, and it is also not C# acting strange. It is simply the default behavior of classes in .NET. So this is really the rule of thumb here. A value object is equal because of what it contains, not because it is the same object instance. That is a big one. And since we need to think about how we design these types, it affects equality, mutation, construction, and more.

Introducing records for value objects

Now we have seen the need for immutable value-like concepts that compare content. With this in mind, let's look at records. Records are a C# feature that brings support for the kind of behavior value objects often need. That is, they support a value-based equality by default. And they work well with immutable style properties. They give us a clean way to express that a type is more value-like than entity-like because of the new keyword. A record is still a C# type. In C#, behind the scenes, it gets converted to a class even. So we should not treat it as something completely separate from the rest of object-oriented design, but records are designed to work very well for value like modeling. For concepts like money, email address, and shipping address, that is exactly what we need, a



small, meaningful type that behaves like a value. It supports working like a value much better than a regular class. Here we see a record for `EmailAddress`. It looks really a lot like a class, except, of course, for the use of the `record` keyword instead of `class`. Inside, we have a property and a constructor. The difference is that by making this a record, we get value-oriented behavior from C#. And with `init`, the value can be assigned during creation but not freely changed afterwards. That is a plus. Here's another example, the `ShippingAddress` record. And we're using the property-based record form. Again, it looks very close to what a normal class looks like. We can see properties clearly, and we can now see how the constructor fills them up. Also, notice I'm using `sealed` here too, which means we're not allowing other types to inhabit from this record. For simple value objects, that is often a good constraint. C# also supports something called positional records, which is what you see here in the snippet on the right. That is the very compact form where the main data appears directly in the type declaration instead of inside the constructor. These are effectively parameters you see here. The syntax can be very useful for simple values where there is not much validation or extra behavior. Using both these record types is, well, nothing special. The syntax is the same as when using a regular class, as we can see here. Now, and this is crucial, once created, we can't just change the value. The value remains stable. That is because using `init` in the properties, we cannot just change those values later on. Once we use records, we get value equality checks automatically. As we already discussed with a normal class, the two objects containing the same data do not automatically compare as equal. By default, the class comparison is usually reference-based. With the record, C# gives us content-based equality, so two money records with the same amount compared as equal, which is exactly what we want for the value. So, records bring immutability and value semantics together very nicely. Let's now apply records in a model. In the demo, we will look at value-like concepts and convert them into records where it makes sense. Now I'm going to use those value objects in the actual model. I'm not going to do this refactor manually. I'm just going to show you the end result. Now, probably the most important part of that end state is that `Money`, `EmailAddress`, and `ShippingAddress` are now all records rather than ordinary classes. We've covered them all in the slides. Records give us value-based equality by default, which is what these concepts need. So up until now, `Money` was an ordinary class. But here now in the end state, it is a record because records are a much better fit for value-style semantics. Now, I've applied the change throughout the model. Let me show you, and let's start with the customer. Now, importantly, `Customer` is still a class. We're not turning `Customer` into a record because `Customer` is still an entity. It has an identity and a lifecycle, but the email is no longer a string. It is now an `EmailAddress`. That means that the customer no longer needs to know all the rules for what makes an email address acceptable. It just requires an `EmailAddress` object. The `EmailAddress` itself carries all those rules. Next, let's look at the product. Again, `Product` is still a class. It's an entity, but `UnitPrice` now becomes `Money`, and that is important because price is not just a decimal anymore. The price cannot accidentally



receive a negative decimal because of this rule here. And we also don't have product containing that validation anymore. Money now owns the money rule, and product owns product behavior, and it also continues in order. Here we see that shipping creation now takes in a ShippingAddress, not a string anymore. So the order creation method now says clearly what it needs. It doesn't need a string; it needs a shipping address. Secondly, the pricing methods now return money. That's also a better domain result. A subtitle and a total are money values, so the code now says that directly. The delivery classes have changed as well. The shipping delivery does not hold a raw address string anymore; it holds a shipping address value. And your shipping cost is returned as money, not a decimal. Now I'm going to look at the Program.cs here where I'm using this refactored model. What we see here is that this code still reads very naturally. We create products with money values. We create a Customer with an EmailAddress and a ShippingOrder with a ShippingAddress. The model now communicates more to its types. If a method asks for a shipping address, I know what kind of values belong there. If a property returns money, I know it is not just a decimal anymore.

Updating immutable values with "with" expressions

There is one more important part of working with immutable values, and that is updating them. And that may sound a bit strange at first. If something is immutable, how do we then update it? The answer is that we do not update the existing object. We create a new value based on the old one. That is where the with expression becomes useful. Sometimes we have a value that is almost right, but one part needs to be different. For example, an address where the street changes, but the city and the postal code remain the same. With immutable values, we should not mutate the original address. We create a modified copy. A with expression does exactly that. It takes an existing record, copies its current values, and then applies the changes we specify for the new copy. So in this example, the updated address gets a different street, but the original address is still the original address. So, this is not mutation with nice syntax. The original object stays unchanged. A new object is created with the modified value. That means we keep the safety of immutability while still having a convenient way to express small changes. And let's see that in the last demo of this module. So here we are in the Program.cs, and I've added a small updatedAddress example. We see here the use of the with keyword. It does not modify the shippingAddress. It creates a new shippingAddress value by copying the existing one and changing only the street in the new copy. Now, this is not mutation with nice syntax. The original value remains unchanged. UpdatedAddress points to a new value. So if the address changes, we do not mutate the old address value. We use a new one. Let's verify that. So we can see the original address and the updated address side by side. That is the balance that we get from records. The value is table after creation, which makes it safe, but creating a slightly different value is still easy to do.



When records are (and aren't) a good fit

Now to end this module, I want to give you a clear understanding on where records are a good fit and where they are not. That is still a modeling decision. We choose records when the concept behaves like a value. So Money, EmailAddress, and ShippingAddress are good fits. They are definitely defined by their content, and they are easy to replace with another value, and immutability and value-based equality makes sense for them. So for these kinds of domain concepts, records are a good fit. Records can also be useful for DTO-like behavior. DTO is short for data transfer objects. If you have a type whose main job is to just carry data from one place to another and where content-based comparison and immutability makes sense, there a record might be a good option too. Entities are usually a poor fit for records. An order or a customer has an identity. It has a lifecycle. It can change over time and still be the same thing. We've seen that. If you make that kind of concept, compare only by content, we may lose the very thing that makes it an entity. So for entities, normal classes are still usually the better fit. So the decision is not difficult if we simply ask the right question. Is the concept defined by the content? Should equality follow the data inside of it? And does immutability make sense? Well, if yes, then a record is probably the best fit. But if the concept has an identity, a lifecycle, evolving state, then it is probably an entity and not a value.

Keeping domain models maintainable over time

Module introduction

Hello there, and welcome to this final module where we are going to be looking at maintainability over time. We've built up quite a bit. We started with very basic objects in classes. And step by step, we turned that into a stronger domain model. But now comes the real question. Will that model still help us when the next change arrives? That's really what I want to focus on here. A domain model may look clean today. The names may look good, the classes may be nicely organized, and the current scenario may work. But that does not prove all that much yet, sadly. The real test comes when the business asks for something new. If a small change immediately spreads everywhere, then the model is more fragile than it looks. So for me, success is not that the model is somehow perfect. That is not realistic anyway. Success is that the new behavior has, let's say, a natural place to go. Responsibilities are clear, and the decisions that we made earlier keep paying off instead of becoming things we now need to fight against. That is what maintainability is all about here. Here we see a small example of that. Say that local delivery has been added to the checkout row, but the orchestration code still reads quite clearly. We're not seeing a huge block of delivery checks or pricing rules here. That's the important part. The model did not prevent the



change. It did make the change fit into the existing structure. Where we need to land is this. A strong domain model is not just expressive; it is resilient as change comes in. In other words, the model can keep its shape even when new requirements will appear. Not forever, possibly, but much better than a model where every rule is just scattered wherever it was the easiest to type at the time. And that's really what we all often see and what I want to prevent from happening.

Knowing where behavior belongs

So, perhaps the most important question is where does behavior belong. You may be thinking, we asked that question already many times in this course. But now at the end, we can look at it with the full model in mind. Keep in mind that if behavior ends up in the wrong place, change becomes harder very quickly. Believe me, in a real typical application, we often see different kinds of logic getting mixed together. There is domain logic, which is really about business rules and business behavior. Then there is application logic, which coordinates a use case. There is infrastructure concerns like database access, storage, external systems, messaging, and so on. Those things work closely together, but they should not all become one big pile of code. It's probably quite logical, but I'm still going to say it here anyway. Domain logic belongs in the domain model. That being said, if a rule is about shipping cost or quantity or discounts or whether an order is valid, then the first question should be which domain concept owns this? We're not looking for a place to put code; we're looking for the place where that behavior makes sense. You may see some application logic doing what application logic should do really. That is the checkout service, which coordinates the flow. It creates the right kind of order, adds the requested lines, and builds a summary. That's typically what application logic needs to do. But notice what it does not do. It does not calculate the delivery rules itself. It does not decide the discount rules itself. It tells the domain model what needs to happen, and then the domain model does the meaningful work. So in other words, it is delegating the work a bit like a manager that asks someone to do something as it doesn't know how to do it but knows that it needs to be done by someone, and that is the domain model that knows how to do it. Infrastructure is another building block. Now, we're not going to go deep in infrastructure in this course, but the area, of course, still matters. Databases, external APIs, files, messaging, all those things are important, but they are technical integration concerns. If that kind of detail starts appearing in the domain model, the business meaning becomes harder to see. That's something you really want to avoid. So, let's look at the checkout flow with these boundaries in mind in the code. So what I want to do is not first add code. I want to look at the existing model, and we'll ask a maintainability question. Where does behavior live now? So here we are in the application service. What we see here is application logic. This method here coordinates the checkout use case. It creates the right kind of order, adds the



requested lines, calculates the total, and returns a summary. That's perfectly fine. That is the kind of thing an application service can do. Perhaps more important, what it does not do. It does not decide what shipping means. It does not inspect delivery types with if statements. And it also does not calculate the discounts itself. It is not a passthrough. It is orchestration. It tells the domain what needs to happen. Let's look at order next. Here we are in the order, and this is closer to the domain behavior. The order, for example, knows how to calculate its subtotal, its total. And if a discount policy is provided, it asks to apply the discount. That is an important boundary. The application service does not need to know how the order totals are calculated. It just calls to order for that. And the order does not need to know the concrete discount implementation. It depends on the discount capability. Let's take a look at delivery and discount. What we see here is that delivery-specific behavior and discount-specific behavior each have their own place. Delivery behavior lives behind the delivery abstraction. Discount behavior lives behind the discount interface. So, we see clear boundaries. The application service stays in an orchestration role. The domain model owns the meaningful behavior, and the main responsibilities have a good place to live in. That's a good separation. That is definitely a good place to be in.

Guardrails that protect your domain model

Guardrails are important to keep the model valid. A model does not stay clean just because it started clean. That would be nice, but that is not how real projects work. New features arrive, people are under time pressure, and it becomes very tempting to add logic where it is most convenient at the time. That is why a maintainable model needs protection, not just good intentions that we may have come up with initially. If the design leaves every door open, eventually, someone will walk through the wrong one. And honestly, that someone might just be us under pressure 3 months later. What we need is guardrails. They help us keep behavior from drifting back into weak places. Typically, about what they are. Throughout the course, we have, in fact, added several of them. Encapsulation kept behavior close to the state it depends on. Controlled creation makes it harder to create weak objects. Composition and interfaces help us avoid unnecessary inheritance pressure. And we have seen immutable values. They help us avoid hidden side effects. Together, all these make the model stronger. And we can see this code was in concrete parts of our model. A few examples of how we've done this. Order controls how lines are added and changed. Delivery keeps delivery behavior behind the right abstraction really. `IDiscountPolicy` lets callers depend on the discount capability, not on one fixed discount class. And money is stable and compares by value. These are just some of the choices we've made throughout this course. Absolutely, don't see guardrails as, let's say, a bad thing. It is, in fact, quite the opposite. A good guardrail does not make the model harder to use. If it does, developers will try to get around it. In fact, a good guardrail makes correct use feel natural and



dangerous usage harder.

Evolving a domain model without breaking it

Now it is one thing to set things up correctly; it's another one to change to evolve over time. Changes and change requests will come in, and we'll need to handle those in the model. As said, the proof is in the pudding. Did we do a good job? Say that we get in the following change request. We need to add that local delivery. We also need free shipping above a subtotal threshold. And we want the checkout flow to remain readable. That is a normal business change, not huge, but enough to reveal whether our model has useful structure. In a weaker model, this kind of change will tend to spread, making the code less maintainable. Shipping rules might appear in the checkout service. More conditionals might appear in several places even. And existing code might need broader rewrites than we expected. That is when a small feature starts feeling much larger than it actually should. The model is not helping us anymore; it is making the change more expensive. And imagine how bad things would get if we're getting in a much bigger change request. Now, we are in a good place. Our current model gives us something better to work with. We already have a delivery abstraction, so adding a new type here is pretty easy. We already have thin application orchestration. We already have value objects for things like money and addresses. So the new requirement does not arrive in an empty field. It arrives in a model that already has places for these IDs. That'll be a nice change for you as a developer working on this since you can easily fit things in and put them in the right place easily. And let's look at this change request in the code. You'll see how it blends in easily. We are asked to bring in local delivery, and delivery should have free shipping above a subtotal threshold. That is the kind of change that tells us whether our model is maintainable or not. In a weak model, we would immediately go to the checkout service and add a few if statements. If delivery is local, do this. If the subtotal is above 50, do that. And that would work, but it would also push the delivery rules in the orchestration code. Now, we have a better place, and that is delivery abstraction. So I start here in delivery and update the abstraction. There are two changes here. First, `CalculateShippingCost` now receives the subtotal. That is needed because the delivery rule depends on the order subtotal, so passing in the information the delivery object needs. But the rule still belongs in the delivery. Secondly, I also added `ForLocalDelivery`. This keeps creation consistent with the order delivery types. We're not newing up local delivery all over the application. No, we keep that creation part inside this delivery abstraction. Now, because I changed the abstract method, the existing delivery types need to be updated as well. Let's first update `PickupDelivery`. `Pickup` receives a subtotal because the abstraction now requires that. But `Pickup` does not need to use it. It always returns 0. Next, let me update `ShippingDelivery`. What we see here is that shipping now also has a threshold rule. If the subtotal is at least 50, shipping is



free. Otherwise, it still costs 5. Again, notice where this rule lives. It lives in `ShippingDelivery`. The checkout service does not need to know about the threshold rule. Next, I will bring in the `LocalDelivery` class. This is now the new behavior of fitting into the existing abstraction. `LocalDelivery` is another kind of delivery. It has a display name, an address, and it knows how to calculate its own shipping cost. And the rule is also a bit different. Below the threshold, local delivery costs 2.5. At or above it, it's 3. So we have a variation, but it's owned by the delivery type that needs it. Next, I need to update order so it can create a local delivery order, and so it passes subtotal into the delivery calculation. There are a few important pieces here. `CreateLocalDeliveryOrder` adds a new valid creation path. It follows the same pattern that we already had for pickup and shipping. `ChangeToLocalDelivery` adds the behavior for changing an existing order to local delivery or, again, not exposing the delivery property for outside code to set directly. The order still controls its own delivery change. And then, `CalculateShippingCost` is new on order. The order calculates its subtotal and then asks the current delivery object to calculate shipping based on the subtotal. This is a small but important point. The order has the lines, so the order can calculate the subtotal. The delivery owns the delivery-specific pricing rule. Now we also need to update the application service so that local delivery is available as a use case while still keeping the whole orchestration layer thin. I've updated the existing `CheckoutShipping` and `CheckoutPickup`, and I've added the `CheckoutLocalDelivery`. They all follow the same pattern. Create the order, add the lines, and now, build the summary. The repeated summary building code I've now moved into this `BuildSummary` method. Indeed, remember that the application service should stay focused on orchestration. It's not calculating the threshold, not deciding what local delivery costs. It just coordinates the flow and asks the domain model for the results. Let's now finally update our `Program.cs` to prove the scenario. What we now see here is the actual change request being used. The first call shows the normal shipping below the threshold. The second call shows shipping above it where the shipping becomes free. And the third call shows local delivery below the threshold so that the local delivery tool is being used. And the fourth call proves that discounts still work on top of new delivery behavior. That is important, by the way. We did not break the existing discount abstraction while adding delivery behavior. Let me run this to verify. We see here in the output that local delivery has its own cost, and shipping can become free above the threshold. So we've now implemented a real change without having to rewrite or redo the whole model. It absorbed the change very easily.

Extension methods as a design tool

Another tool in our toolbox is extension methods, and I want to briefly look at these here. It's a small C# feature that can be useful. When we get in a new feature request, we might decide it's a good idea to create an extension method to add a new feature in. But this then comes up with an important design



question again. If we add the method from the outside, are we improving usability or are we moving real behavior away from the model? What are our extension methods? Well, they let us call a method as if it belonged to a type, even though the method is defined somewhere else in another class. So they extend an already existing type. It can be very convenient, but convenience is not the same thing as good design. So the question is not can we make this as an extension method? In C#, we often can. The better question is, should we do it? Here we see the basic syntax of an extension method. The method lives in a static class. That's a requirement. And the method should also be declared as a static. The first parameter should start with this, so this `Order order`. That this is what tells C# that this method can be called as if it were available on the `Order` object. So we're really now adding a method in a separate class on the `Order` class. So the syntax, well, it's pretty simple. The design decision, that's the more interesting part. Because when we call `order.ToOrderLabel`, that reads as if `ToOrderLabel` belongs to `order`. Can be nice that call side is clean, but the behavior is actually defined outside of the `Order` class. So we need to ask what kind of behavior is this? Is it just a helpful support method, or is it something that the `Order` should own itself? In this case, returning a label can be a reasonable support method. It helps with usability. It creates a readable piece of text. But it does not calculate discounts. It does not enforce order invariants, or it does not decide on shipping rules. So this is support behavior. It is useful, but it is not taking over the domain model's responsibility. But this is also where extension methods can become dangerous. If we start calculating discounts in an extension method or deciding shipping rules there or enforcing invariants from outside the model, then we have a problem. That would mean domain behavior is leaving the domain model again, and we've spent the whole course trying to avoid exactly that. So, the rule is pretty simple. If the method contains real business rules, it probably belongs back inside the domain model. Extension methods are fine as a support tool, but they should not become a back door where we hide domain logic because it was convenient. Let's quickly see in code where I would say extension methods are okay to use. Now I want to keep the extension method small, and I want the usage to be real. So what I'm going to add is a reusable label that an operations screen or packing overview could use. So I'm going to add an `ExtensionMethodsExample` class under the `DesignExamples` folder. An extension method has to live in a static class, and the method itself is static as well. And the special part is this first parameter, this `Order order`. This here tells C# that the method can be called as if it were defined on the `Order` itself. So `ToOrderLabel` is defined outside the `Order` class, but we can still call `order.ToOrderLabel`. Now, that is exactly where we need to be careful with extension methods. They read as if the behavior belongs to the type. So if the method starts calculating discounts, deciding shipping, or enforcing invariants, it is probably in the wrong place. So here it only creates a presentation-friendly label, so I'm comfortable keeping this outside of the entity itself. Now, let's use this on some orders from the `Program.cs`. So here we now see the extension method being used. So we're not inventing a new domain rule here. We are centralizing a little bit of the



repeated formatting that the caller may need in more than one place. So the value is indeed that the call side can ask for a ready-to-use order label. So let's run this and look at the result. So indeed, the output shows reusable labels for orders. That is the kind of extension method usage that I think is useful in a model like this, supportive behavior that improves usability but does not take ownership or does not contain any business behavior.

Course wrap-up

And that brings us to the end of this course. We set out with a clear goal, go further than the OO features given to us by C# and build us a domain model as it should be done, a clear and easy-to-understand model where things live in the right place and can be changed if needed. And we didn't stop at individual OO features. Yes, we looked at classes and objects and core OO thinking like polymorphism and interfaces. But then, we strengthened the model through richer behavior, controlled creation, composition, polymorphism, interfaces, and value objects. And now we end up with a model that can absorb change more safely than the first version could ever do. That's a lot better, as you've seen in this final module. So, when you design your own models, start with the domain concepts. Do not start with patterns. Don't start with clever abstractions and ask where the behavior belongs. Add abstraction where it improves the model and use language features in support of the design and not as the goal. I think a good model is never finished, or at least it'll demand that we keep looking at it. Practice recognizing domain concepts and boundaries, and don't be afraid to refactor toward behavior instead of accumulating conditionals. Watch for identity versus value and keep asking whether the model will still make sense after the next change. That question is simple but really useful. And with that, we have reached the end of this course. Thanks for watching.